LM-Tutorial

May 2, 2026

0.1 Language Modeling Tutorial

Prepared by Ye Kyaw Thu, Language Understanding Lab., Myanmar
For AI Fundamental Class
Last updated: 2 May 2026

[30]:
```
%cd /mnt/disk1/ye/exp/LM-Tutorial
```

```
/mnt/disk1/ye/exp/LM-Tutorial
```

[32]:
```
!ls
```

```
bk          general.log  lm_toy.test.txt   lm_toy.val.txt  transformer
data        general.txt  lm_toy.train.txt  lstm
domain.log  kenlm        lm_toy.txt        tmp_kenlm
```

[33]:
```
import torch
import os

# Set working directory to your fast NVMe drive
WORK_DIR = "/mnt/disk1/ye/exp/LM-Tutorial/"
os.makedirs(WORK_DIR, exist_ok=True)
os.chdir(WORK_DIR)

print(f"Working Directory: {os.getcwd()}")

# Verify GPU
print("\n--- GPU Status ---")
if torch.cuda.is_available():
    device = torch.device("cuda")
    gpu_name = torch.cuda.get_device_name(0)
    gpu_mem = torch.cuda.get_device_properties(0).total_memory / (1024**3)
    print(f"GPU Detected: {gpu_name}")
    print(f"VRAM: {gpu_mem:.2f} GB")
    # Enable TF32 for faster matrix math on Ampere architecture (RTX 3090)
    torch.backends.cuda.matmul.allow_tf32 = True
    torch.backends.cudnn.allow_tf32 = True
else:
    device = torch.device("cpu")
```

```
    print("WARNING: CUDA not found. Falling back to CPU.")

# Verify CPU cores
import multiprocessing
print(f"\n--- CPU Status ---")
print(f"CPU Cores available: {multiprocessing.cpu_count()}")
```

```
Working Directory: /mnt/disk1/ye/exp/LM-Tutorial

--- GPU Status ---
GPU Detected: NVIDIA GeForce RTX 3090 Ti
VRAM: 23.68 GB

--- CPU Status ---
CPU Cores available: 24
```

[34]:
```
import math
from pathlib import Path
from typing import List, Tuple

def split_lines_file(path: str | Path, train_ratio: float = 0.8, seed: int = 0):
    import random
    rng = random.Random(seed)
    lines = Path(path).read_text(encoding="utf-8", errors="ignore").splitlines()
    rng.shuffle(lines)
    n = len(lines)
    n_train = int(n * train_ratio)
    n_val = int(n * (1 - train_ratio) / 2)
    train, val, test = lines[:n_train], lines[n_train:n_train+n_val],␣
 ↪lines[n_train+n_val:]
    def write(name, rows):
        p = Path(path).parent / f"{Path(path).stem}.{name}.txt"
        p.write_text("\n".join(rows), encoding="utf-8")
        return str(p)
    return write("train", train), write("val", val), write("test", test)

def compute_ppl_and_entropy(nll_sum: float, n_tokens: int) -> Tuple[float,␣
 ↪float]:
    """NLL should be in NATS (natural log). Returns (Perplexity, Entropy_nats).
 ↪"""
    if n_tokens == 0: return float("inf"), float("inf")
    avg_nll = nll_sum / n_tokens
    ppl = math.exp(avg_nll)
    entropy = avg_nll # Entropy H = Cross-Entropy when model matches empirical␣
 ↪dist
    return ppl, entropy
```

```python
def log10_to_nats(log10_val: float) -> float:
    return log10_val * math.log(10)
```

[35]:
```python
import textwrap

# A slightly richer corpus so statistical smoothing has enough data to work with
corpus_text = textwrap.dedent("""\
I love NLP .
I love speech recognition .
I love language models .
Language models are fun .
Speech recognition needs language models .
NLP stands for natural language processing .
Deep learning improved NLP significantly .
Transformers changed deep learning forever .
Recurrent neural networks were used for sequence modeling .
Long short term memory networks solved the vanishing gradient problem .
Attention is all you need .
Retrieval augmented generation helps reduce hallucinations .
Language models assign probabilities to sequences of words .
Perplexity is a common metric for evaluating language models .
Cross entropy measures the difference between two probability distributions .
Smoothing techniques help handle unseen words in n-gram models .
Backoff models fall back to lower order n-grams when higher orders are missing .
Kneser Ney smoothing is the state of the art for statistical language models .
Neural language models use continuous representations of words .
GPT models are autoregressive language models .
BERT is a masked language model .
Fine tuning adapts pretrained models to specific downstream tasks .
Large language models require massive amounts of training data .
Scaling laws dictate the relationship between model size and performance .
""".strip())

corpus_file = Path("lm_toy.txt")
corpus_file.write_text(corpus_text, encoding="utf-8")

# Split 80/10/10
train_f, val_f, test_f = split_lines_file(corpus_file)
print(f"Train lines: {len(Path(train_f).read_text().splitlines())}")
print(f"Test lines: {len(Path(test_f).read_text().splitlines())}")
```

```
Train lines: 19
Test lines: 3
```

[36]:
```python
!ls
```

```
bk          general.log  lm_toy.test.txt   lm_toy.val.txt  transformer
data        general.txt  lm_toy.train.txt  lstm
```

```
domain.log  kenlm        lm_toy.txt        tmp_kenlm
```

[41]:
```
!wc *.txt
```

```
   2   24  151 lm_toy.test.txt
  18  151  977 lm_toy.train.txt
  23  198 1270 lm_toy.txt
   1   23  140 lm_toy.val.txt
  44  396 2538 total
```

[37]:
```
import kenlm

arpa_file = "lm_toy_3gram.arpa"
tmp_dir = f"{WORK_DIR}/tmp_kenlm"
!mkdir -p {tmp_dir}

# FIX 1: Added '--discount_fallback' so it won't crash on small datasets
# FIX 2: Added '2>{arpa_file}.err' to catch any remaining errors in a file
!cat {train_f} | lmplz -o 3 -S 16G -T {tmp_dir} --discount_fallback >␣
 ↪{arpa_file} 2>{arpa_file}.err

# Safety check: Make sure the file isn't empty before loading!
if Path(arpa_file).stat().st_size == 0:
    print("ERROR: ARPA file is empty! Check the error log:")
    print(Path(f"{arpa_file}.err").read_text())
else:
    model_kenlm = kenlm.LanguageModel(arpa_file)
    print(f"Successfully loaded {model_kenlm.order}-gram model.")

    def eval_kenlm(path: str):
        sum_log10, n_tokens = 0.0, 0
        for line in Path(path).read_text(encoding="utf-8").splitlines():
            if not line.strip(): continue
            for prob, length, oov in model_kenlm.full_scores(line):
                sum_log10 += prob
                n_tokens += 1
        sum_nats = log10_to_nats(sum_log10)
        ppl, entropy = compute_ppl_and_entropy(-sum_nats, n_tokens)
        return ppl, entropy

    ppl_kenlm, ent_kenlm = eval_kenlm(test_f)
    print(f"KenLM Test -> PPL: {ppl_kenlm:.2f}, Entropy: {ent_kenlm:.2f} nats")
```

```
Successfully loaded 3-gram model.
KenLM Test -> PPL: 25.80, Entropy: 3.25 nats
```

```
Loading the LM will be faster if you build a binary file.
Reading /mnt/disk1/ye/exp/LM-Tutorial/lm_toy_3gram.arpa
----5---10---15---20---25---30---35---40---45---50---55---60---65---70---75---80
```

```
---85---90---95--100
********************************************************************************
********************
```

[37]:
```
!ls
```

```
lm_toy_3gram.arpa      lm_toy.test.txt   lm_toy.txt      tmp_kenlm
lm_toy_3gram.arpa.err  lm_toy.train.txt  lm_toy.val.txt
```

0.2 Learn ARPA Format

Kenlm နဲ့ ဆောက်ပြီးထွက်လာတဲ့ Language Model ဖိုင်ကို လေ့လာကြရအောင်။

[38]:
```
!cat ./lm_toy_3gram.arpa
```

```
\data\
ngram 1=102
ngram 2=140
ngram 3=143

\1-grams:
-2.14211        <unk>   0
0       <s>     -0.30103
-2.100283       </s>    0
-2.100283       Cross   -0.30103
-2.100283       entropy -0.30103
-2.100283       measures        -0.30103
-1.6676633      the     -0.30103
-2.100283       difference      -0.30103
-1.7268959      between -0.30103
-2.100283       two     -0.30103
-2.100283       probability     -0.30103
-2.100283       distributions   -0.30103
-1.0318743      .       -1
-2.100283       I       -0.30103
-2.100283       love    -0.30103
-2.14211        NLP     -0.30103
-2.100283       Kneser  -0.30103
-2.100283       Ney     -0.30103
-2.100283       smoothing       -0.30103
-2.14211        is      -0.30103
-2.100283       state   -0.30103
-2.14211        of      -0.30103
-2.100283       art     -0.30103
-1.7268959      for     -0.30103
-2.100283       statistical     -0.30103
-1.3673046      language        -0.50514996
-1.6676633      models  -0.30103
-2.100283       Transformers    -0.30103
```

```
-2.100283       changed -0.30103
-2.100283       deep    -0.30103
-1.7268959      learning        -0.30103
-2.100283       forever -0.30103
-2.100283       Deep    -0.30103
-2.100283       improved        -0.30103
-2.100283       significantly   -0.30103
-2.100283       Retrieval       -0.30103
-2.100283       augmented       -0.30103
-2.100283       generation      -0.30103
-2.100283       helps   -0.30103
-2.100283       reduce  -0.30103
-2.100283       hallucinations  -0.30103
-2.100283       Large   -0.30103
-2.100283       require -0.30103
-2.100283       massive -0.30103
-2.100283       amounts -0.30103
-2.100283       training        -0.30103
-2.100283       data    -0.30103
-2.100283       Fine    -0.30103
-2.100283       tuning  -0.30103
-2.100283       adapts  -0.30103
-2.100283       pretrained      -0.30103
-2.100283       to      -0.30103
-2.100283       specific        -0.30103
-2.100283       downstream      -0.30103
-2.100283       tasks   -0.30103
-2.100283       Attention       -0.30103
-2.100283       all     -0.30103
-2.100283       you     -0.30103
-2.100283       need    -0.30103
-2.100283       Speech  -0.30103
-2.100283       recognition     -0.30103
-2.100283       needs   -0.30103
-2.100283       BERT    -0.30103
-2.100283       a       -0.30103
-2.100283       masked  -0.30103
-1.7268959      model   -0.30103
-2.100283       Language        -0.30103
-2.100283       are     -0.30103
-2.100283       fun     -0.30103
-2.100283       GPT     -0.30103
-2.100283       autoregressive  -0.30103
-2.100283       stands  -0.30103
-2.100283       natural -0.30103
-2.100283       processing      -0.30103
-2.100283       Neural  -0.30103
-2.100283       use     -0.30103
```

```
-2.100283       continuous      -0.30103
-2.100283       representations -0.30103
-1.7268959      words   -0.30103
-2.100283       Long    -0.30103
-2.100283       short   -0.30103
-2.100283       term    -0.30103
-2.100283       memory  -0.30103
-2.100283       networks        -0.30103
-2.100283       solved  -0.30103
-2.100283       vanishing       -0.30103
-2.100283       gradient        -0.30103
-2.100283       problem -0.30103
-2.100283       Scaling -0.30103
-2.100283       laws    -0.30103
-2.100283       dictate -0.30103
-2.100283       relationship    -0.30103
-2.100283       size    -0.30103
-2.100283       and     -0.30103
-2.100283       performance     -0.30103
-2.100283       Smoothing       -0.30103
-2.100283       techniques      -0.30103
-2.100283       help    -0.30103
-2.100283       handle  -0.30103
-2.100283       unseen  -0.30103
-2.100283       in      -0.30103
-2.100283       n-gram  -0.30103

\2-grams:
-0.045374617    . </s>  0
-1.5187746      <s> Cross       -0.30103
-0.29759613     Cross entropy   -0.30103
-0.29759613     entropy measures        -0.30103
-0.29179376     measures the    -0.30103
-0.7510117      is the  -0.30103
-0.7510117      of the  -0.30103
-0.29179376     solved the      -0.30103
-0.29179376     dictate the     -0.30103
-0.9830959      the difference  -0.30103
-0.29296046     difference between      -0.30103
-0.29296046     relationship between    -0.30103
-0.59521925     between two     -0.30103
-0.29759613     two probability -0.30103
-0.29759613     probability distributions       -0.30103
-0.26244023     distributions . -0.30103
-0.6713586      NLP .   -0.30103
-0.722806       models .        -0.42596874
-0.26244023     forever .       -0.30103
-0.26244023     significantly . -0.30103
```

```
-0.26244023     hallucinations .        -0.30103
-0.26244023     data .  -0.30103
-0.26244023     tasks . -0.30103
-0.26244023     need .  -0.30103
-0.52803135     model . -0.30103
-0.26244023     fun .   -0.30103
-0.26244023     processing .    -0.30103
-0.52803135     words . -0.30103
-0.26244023     problem .       -0.30103
-0.26244023     performance .   -0.30103
-1.2471787      <s> I   -0.30103
-0.29759613     I love  -0.30103
-1.5240324      <s> NLP -0.30103
-0.59584284     love NLP        -0.30103
-0.29791027     improved NLP    -0.30103
-1.5187746      <s> Kneser      -0.30103
-0.29759613     Kneser Ney      -0.30103
-0.29759613     Ney smoothing   -0.30103
-0.29791027     smoothing is    -0.30103
-0.29791027     Attention is    -0.30103
-0.29791027     BERT is -0.30103
-0.9830959      the state       -0.30103
-0.29791027     state of        -0.30103
-0.29791027     amounts of      -0.30103
-0.29791027     representations of      -0.30103
-0.9830959      the art -0.30103
-0.29296046     art for -0.30103
-0.29296046     stands for      -0.30103
-0.59521925     for statistical -0.30103
-0.5662914      love language   -0.30103
-0.28277752     statistical language    -0.30103
-0.28277752     Large language  -0.30103
-0.28277752     needs language  -0.30103
-0.28277752     masked language -0.30103
-0.28277752     autoregressive language -0.30103
-0.28277752     natural language        -0.30103
-0.28277752     Neural language -0.30103
-0.244722       language models -0.38021123
-0.29179376     pretrained models       -0.30103
-0.29179376     Language models -0.30103
-0.29179376     GPT models      -0.30103
-0.29179376     n-gram models   -0.30103
-1.5187746      <s> Transformers        -0.30103
-0.29759613     Transformers changed    -0.30103
-0.29759613     changed deep    -0.30103
-0.29296046     deep learning   -0.30103
-0.29296046     Deep learning   -0.30103
-0.59521925     learning forever        -0.30103
```

```
-1.5187746      <s> Deep        -0.30103
-0.59521925     learning improved       -0.30103
-0.76793003     NLP significantly       -0.30103
-1.5187746      <s> Retrieval   -0.30103
-0.29759613     Retrieval augmented     -0.30103
-0.29759613     augmented generation    -0.30103
-0.29759613     generation helps        -0.30103
-0.29759613     helps reduce    -0.30103
-0.29759613     reduce hallucinations   -0.30103
-1.5187746      <s> Large       -0.30103
-1.1226423      models require  -0.30103
-0.29759613     require massive -0.30103
-0.29759613     massive amounts -0.30103
-0.76793003     of training     -0.30103
-0.29759613     training data   -0.30103
-1.5187746      <s> Fine        -0.30103
-0.29759613     Fine tuning     -0.30103
-0.29759613     tuning adapts   -0.30103
-0.29759613     adapts pretrained       -0.30103
-1.1226423      models to       -0.30103
-0.29759613     to specific     -0.30103
-0.29759613     specific downstream     -0.30103
-0.29759613     downstream tasks        -0.30103
-1.5187746      <s> Attention   -0.30103
-0.76793003     is all  -0.30103
-0.29759613     all you -0.30103
-0.29759613     you need        -0.30103
-1.5187746      <s> Speech      -0.30103
-0.29759613     Speech recognition      -0.30103
-0.29759613     recognition needs       -0.30103
-1.5187746      <s> BERT        -0.30103
-0.76793003     is a    -0.30103
-0.29759613     a masked        -0.30103
-0.5860682      between model   -0.30103
-1.1651931      language model  -0.30103
-1.5187746      <s> Language    -0.30103
-0.8331964      models are      -0.30103
-0.59521925     are fun -0.30103
-1.5187746      <s> GPT -0.30103
-0.59521925     are autoregressive      -0.30103
-0.76793003     NLP stands      -0.30103
-0.59521925     for natural     -0.30103
-1.1872159      language processing     -0.30103
-1.5187746      <s> Neural      -0.30103
-1.1226423      models use      -0.30103
-0.29759613     use continuous  -0.30103
-0.29759613     continuous representations      -0.30103
-0.75437903     of words        -0.30103
```

```
-0.29296046     unseen words    -0.30103
-1.5187746      <s> Long        -0.30103
-0.29759613     Long short      -0.30103
-0.29759613     short term      -0.30103
-0.29759613     term memory     -0.30103
-0.29759613     memory networks -0.30103
-0.29759613     networks solved -0.30103
-0.9830959      the vanishing   -0.30103
-0.29759613     vanishing gradient      -0.30103
-0.29759613     gradient problem        -0.30103
-1.5187746      <s> Scaling     -0.30103
-0.29759613     Scaling laws    -0.30103
-0.29759613     laws dictate    -0.30103
-0.9830959      the relationship        -0.30103
-0.59521925     model size      -0.30103
-0.29759613     size and        -0.30103
-0.29759613     and performance -0.30103
-1.5187746      <s> Smoothing   -0.30103
-0.29759613     Smoothing techniques    -0.30103
-0.29759613     techniques help -0.30103
-0.29759613     help handle     -0.30103
-0.29759613     handle unseen   -0.30103
-0.59521925     words in        -0.30103
-0.29759613     in n-gram       -0.30103

\3-grams:
-0.022094991    distributions . </s>
-0.022094991    NLP . </s>
-0.016464982    models . </s>
-0.022094991    forever . </s>
-0.022094991    significantly . </s>
-0.022094991    hallucinations . </s>
-0.022094991    data . </s>
-0.022094991    tasks . </s>
-0.022094991    need . </s>
-0.022094991    model . </s>
-0.022094991    fun . </s>
-0.022094991    processing . </s>
-0.022094991    words . </s>
-0.022094991    problem . </s>
-0.022094991    performance . </s>
-0.12379109     <s> Cross entropy
-0.12379109     Cross entropy measures
-0.12183814     entropy measures the
-0.23010074     smoothing is the
-0.23010074     state of the
-0.12183814     networks solved the
-0.12183814     laws dictate the
```

```
-0.2580731      measures the difference
-0.1222322      the difference between
-0.1222322      the relationship between
-0.20274317     difference between two
-0.12379109     between two probability
-0.12379109     two probability distributions
-0.1116908      probability distributions .
-0.21712321     love NLP .
-0.30491292     language models .
-0.22573169     n-gram models .
-0.1116908      learning forever .
-0.1116908      NLP significantly .
-0.1116908      reduce hallucinations .
-0.1116908      training data .
-0.1116908      downstream tasks .
-0.1116908      you need .
-0.18827027     language model .
-0.1116908      are fun .
-0.1116908      language processing .
-0.18827027     of words .
-0.1116908      gradient problem .
-0.1116908      and performance .
-0.12379109     <s> I love
-0.42388642     I love NLP
-0.123896345    learning improved NLP
-0.12379109     <s> Kneser Ney
-0.12379109     Kneser Ney smoothing
-0.123896345    Ney smoothing is
-0.123896345    <s> Attention is
-0.123896345    <s> BERT is
-0.2580731      is the state
-0.123896345    the state of
-0.123896345    massive amounts of
-0.123896345    continuous representations of
-0.2580731      of the art
-0.1222322      the art for
-0.1222322      NLP stands for
-0.20274317     art for statistical
-0.4137156      I love language
-0.11876897     for statistical language
-0.11876897     <s> Large language
-0.11876897     recognition needs language
-0.11876897     a masked language
-0.11876897     are autoregressive language
-0.11876897     for natural language
-0.11876897     <s> Neural language
-0.10534694     love language models
-0.10534694     statistical language models
```

```
-0.10534694     Large language models
-0.10534694     needs language models
-0.10534694     autoregressive language models
-0.10534694     Neural language models
-0.12183814     adapts pretrained models
-0.12183814     <s> Language models
-0.12183814     <s> GPT models
-0.12183814     in n-gram models
-0.12379109     <s> Transformers changed
-0.12379109     Transformers changed deep
-0.1222322      changed deep learning
-0.1222322      <s> Deep learning
-0.20274317     deep learning forever
-0.20274317     Deep learning improved
-0.23260823     improved NLP significantly
-0.12379109     <s> Retrieval augmented
-0.12379109     Retrieval augmented generation
-0.12379109     augmented generation helps
-0.12379109     generation helps reduce
-0.12379109     helps reduce hallucinations
-0.94025105     language models require
-0.12379109     models require massive
-0.12379109     require massive amounts
-0.23260823     amounts of training
-0.12379109     of training data
-0.12379109     <s> Fine tuning
-0.12379109     Fine tuning adapts
-0.12379109     tuning adapts pretrained
-0.26946092     pretrained models to
-0.12379109     models to specific
-0.12379109     to specific downstream
-0.12379109     specific downstream tasks
-0.23260823     Attention is all
-0.12379109     is all you
-0.12379109     all you need
-0.12379109     <s> Speech recognition
-0.12379109     Speech recognition needs
-0.23260823     BERT is a
-0.12379109     is a masked
-0.20087416     relationship between model
-0.27231205     masked language model
-0.24153237     Language models are
-0.24153237     GPT models are
-0.42367646     models are fun
-0.42367646     models are autoregressive
-0.23260823     <s> NLP stands
-0.20274317     stands for natural
-0.2736883      natural language processing
```

```
-0.94025105     language models use
-0.12379109     models use continuous
-0.12379109     use continuous representations
-0.23060648     representations of words
-0.1222322      handle unseen words
-0.12379109     <s> Long short
-0.12379109     Long short term
-0.12379109     short term memory
-0.12379109     term memory networks
-0.12379109     memory networks solved
-0.2580731      solved the vanishing
-0.12379109     the vanishing gradient
-0.12379109     vanishing gradient problem
-0.12379109     <s> Scaling laws
-0.12379109     Scaling laws dictate
-0.2580731      dictate the relationship
-0.20274317     between model size
-0.12379109     model size and
-0.12379109     size and performance
-0.12379109     <s> Smoothing techniques
-0.12379109     Smoothing techniques help
-0.12379109     techniques help handle
-0.12379109     help handle unseen
-0.20274317     unseen words in
-0.12379109     words in n-gram

\end\
```

[42]:
```
!ls -lh
```

```
total 36K
-rw-rw-r-- 1 ye ye  12K May  1 12:50 lm_toy_3gram.arpa
-rw-rw-r-- 1 ye ye 1.2K May  1 12:50 lm_toy_3gram.arpa.err
-rw-rw-r-- 1 ye ye  151 May  1 12:48 lm_toy.test.txt
-rw-rw-r-- 1 ye ye  977 May  1 12:48 lm_toy.train.txt
-rw-rw-r-- 1 ye ye 1.3K May  1 12:48 lm_toy.txt
-rw-rw-r-- 1 ye ye  140 May  1 12:48 lm_toy.val.txt
drwxrwxr-x 2 ye ye 4.0K May  1 12:50 tmp_kenlm
```

Error ဖိုင်ကိုလည်း လေ့လာကြည့်ရအောင်။

တကယ်က error ဖိုင်လို့ နာမည်ပေးခဲ့ပေမဲ့ build log ဖိုင်ပါ။ Python code ထဲမှာ

`lmplz command > {arpa_file} 2>{arpa_file}.err` ဆိုပြီး သုံးခဲ့လို့ မော်ဒယ်ဖိုင်ကိုတော့ {arpa_file} အနေနဲ့ သိမ်းပေးပါ။ တခြား progress logs တွေ warning တွေနဲ့ error တွေကိုတော့ `{arpa_file}.err` မှာ သိမ်းပေးပါလို့ ရေးထားခဲ့လို့ ထွက်လာတဲ့ ဖိုင်ပါ။

[43]:
```
!cat lm_toy_3gram.arpa.err
```

```
=== 1/5 Counting and sorting n-grams ===
File stdin isn't normal.  Using slower read() instead of mmap().  No progress
bar.
Unigram tokens 151 types 102
=== 2/5 Calculating and sorting adjusted counts ===
Chain sizes: 1:1224 2:5975606272 3:11204261888
Substituting fallback discounts for order 1: D1=0.5 D2=1 D3+=1.5
Substituting fallback discounts for order 2: D1=0.5 D2=1 D3+=1.5
Statistics:
1 102 D1=0.897959 D2=0.383673 D3+=3
2 140 D1=0.5 D2=1 D3+=1.5
3 143 D1=0.5 D2=1 D3+=1.5
Memory estimate for binary LM:
type       B
probing 8596 assuming -p 1.5
probing 9848 assuming -r models -p 1.5
trie    4563 without quantization
trie    6401 assuming -q 8 -b 8 quantization
trie    4565 assuming -a 22 array pointer compression
trie    6403 assuming -a 22 -q 8 -b 8 array pointer compression and quantization
=== 3/5 Calculating and sorting initial probabilities ===
Chain sizes: 1:1224 2:2240 3:2860
=== 4/5 Calculating and writing order-interpolated probabilities ===
Chain sizes: 1:1224 2:2240 3:2860
=== 5/5 Writing ARPA model ===
Name:lmplz      VmPeak:16942008 kB      VmRSS:5976 kB   RSSMax:3870720 kB
user:0.127796   sys:0.821694    CPU:0.949531    real:1.19766
```

ဒီနေရာမှာ ပြောနေတဲ့

```
Substituting fallback discounts for order 1: D1=0.5 D2=1 D3+=1.5
Substituting fallback discounts for order 2: D1=0.5 D2=1 D3+=1.5
```

message တွေက ဒေတာမှာ sparsity problem ရှိနေတယ် ဆိုတာကို ဆိုလိုတာပါ။ ဆရာတို့ ဒေတာဖိုင်က ဖိုင်အသေးလေးမို့လို့ပါ။ တကယ်တမ်း ngram LM ကို ဆောက်ဖို့က ngram pair တွေက ထပ်ခါထပ်ခါ ပါနေဖို့ လိုအပ်ပါတယ်။ အဲဒါမှာ LM ကောင်းကောင်း ရမှာပါ။

မင်းရဲ့ data နဲ့ bigram, trigram counting တန်ဖိုးတွေက မငြိမ်ဘူး။ ပရမ်းပတာဖြစ်နေတယ်လို့ ဆိုလိုတာပါ။ အဲဒါကြောင့် Kneser-Ney smoothing discounts ကို တွက်လို့ မရဘူး။ Default training အတိုင်းပဲ ဆက်လုပ်သွားမယ် စသည်ဖြင့် information ပေးတာပါ။

0.3 SRILM Installation

- Put the SRILM source under /home/ye/tool/srilm
- Compile it there (binaries will appear under $SRILM/bin/$MACHINE_TYPE)
- Add the binaries to your PATH by setting environment variables in your shell config.

This matches the official INSTALL instructions: unpack, set SRILM, run make World, then add `$SRILM/bin/$MACHINE_TYPE` and `$SRILM/bin to PATH`, and `$SRILM/man` to MANPATH.

0.3.1 0) Install OS prerequisites

SRILM needs: a C/C++ compiler (GCC), GNU make, gawk, gzip, and (importantly) a C-shell (csh/tcsh) because the sbin/machine-type script used during the build is a csh script.

[ ]:
```
#sudo apt-get update
#sudo apt-get install -y build-essential gawk gzip bzip2 tcsh
```

Note: The FAQ explicitly says that without /bin/csh you'll see "make: /sbin/machine-type: Command not found"; installing tcsh fixes this.

Verify key tools exist:

[19]:
```
!which gcc g++ make gawk gzcat gunzip
```

```
/usr/lib/nvidia-cuda-toolkit/bin/gcc
/usr/lib/nvidia-cuda-toolkit/bin/g++
/usr/bin/make
/usr/bin/gunzip
```

[22]:
```
!which csh   # should resolve to /bin/csh or a link to tcsh
```

အထက်မှာ မြင်ရတဲ့အတိုင်း ဆရာ့စက်ထဲမှာ csh ကို which command နဲ့ ခေါ်ကြည့်တော့ ဘာမှမပြလို့ အောက်ပါအတိုင်း terminal ထွက်ပြီး installation လုပ်ခဲ့တယ်။ Jupyter Notebook cell ထဲကနေက admin password ကို ရိုက်ပေးဖို့ခက်လို့...

```
ye@lst-hpc3090:~/tool$ sudo apt-get install -y tcsh
[sudo] password for ye:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  tcsh
0 upgraded, 1 newly installed, 0 to remove and 95 not upgraded.
Need to get 450 kB of archives.
After this operation, 1,418 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 tcsh amd64 6.24.10-4build1 [450 kB]
Fetched 450 kB in 2s (278 kB/s)
Selecting previously unselected package tcsh.
(Reading database ... 425519 files and directories currently installed.)
Preparing to unpack .../tcsh_6.24.10-4build1_amd64.deb ...
Unpacking tcsh (6.24.10-4build1) ...
Setting up tcsh (6.24.10-4build1) ...
update-alternatives: using /bin/tcsh to provide /bin/csh (csh) in auto mode
Processing triggers for debianutils (5.17build1) ...
Processing triggers for man-db (2.12.0-4build2) ...
ye@lst-hpc3090:~/tool$
ye@lst-hpc3090:~/tool$ which csh
/usr/bin/csh
ye@lst-hpc3090:~/tool$
```

SRILM Download Page:http://www.speech.sri.com/projects/srilm/download.html
SRILM GitHub Page: https://github.com/BitSpeech/SRILM

[38]:
```
!tcsh --help
```

```
tcsh 6.24.10 (Astron) 2023-04-14 (x86_64-unknown-linux) options
wide,nls,dl,al,kan,sm,rh,nd,color,filec

-b file         batch mode, read and execute commands from `file'
-c command      run `command' from next argument
-d              load directory stack from `~/.cshdirs'
-Dname[=value]  define environment variable `name' to `value' (DomainOS only)
-e              exit on any error
-f              start faster by ignoring the start-up file
-F              use fork() instead of vfork() when spawning (ConvexOS only)
-i              interactive, even when input is not from a terminal
-l              act as a login shell, must be the only option specified
-m              load the start-up file, whether or not owned by effective user
-n file         no execute mode, just check syntax of the following `file'
-q              accept SIGQUIT for running under a debugger
-s              read commands from standard input
-t              read one line from standard input
-v              echo commands after history substitution
-V              like -v but including commands read from the start-up file
-x              echo commands immediately before execution
-X              like -x but including commands read from the start-up file
--help          print this message and exit
--version       print the version shell variable and exit

See the tcsh(1) manual page for detailed information.
```

[39]:
```
%pwd
```

[39]: '/mnt/disk1/ye/exp/LM-Tutorial'

[40]:
```
%cd /home/ye/tool/
```

```
/home/ye/tool
```

[29]:
```
!git clone https://github.com/BitSpeech/SRILM
```

```
Cloning into 'SRILM'…
remote: Enumerating objects: 4543, done.
remote: Counting objects: 100% (336/336), done.
remote: Compressing objects: 100% (299/299), done.
remote: Total 4543 (delta 44), reused 99 (delta 31), pack-reused 4207 (from
1)
Receiving objects: 100% (4543/4543), 65.67 MiB | 18.78 MiB/s, done.
Resolving deltas: 100% (2304/2304), done.
```

the most common way to check the hardware architecture:

[31]:
```
!uname -m
```

```
x86_64
```

[32]:
```
!lscpu | grep Architecture
```

```
Architecture:                           x86_64
```

Updating the "SRILM path" inside "Makefile"

```
# the following is the original example path of Makefile
# SRILM = /home/speech/stolcke/project/srilm/devel

# Add your path here, for example
SRILM = /home/ye/tool/SRILM
```

make command က Jupyter Notebook ရဲ့ cell အတွင်းက run ရင် အလုပ်မလုပ်တဲ့အခါရှိပါတယ်။ အကြောင်းအရင်းကတော့ bash environment variable, paths တွေနဲ့ shell behavior တွေက မတူကြလို့ပါ။

ဒီ tutorial အတွက် SRILM ကို terminal မှာပဲ ထွက်ပြီး installation လုပ်ခဲ့တယ်။ အောက်ပါအတိုင်းပါ။

```
ye@lst-hpc3090:~/tool/SRILM$ make SRILM=/home/ye/tool/SRILM World | tee srilm_install.log
...
...
...
make[2]: Entering directory '/home/ye/tool/SRILM/lm/src'
make[2]: Nothing to be done for 'release-scripts'.
make[2]: Leaving directory '/home/ye/tool/SRILM/lm/src'
make[2]: Entering directory '/home/ye/tool/SRILM/flm/src'
make[2]: Nothing to be done for 'release-scripts'.
make[2]: Leaving directory '/home/ye/tool/SRILM/flm/src'
make[2]: Entering directory '/home/ye/tool/SRILM/lattice/src'
make[2]: Nothing to be done for 'release-scripts'.
make[2]: Leaving directory '/home/ye/tool/SRILM/lattice/src'
make[2]: Entering directory '/home/ye/tool/SRILM/utils/src'
/home/ye/tool/SRILM/sbin/decipher-install 0555 change-lm-vocab ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 empty-sentence-lm ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 rescore-decipher ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 rescore-acoustic ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 rescore-reweight ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 rescore-minimize-wer ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 make-batch-counts ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 merge-batch-counts ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 make-big-lm ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 make-multiword-pfsg ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 pfsg-from-ngram ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 nbest-error ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 nbest-rover ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 search-rover-combo ../../bin
```

```
/home/ye/tool/SRILM/sbin/decipher-install 0555 rexport.gnumake ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 align-with-tags ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 compute-sclite ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 compute-sclite-nbest ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 compare-sclite ../../bin
/home/ye/tool/SRILM/sbin/decipher-install 0555 cumbin ../../bin
make[2]: Leaving directory '/home/ye/tool/SRILM/utils/src'
make[2]: Entering directory '/home/ye/tool/SRILM/zlib/src'
make[2]: Nothing to be done for 'release-scripts'.
make[2]: Leaving directory '/home/ye/tool/SRILM/zlib/src'
make[1]: Leaving directory '/home/ye/tool/SRILM'
ye@lst-hpc3090:~/tool/SRILM$
```

[44]:
```
%pwd
```

[44]:
```
'/home/ye/tool'
```

[45]:
```
%cd SRILM
```

```
/home/ye/tool/SRILM
```

[46]:
```
!ls
```

```
ACKNOWLEDGEMENTS  Copyright  include  License   misc     srilm_install.log
bin               doc        INSTALL  lm        README   utils
CHANGES           dstruct    lattice  Makefile  RELEASE  visual_studio
common            flm        lib      man       sbin     zlib
```

SRILM ကို make နဲ့ compile လုပ်တာက ကိုယ့်စက်ထဲမှာ အဆင်ပြေပြေနဲ့ ပြီးသွားရင် အောက်ပါလိုမျိုး statistical LM နဲ့ ပတ်သက်တဲ့ tool တွေကို တွေ့ရလိမ့်မယ်။

[49]:
```
!ls ./bin/i686-m64/* --color=auto -F
```

```
./bin/i686-m64/add-classes-to-pfsg*
./bin/i686-m64/add-dummy-bows*
./bin/i686-m64/add-pauses-to-pfsg*
./bin/i686-m64/add-ppls*
./bin/i686-m64/anti-ngram*
./bin/i686-m64/bytelog-to-log10*
./bin/i686-m64/classes-to-fsm*
./bin/i686-m64/combine-acoustic-scores*
./bin/i686-m64/combine-rover-controls*
./bin/i686-m64/compare-ppls*
./bin/i686-m64/compute-best-mix*
./bin/i686-m64/compute-best-rover-mix*
./bin/i686-m64/compute-best-sentence-mix*
./bin/i686-m64/compute-oov-rate*
./bin/i686-m64/concat-sausages*
./bin/i686-m64/context-ngrams*
./bin/i686-m64/continuous-ngram-count*
```

```
./bin/i686-m64/de-vq-lm*
./bin/i686-m64/disambig*
./bin/i686-m64/extract-skip-probs*
./bin/i686-m64/filter-event-counts*
./bin/i686-m64/find-reference-posteriors*
./bin/i686-m64/fix-ctm*
./bin/i686-m64/fngram*
./bin/i686-m64/fngram-count*
./bin/i686-m64/fsm-to-pfsg*
./bin/i686-m64/get-gt-counts*
./bin/i686-m64/get-unigram-probs*
./bin/i686-m64/hidden-ngram*
./bin/i686-m64/hits-from-log*
./bin/i686-m64/htklat-vocab*
./bin/i686-m64/lattice-tool*
./bin/i686-m64/log10-to-bytelog*
./bin/i686-m64/make-abs-discount*
./bin/i686-m64/make-diacritic-map*
./bin/i686-m64/make-google-ngrams*
./bin/i686-m64/make-gt-discounts*
./bin/i686-m64/make-hiddens-lm*
./bin/i686-m64/make-kn-counts*
./bin/i686-m64/make-kn-discounts*
./bin/i686-m64/make-lm-subset*
./bin/i686-m64/make-nbest-pfsg*
./bin/i686-m64/make-ngram-pfsg*
./bin/i686-m64/make-sub-lm*
./bin/i686-m64/maxalloc*
./bin/i686-m64/merge-nbest*
./bin/i686-m64/metadb*
./bin/i686-m64/multi-ngram*
./bin/i686-m64/nbest2-to-nbest1*
./bin/i686-m64/nbest-lattice*
./bin/i686-m64/nbest-mix*
./bin/i686-m64/nbest-oov-counts*
./bin/i686-m64/nbest-optimize*
./bin/i686-m64/nbest-optimize-args-from-rover-control*
./bin/i686-m64/nbest-posteriors*
./bin/i686-m64/nbest-pron-score*
./bin/i686-m64/nbest-rover-helper*
./bin/i686-m64/nbest-vocab*
./bin/i686-m64/nbest-words*
./bin/i686-m64/ngram*
./bin/i686-m64/ngram-class*
./bin/i686-m64/ngram-count*
./bin/i686-m64/ngram-merge*
./bin/i686-m64/pfsg-to-dot*
./bin/i686-m64/pfsg-to-fsm*
```

```
./bin/i686-m64/pfsg-vocab*
./bin/i686-m64/ppl-from-log*
./bin/i686-m64/prettify*
./bin/i686-m64/remove-lowprob-ngrams*
./bin/i686-m64/replace-unk-words*
./bin/i686-m64/replace-words-with-classes*
./bin/i686-m64/reverse-lm*
./bin/i686-m64/reverse-ngram-counts*
./bin/i686-m64/reverse-text*
./bin/i686-m64/rover-control-tying*
./bin/i686-m64/rover-control-weights*
./bin/i686-m64/segment*
./bin/i686-m64/segment-nbest*
./bin/i686-m64/select-vocab*
./bin/i686-m64/sentid-to-ctm*
./bin/i686-m64/sentid-to-sclite*
./bin/i686-m64/sort-lm*
./bin/i686-m64/split-tagged-ngrams*
./bin/i686-m64/subset-context-ngrams*
./bin/i686-m64/subtract-ppls*
./bin/i686-m64/tolower-ngram-counts*
./bin/i686-m64/uniform-classes*
./bin/i686-m64/uniq-ngram-counts*
./bin/i686-m64/vp2text*
./bin/i686-m64/wlat-stats*
./bin/i686-m64/wlat-to-dot*
./bin/i686-m64/wlat-to-pfsg*
./bin/i686-m64/wordlat-to-lisp*
```

updating your .bashrc file

```
export PATH="/home/ye/tool/SRILM/bin/i686-m64:$PATH"
```

ပြီးရင် .bashrc ကို ပြန် activate လုပ်ပါ။

```
source /home/ye/.bashrc
```

[1]:
```
%cd /mnt/disk1/ye/exp/LM-Tutorial
```

```
/mnt/disk1/ye/exp/LM-Tutorial
```

[2]:
```
!ngram-count --help
```

```
/bin/bash: line 1: ngram-count: command not found
```

[3]:
```
!echo $PATH
```

```
/home/ye/tool/speech_tools/bin:/home/ye/tool/festival/bin:/home/ye/tool/festvox/
bin:/home/ye/tool/flite/bin:/home/ye/.nvm/versions/node/v20.19.3/bin:/usr/lib/nv
idia-cuda-toolkit/bin:/usr/bin:/bin:/home/ye/tool/marian/build:/home/ye/.local/b
in:/home/ye/tool/speech_tools/bin:/home/ye/tool/festival/bin:/home/ye/tool/festv
```

```
ox/bin:/home/ye/tool/flite/bin:/home/ye/.nvm/versions/node/v20.19.3/bin:/home/ye
/.cargo/bin:/usr/lib/nvidia-cuda-toolkit/bin:/usr/bin:/bin:/home/ye/tool/marian/
build:/home/ye/miniforge3/condabin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr
/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/home/ye/.local/bin:/home/
ye/tool/kenlm/build/bin/:/home/ye/.local/bin:/home/ye/.local/bin:/home/ye/tool/k
enlm/build/bin/
```

အောက်ပါ command က တမင်တကာ run ပြတာ။ source ကို activate လုပ်လည်း updated PATH က ရမလာဘူး။

[4]:
```
!source /home/ye/.bashrc
```

[5]:
```
!echo $PATH
```

```
/home/ye/tool/speech_tools/bin:/home/ye/tool/festival/bin:/home/ye/tool/festvox/
bin:/home/ye/tool/flite/bin:/home/ye/.nvm/versions/node/v20.19.3/bin:/usr/lib/nv
idia-cuda-toolkit/bin:/usr/bin:/bin:/home/ye/tool/marian/build:/home/ye/.local/b
in:/home/ye/tool/speech_tools/bin:/home/ye/tool/festival/bin:/home/ye/tool/festv
ox/bin:/home/ye/tool/flite/bin:/home/ye/.nvm/versions/node/v20.19.3/bin:/home/ye
/.cargo/bin:/usr/lib/nvidia-cuda-toolkit/bin:/usr/bin:/bin:/home/ye/tool/marian/
build:/home/ye/miniforge3/condabin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr
/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/home/ye/.local/bin:/home/
ye/tool/kenlm/build/bin/:/home/ye/.local/bin:/home/ye/.local/bin:/home/ye/tool/k
enlm/build/bin/
```

[44]:
```
import os
# This updates the PATH for the duration of the notebook session
os.environ['PATH'] = "/home/ye/tool/SRILM/bin/i686-m64:" + os.environ['PATH']

# Verify
!echo $PATH
```

```
/home/ye/tool/SRILM/bin/i686-
m64:/home/ye/tool/speech_tools/bin:/home/ye/tool/festival/bin:/home/ye/tool/fest
vox/bin:/home/ye/tool/flite/bin:/home/ye/.nvm/versions/node/v20.19.3/bin:/usr/li
b/nvidia-cuda-toolkit/bin:/usr/bin:/bin:/home/ye/tool/marian/build:/home/ye/.loc
al/bin:/home/ye/tool/speech_tools/bin:/home/ye/tool/festival/bin:/home/ye/tool/f
estvox/bin:/home/ye/tool/flite/bin:/home/ye/.nvm/versions/node/v20.19.3/bin:/hom
e/ye/.cargo/bin:/usr/lib/nvidia-cuda-toolkit/bin:/usr/bin:/bin:/home/ye/tool/mar
ian/build:/home/ye/miniforge3/condabin:/usr/local/sbin:/usr/local/bin:/usr/sbin:
/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/home/ye/.local/bin:/h
ome/ye/tool/kenlm/build/bin/:/home/ye/.local/bin:/home/ye/.local/bin:/home/ye/to
ol/kenlm/build/bin/
```

အထက်မှာ မြင်ရတဲ့အတိုင်း /home/ye/tool/SRILM/bin/i686-m64 ကို PATH environment variable မှာ သိသွားပြီမို့လို့ SRILM command တွေကို ခုဆိုရင်တော့ ခေါ် run လို့ ရလိမ့်မယ်။

[45]:
```
!ngram-count -help
```

```
Usage of command "ngram-count"
 -version:                 print version information
 -order:                   max ngram order
                Default value: 3
 -varprune:                pruning threshold for variable order ngrams
                Default value: 0
 -debug:                   debugging level for LM
                Default value: 0
 -recompute:               recompute lower-order counts by summation
 -sort:                    sort ngrams output
 -write-order:             output ngram counts order
                Default value: 0
 -tag:                     file tag to use in messages
 -text:                    text file to read
 -text-has-weights:        text file contains count weights
 -text-has-weights-last:   text file contains count weights at ends of lines
 -no-sos:                  don't insert start-of-sentence tokens
 -no-eos:                  don't insert end-of-sentence tokens
 -read:                    counts file to read
 -intersect:               intersect counts with this file
 -read-with-mincounts:     apply minimum counts when reading counts file
 -read-google:             Google counts directory to read
 -write:                   counts file to write
 -write1:                  1gram counts file to write
 -write2:                  2gram counts file to write
 -write3:                  3gram counts file to write
 -write4:                  4gram counts file to write
 -write5:                  5gram counts file to write
 -write6:                  6gram counts file to write
 -write7:                  7gram counts file to write
 -write8:                  8gram counts file to write
 -write9:                  9gram counts file to write
 -write-binary:            binary counts file to write
 -gtmin:                   lower GT discounting cutoff
                Default value: 1
 -gtmax:                   upper GT discounting cutoff
                Default value: 5
 -gt1min:                  lower 1gram discounting cutoff
                Default value: 1
 -gt1max:                  upper 1gram discounting cutoff
                Default value: 1
 -gt2min:                  lower 2gram discounting cutoff
                Default value: 1
 -gt2max:                  upper 2gram discounting cutoff
                Default value: 7
 -gt3min:                  lower 3gram discounting cutoff
                Default value: 2
 -gt3max:                  upper 3gram discounting cutoff
```

```
                Default value: 7
-gt4min:                   lower 4gram discounting cutoff
                Default value: 2
-gt4max:                   upper 4gram discounting cutoff
                Default value: 7
-gt5min:                   lower 5gram discounting cutoff
                Default value: 2
-gt5max:                   upper 5gram discounting cutoff
                Default value: 7
-gt6min:                   lower 6gram discounting cutoff
                Default value: 2
-gt6max:                   upper 6gram discounting cutoff
                Default value: 7
-gt7min:                   lower 7gram discounting cutoff
                Default value: 2
-gt7max:                   upper 7gram discounting cutoff
                Default value: 7
-gt8min:                   lower 8gram discounting cutoff
                Default value: 2
-gt8max:                   upper 8gram discounting cutoff
                Default value: 7
-gt9min:                   lower 9gram discounting cutoff
                Default value: 2
-gt9max:                   upper 9gram discounting cutoff
                Default value: 7
-gt:                       Good-Turing discount parameter file
-gt1:                      Good-Turing 1gram discounts
-gt2:                      Good-Turing 2gram discounts
-gt3:                      Good-Turing 3gram discounts
-gt4:                      Good-Turing 4gram discounts
-gt5:                      Good-Turing 5gram discounts
-gt6:                      Good-Turing 6gram discounts
-gt7:                      Good-Turing 7gram discounts
-gt8:                      Good-Turing 8gram discounts
-gt9:                      Good-Turing 9gram discounts
-cdiscount:                discounting constant
                Default value: -1
-cdiscount1:               1gram discounting constant
                Default value: -1
-cdiscount2:               2gram discounting constant
                Default value: -1
-cdiscount3:               3gram discounting constant
                Default value: -1
-cdiscount4:               4gram discounting constant
                Default value: -1
-cdiscount5:               5gram discounting constant
                Default value: -1
-cdiscount6:               6gram discounting constant
```

```
                Default value: -1
-cdiscount7:              7gram discounting constant
                Default value: -1
-cdiscount8:              8gram discounting constant
                Default value: -1
-cdiscount9:              9gram discounting constant
                Default value: -1
-ndiscount:               use natural discounting
-ndiscount1:              1gram natural discounting
-ndiscount2:              2gram natural discounting
-ndiscount3:              3gram natural discounting
-ndiscount4:              4gram natural discounting
-ndiscount5:              5gram natural discounting
-ndiscount6:              6gram natural discounting
-ndiscount7:              7gram natural discounting
-ndiscount8:              8gram natural discounting
-ndiscount9:              9gram natural discounting
-addsmooth:               additive smoothing constant
                Default value: -1
-addsmooth1:              1gram additive smoothing constant
                Default value: -1
-addsmooth2:              2gram additive smoothing constant
                Default value: -1
-addsmooth3:              3gram additive smoothing constant
                Default value: -1
-addsmooth4:              4gram additive smoothing constant
                Default value: -1
-addsmooth5:              5gram additive smoothing constant
                Default value: -1
-addsmooth6:              6gram additive smoothing constant
                Default value: -1
-addsmooth7:              7gram additive smoothing constant
                Default value: -1
-addsmooth8:              8gram additive smoothing constant
                Default value: -1
-addsmooth9:              9gram additive smoothing constant
                Default value: -1
-wbdiscount:              use Witten-Bell discounting
-wbdiscount1:             1gram Witten-Bell discounting
-wbdiscount2:             2gram Witten-Bell discounting
-wbdiscount3:             3gram Witten-Bell discounting
-wbdiscount4:             4gram Witten-Bell discounting
-wbdiscount5:             5gram Witten-Bell discounting
-wbdiscount6:             6gram Witten-Bell discounting
-wbdiscount7:             7gram Witten-Bell discounting
-wbdiscount8:             8gram Witten-Bell discounting
-wbdiscount9:             9gram Witten-Bell discounting
-kndiscount:              use modified Kneser-Ney discounting
```

```
 -kndiscount1:             1gram modified Kneser-Ney discounting
 -kndiscount2:             2gram modified Kneser-Ney discounting
 -kndiscount3:             3gram modified Kneser-Ney discounting
 -kndiscount4:             4gram modified Kneser-Ney discounting
 -kndiscount5:             5gram modified Kneser-Ney discounting
 -kndiscount6:             6gram modified Kneser-Ney discounting
 -kndiscount7:             7gram modified Kneser-Ney discounting
 -kndiscount8:             8gram modified Kneser-Ney discounting
 -kndiscount9:             9gram modified Kneser-Ney discounting
 -ukndiscount:             use original Kneser-Ney discounting
 -ukndiscount1:            1gram original Kneser-Ney discounting
 -ukndiscount2:            2gram original Kneser-Ney discounting
 -ukndiscount3:            3gram original Kneser-Ney discounting
 -ukndiscount4:            4gram original Kneser-Ney discounting
 -ukndiscount5:            5gram original Kneser-Ney discounting
 -ukndiscount6:            6gram original Kneser-Ney discounting
 -ukndiscount7:            7gram original Kneser-Ney discounting
 -ukndiscount8:            8gram original Kneser-Ney discounting
 -ukndiscount9:            9gram original Kneser-Ney discounting
 -kn:                      Kneser-Ney discount parameter file
 -kn1:                     Kneser-Ney 1gram discounts
 -kn2:                     Kneser-Ney 2gram discounts
 -kn3:                     Kneser-Ney 3gram discounts
 -kn4:                     Kneser-Ney 4gram discounts
 -kn5:                     Kneser-Ney 5gram discounts
 -kn6:                     Kneser-Ney 6gram discounts
 -kn7:                     Kneser-Ney 7gram discounts
 -kn8:                     Kneser-Ney 8gram discounts
 -kn9:                     Kneser-Ney 9gram discounts
 -kn-counts-modified:      input counts already modified for KN smoothing
 -kn-modify-counts-at-end: modify counts after discount estimation rather than
before
 -interpolate:             use interpolated estimates
 -interpolate1:            use interpolated 1gram estimates
 -interpolate2:            use interpolated 2gram estimates
 -interpolate3:            use interpolated 3gram estimates
 -interpolate4:            use interpolated 4gram estimates
 -interpolate5:            use interpolated 5gram estimates
 -interpolate6:            use interpolated 6gram estimates
 -interpolate7:            use interpolated 7gram estimates
 -interpolate8:            use interpolated 8gram estimates
 -interpolate9:            use interpolated 9gram estimates
 -lm:                      LM to estimate
 -write-binary-lm:         output LM in binary format
 -init-lm:                 initial LM for EM estimation
 -unk:                     keep <unk> in LM
 -map-unk:                 word to map unknown words to
 -meta-tag:                meta tag used to input count-of-count information
```

```
 -float-counts:            use fractional counts
 -tagged:                  build a tagged LM
 -count-lm:                train a count-based LM
 -skip:                    build a skip N-gram LM
 -skip-init:               default initial skip probability
                Default value: 0.5
 -em-iters:                max number of EM iterations
                Default value: 100
 -em-delta:                min log likelihood delta for EM
                Default value: 0.001
 -stop-words:              stop-word vocabulary for stop-Ngram LM
 -maxent:                  Estimate maximum entropy model
 -maxent-alpha:            The L1 regularisation constant for max-ent estimation
                Default value: 0.5
 -maxent-sigma2:           The L2 regularisation constant for max-ent estimation
(default: 6 for estimation, 0.5 for adaptation)
                Default value: 0
 -maxent-convert-to-arpa:  Save estimated max-ent model as a regular ARPA
backoff model
 -tolower:                 map vocabulary to lowercase
 -trust-totals:            trust lower-order counts for estimation
 -prune:                   prune redundant probs
                Default value: 0
 -minprune:                prune only ngrams at least this long
                Default value: 2
 -vocab:                   vocab file
 -vocab-aliases:           vocab alias file
 -nonevents:               non-event vocabulary
 -limit-vocab:             limit count reading to specified vocabulary
 -write-vocab:             write vocab to file
 -write-vocab-index:       write vocab index map to file
 -write-text:              write input text to file (for validation)
 -memuse:                  show memory usage
 the default action is to write counts to stdout
 -help:                    Print this message
```

[47]:
```
%cd /mnt/disk1/ye/exp/LM-Tutorial
```

```
/mnt/disk1/ye/exp/LM-Tutorial
```

[45]:
```
!ls
```

```
lm_toy_3gram.arpa      lm_toy.test.txt   lm_toy.txt      tmp_kenlm
lm_toy_3gram.arpa.err  lm_toy.train.txt  lm_toy.val.txt
```

[48]:
```
# Train KN-smoothed 3-gram
!ngram-count -order 3 -kndiscount -text lm_toy.train.txt  -lm lm_toy_srilm.arpa
```

```
one of required modified KneserNey count-of-counts is zero
```

```
error in discount estimator for order 1
```

[48]:
```
!ls
```

```
lm_toy_3gram.arpa      lm_toy.test.txt   lm_toy.txt      tmp_kenlm
lm_toy_3gram.arpa.err  lm_toy.train.txt  lm_toy.val.txt
```

0.4 Why did SRILM crash?

Modified Kneser-Ney (KN) smoothing calculates "count-of-counts" (how many words appear exactly once, exactly twice, exactly three times, etc.) to figure out how much probability mass to steal from seen words to give to unseen words.

Because your training data is so small, one of those counts (likely the number of bigrams/trigrams that appear exactly twice) is exactly zero. The KN math formula involves dividing by that number. Dividing by zero causes the C++ code to crash hard.

Compare this to KenLM: Remember how KenLM printed Substituting fallback discounts… in your .err file and kept running? KenLM has modern "training wheels" for tiny datasets. SRILM is older, stricter research code—it sees a divide-by-zero coming and immediately halts to protect the mathematical integrity of the model.

0.5 Using Witten-Bell Discounting

When data is too small for Kneser-Ney, the standard fallback recommended by the SRILM creators is Witten-Bell smoothing (-wbdiscount). Instead of relying on complex count-of-counts, Witten-Bell estimates the probability of seeing a new word based on how many unique words you've seen so far. It never divides by zero.

[49]:
```
!ngram-count -order 3 -wbdiscount -text lm_toy.train.txt  -lm lm_toy_srilm.arpa
```

[50]:
```
!ls
```

```
lm_toy_3gram.arpa      lm_toy_srilm.arpa  lm_toy.train.txt  lm_toy.val.txt
lm_toy_3gram.arpa.err  lm_toy.test.txt    lm_toy.txt        tmp_kenlm
```

SRILM နဲ့ ဆောက်ထားတဲ့ language model တော့ ရလာပြီ။
Evaluation လုပ်ကြည့်ရအောင်။

[50]:
```
!head -n 30 ./lm_toy_srilm.arpa
```

```
\data\
ngram 1=101
ngram 2=140
ngram 3=3

\1-grams:
-1.130334       .       -1.267606
-1.130334       </s>
-99     <s>     -0.2451774
```

```
-2.130334       Attention       -0.2945479
-2.130334       BERT    -0.2945479
-2.130334       Cross   -0.297801
-2.130334       Deep    -0.2961775
-2.130334       Fine    -0.297801
-2.130334       GPT     -0.282966
-1.954242       I       -0.4722688
-2.130334       Kneser  -0.297801
-2.130334       Language        -0.282966
-2.130334       Large   -0.2863067
-2.130334       Long    -0.297801
-1.829304       NLP     -0.2606013
-2.130334       Neural  -0.2863067
-2.130334       Ney     -0.297801
-2.130334       Retrieval       -0.297801
-2.130334       Scaling -0.297801
-2.130334       Smoothing       -0.297801
-2.130334       Speech  -0.297801
-2.130334       Transformers    -0.297801
-2.130334       a       -0.297801
```

[51]:
```
!cat lm_toy.test.txt
```

```
I love speech recognition .
Perplexity is a common metric for evaluating language models .
Language models assign probabilities to sequences of words .
```

[52]:
```
# Evaluate
!ngram -lm lm_toy_srilm.arpa -ppl lm_toy.test.txt
```

```
file lm_toy.test.txt: 3 sentences, 24 words, 8 OOVs
0 zeroprobs, logprob= -18.98879 ppl= 9.986423 ppl1= 15.37444
```

0.6 Reading the ARPA Format

Notice the structure of this file. This is the standard ARPA format (used by SRILM, KenLM, and IRSTLM):

1. **\data\:** Metadata telling us how many N-grams exist in the file.

2. **\1-grams:** Unigram probabilities.
   - Format: log10(Prob) Word [log10(Backoff_Weight)]

   - Example: -1.39794 <unk> -0.477121 means P() = 10^-1.39.
   - The backoff weight is used if the unigram doesn't appear in a bigram context.

3. **\2-grams and \3-grams**: Higher order probabilities.
4. **\end\:** The end of the file.

Key insight: Everything is in log base 10 (to prevent underflow with tiny probabilities).

- A log prob of -1.0 means a 10% probability.
- A log prob of -2.0 means a 1% probability.

0.7 Tracing with -debug 2 Option

How did SRILM get PPL = 9.986? Let's force it to show its work.

[54]:
```
# -debug 2 prints the exact log-probability calculated for EVERY SINGLE WORD
!ngram -lm lm_toy_srilm.arpa -ppl lm_toy.test.txt -debug 2
```

```
reading 101 1-grams
reading 140 2-grams
reading 3 3-grams
I love speech recognition .
        p( I | <s> )    = [2gram] 0.05405402 [ -1.267172 ]
        p( love | I …)        = [3gram] 0.6666666 [ -0.1760913 ]
        p( <unk> | love …)    = [OOV] 0 [ -inf ]
        p( recognition | <unk> …)     = [1gram] 0.007407405 [ -2.130334 ]
        p( . | recognition …)         = [1gram] 0.03731341 [ -1.428135 ]
        p( </s> | . …)        = [2gram] 0.95 [ -0.02227639 ]
1 sentences, 5 words, 1 OOVs
0 zeroprobs, logprob= -5.024009 ppl= 10.11118 ppl1= 18.03027

Perplexity is a common metric for evaluating language models .
        p( <unk> | <s> )        = [OOV] 0 [ -inf ]
        p( is | <unk> …)      = [1gram] 0.01481481 [ -1.829304 ]
        p( a | is …)  = [2gram] 0.1666667 [ -0.7781513 ]
        p( <unk> | a …)       = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( for | <unk> …)     = [1gram] 0.01111112 [ -1.954242 ]
        p( <unk> | for …)     = [OOV] 0 [ -inf ]
        p( language | <unk> …)        = [1gram] 0.03333335 [ -1.477121 ]
        p( models | language …)       = [2gram] 0.5454546 [ -0.2632414 ]
        p( . | models …)      = [3gram] 0.4444445 [ -0.3521825 ]
        p( </s> | . …)        = [3gram] 0.8333333 [ -0.07918125 ]
1 sentences, 10 words, 4 OOVs
0 zeroprobs, logprob= -6.733423 ppl= 9.160467 ppl1= 13.25065

Language models assign probabilities to sequences of words .
        p( Language | <s> )     = [2gram] 0.02702701 [ -1.568202 ]
        p( models | Language …)       = [2gram] 0.5 [ -0.30103 ]
        p( <unk> | models …)  = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( to | <unk> …)      = [1gram] 0.007407405 [ -2.130334 ]
        p( <unk> | to …)      = [OOV] 0 [ -inf ]
        p( of | <unk> …)      = [1gram] 0.01481481 [ -1.829304 ]
        p( words | of …)      = [2gram] 0.1666667 [ -0.7781513 ]
        p( . | words …)       = [2gram] 0.25 [ -0.60206 ]
```

```
        p( </s> | . …)        = [2gram] 0.95 [ -0.02227639 ]
1 sentences, 9 words, 3 OOVs
0 zeroprobs, logprob= -7.231358 ppl= 10.79074 ppl1= 16.04081

file lm_toy.test.txt: 3 sentences, 24 words, 8 OOVs
0 zeroprobs, logprob= -18.98879 ppl= 9.986423 ppl1= 15.37444
```

[54]:
```
!ngram -lm lm_toy_srilm.arpa -ppl lm_toy.test.txt -debug 1
```

```
reading 101 1-grams
reading 140 2-grams
reading 3 3-grams
I love speech recognition .
1 sentences, 5 words, 1 OOVs
0 zeroprobs, logprob= -5.024009 ppl= 10.11118 ppl1= 18.03027

Perplexity is a common metric for evaluating language models .
1 sentences, 10 words, 4 OOVs
0 zeroprobs, logprob= -6.733423 ppl= 9.160467 ppl1= 13.25065

Language models assign probabilities to sequences of words .
1 sentences, 9 words, 3 OOVs
0 zeroprobs, logprob= -7.231358 ppl= 10.79074 ppl1= 16.04081

file lm_toy.test.txt: 3 sentences, 24 words, 8 OOVs
0 zeroprobs, logprob= -18.98879 ppl= 9.986423 ppl1= 15.37444
```

[55]:
```
!ngram -lm lm_toy_srilm.arpa -ppl lm_toy.test.txt -debug 3
```

```
reading 101 1-grams
reading 140 2-grams
reading 3 3-grams
I love speech recognition .
        p( I | <s> )    = [2gram] 0.05405402 [ -1.267172 ] / 0.9999995
        p( love | I …)        = [3gram] 0.6666666 [ -0.1760913 ] / 0.9999998
        p( <unk> | love …)    = [OOV] 0 [ -inf ] / 1
        p( recognition | <unk> …)     = [1gram] 0.007407405 [ -2.130334 ] /
0.9999998
        p( . | recognition …)         = [1gram] 0.03731341 [ -1.428135 ] /
0.9999999
        p( </s> | . …)        = [2gram] 0.95 [ -0.02227639 ] / 1
1 sentences, 5 words, 1 OOVs
0 zeroprobs, logprob= -5.024009 ppl= 10.11118 ppl1= 18.03027

Perplexity is a common metric for evaluating language models .
        p( <unk> | <s> )        = [OOV] 0 [ -inf ] / 0.9999995
        p( is | <unk> …)      = [1gram] 0.01481481 [ -1.829304 ] / 0.9999998
        p( a | is …)  = [2gram] 0.1666667 [ -0.7781513 ] / 0.9999999
        p( <unk> | a …)       = [OOV] 0 [ -inf ] / 0.9999999
```

```
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ] / 0.9999998
        p( for | <unk> …)     = [1gram] 0.01111112 [ -1.954242 ] / 0.9999998
        p( <unk> | for …)     = [OOV] 0 [ -inf ] / 1
        p( language | <unk> …)        = [1gram] 0.03333335 [ -1.477121 ] /
0.9999998
        p( models | language …)       = [2gram] 0.5454546 [ -0.2632414 ] /
0.9999998
        p( . | models …)      = [3gram] 0.4444445 [ -0.3521825 ] / 1
        p( </s> | . …)        = [3gram] 0.8333333 [ -0.07918125 ] / 1
1 sentences, 10 words, 4 OOVs
0 zeroprobs, logprob= -6.733423 ppl= 9.160467 ppl1= 13.25065

Language models assign probabilities to sequences of words .
        p( Language | <s> )     = [2gram] 0.02702701 [ -1.568202 ] / 0.9999995
        p( models | Language …)       = [2gram] 0.5 [ -0.30103 ] / 1
        p( <unk> | models …)  = [OOV] 0 [ -inf ] / 1
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ] / 0.9999998
        p( to | <unk> …)      = [1gram] 0.007407405 [ -2.130334 ] / 0.9999998
        p( <unk> | to …)      = [OOV] 0 [ -inf ] / 0.9999999
        p( of | <unk> …)      = [1gram] 0.01481481 [ -1.829304 ] / 0.9999998
        p( words | of …)      = [2gram] 0.1666667 [ -0.7781513 ] / 1
        p( . | words …)       = [2gram] 0.25 [ -0.60206 ] / 0.9999999
        p( </s> | . …)        = [2gram] 0.95 [ -0.02227639 ] / 1
1 sentences, 9 words, 3 OOVs
0 zeroprobs, logprob= -7.231358 ppl= 10.79074 ppl1= 16.04081

file lm_toy.test.txt: 3 sentences, 24 words, 8 OOVs
0 zeroprobs, logprob= -18.98879 ppl= 9.986423 ppl1= 15.37444
```

0.8 Debugging the Perplexity Calculation

Look closely at the debug output. For every single word, SRILM shows its work. Notice the tags like [3gram], [2gram], and [1gram]. This is Backoff in action!

Example 1: "I love speech recognition ."

p(I | <s>) = [2gram]: It found the bigram <s> I in the training data.
p(love | I …) = [3gram]: It found the trigram I love … (likely I love NLP or similar in training) and used it to predict "love".
p(<unk> | love …) = [OOV] 0 [-inf]: The word "speech" was not in our training vocabulary. SRILM replaces it with <unk>.

Example 2: The OOV ChainLook at the second sentence: Perplexity is a common metric…

Perplexity becomes <unk>. Log prob: [-inf]
common becomes <unk>. Log prob: [-inf]
metric becomes <unk>. Log prob: [-inf]
Notice what happens to the word is: p(is | <unk> …) = [1gram] 0.01481481 [-1.829304] Because <unk> is never appeared in the training data, the Bigram failed. The Trigram failed. It backed off all the way to the Unigram probability of "is"!

0.9 The "Magic" of SRILM's PPL Math

You see a lot of [-inf] in that log. If you add -infinity to anything, the total becomes -infinity, and Perplexity becomes Infinity. But look at the final math at the bottom:logprob= -18.98879 ppl= 9.986423

How did SRILM avoid Infinity? Look at the denominator it used: 3 sentences, 24 words, 8 OOVs.SRILM calculates PPL as: 10^(-logprob / (words - OOVs + sentences))

Numerator: -18.98879
Denominator: 24 - 8 + 3 = 19 (It completely ignores the 8 OOV words!)

By excluding OOVs from the denominator and skipping their -inf scores from the numerator, SRILM gives the LM the "benefit of the doubt." It only judges the model on the words it actually had a chance to learn.

0.10 Extracting Raw N-gram Counts

[56]:
```
# We can tell ngram-count to JUST count, and NOT estimate a language model
!ngram-count -order 3 -text lm_toy.train.txt -write lm_toy.raw_counts␣
↪-write-vocab lm_toy.vocab
```

[58]:
```
!ls
```

```
lm_toy_3gram.arpa      lm_toy_srilm.arpa  lm_toy.txt      tmp_kenlm
lm_toy_3gram.arpa.err  lm_toy.test.txt    lm_toy.val.txt
lm_toy.raw_counts      lm_toy.train.txt   lm_toy.vocab
```

[57]:
```
!wc lm_toy.raw_counts
```

```
 384 1194 6200 lm_toy.raw_counts
```

[70]:
```
!wc lm_toy.vocab
```

```
103 103 757 lm_toy.vocab
```

[58]:
```
!head -20 lm_toy.raw_counts
```

```
stands  1
stands for      1
stands for natural      1
<s>     19
<s> Neural      1
<s> Neural language     1
<s> Cross       1
<s> Cross entropy       1
<s> Large       1
<s> Large language      1
<s> Long        1
<s> Long short  1
<s> Fine        1
<s> Fine tuning 1
```

```
<s> I   2
<s> I love      2
<s> NLP 1
<s> NLP stands  1
<s> Kneser      1
<s> Kneser Ney  1
```

[59]:
```
# Look at the vocabulary size
!echo "--- Vocabulary ---"
!cat lm_toy.vocab
```

```
--- Vocabulary ---
-pau-
.
</s>
<s>
<unk>
Attention
BERT
Cross
Deep
Fine
GPT
I
Kneser
Language
Large
Long
NLP
Neural
Ney
Retrieval
Scaling
Smoothing
Speech
Transformers
a
adapts
all
amounts
and
are
art
augmented
autoregressive
between
changed
continuous
```

data
deep
dictate
difference
distributions
downstream
entropy
for
forever
fun
generation
gradient
hallucinations
handle
help
helps
improved
in
is
language
laws
learning
love
masked
massive
measures
memory
model
models
n-gram
natural
need
needs
networks
of
performance
pretrained
probability
problem
processing
recognition
reduce
relationship
representations
require
short
significantly
size

```
smoothing
solved
specific
stands
state
statistical
tasks
techniques
term
the
to
training
tuning
two
unseen
use
vanishing
words
you
```

[60]:
```
# Look at the top 20 most frequent N-grams overall
# (In SRILM raw counts, unigrams have NO spaces, bigrams have 1 space, trigrams␣
↪have 2 spaces)
!echo "--- Top 20 N-grams (Sorted by Count) ---"
!sort -nr -k2 lm_toy.raw_counts | head -n 20
```

```
--- Top 20 N-grams (Sorted by Count) ---
<s>     19
</s>    19
.       19
models  10
language        8
the     5
of      3
NLP     3
is      3
words   2
model   2
love    2
learning        2
I       2
for     2
between 2
are     2
you     1
vanishing       1
use     1
```

0.11 Raw Counts vs. Smoothed Probabilities (The Root of the Problem)

Let's analyze the raw counts file.

We have 384 lines of N-grams extracted from just ~20 sentences.
Our vocabulary size is 102 words.
Look at the Top 20 N-grams from our sorted output. What is the highest count? <s> (sentence start) has a count of 19. The highest bigram is <s> I with a count of 2.

Almost every other bigram and trigram in this entire file has a count of exactly 1.

The Maximum Likelihood Estimation (MLE) Trap
If we didn't use smoothing (like Witten-Bell or Kneser-Ney), we would calculate probabilities using pure MLE: `P(word | context) = Count(context, word) / Count(context)`

If the test set asks for the probability of the bigram language models, MLE finds a count of 1. But what if the test set asks for the bigram models assign? It's not in this file (count is 0). MLE gives it a probability of exactly 0.0.

If even one word in a sentence gets a probability of 0, the entire sentence probability becomes 0. The log probability becomes -infinity. The Perplexity becomes Infinity. The language model is broken.

Do you remember why the word is had to back off to a [1gram] in our debug output? It was because the bigram <unk> is had a count of 0 in this exact raw counts file!

0.12 Vocabulary Management (change-lm-vocab)

This is the most important practical tool. In real ASR/MT, you cannot have OOVs because your decoder dictionary is fixed. You must "force" the LM to match your dictionary.

[61]:
```
%%bash
# Let's artificially create a restricted vocabulary (e.g., an ASR pronunciation␣
↪dictionary)
# We will remove some words that might appear in the test set
echo "Creating a restricted vocabulary..."
cat > lm_toy.restricted.vocab << EOF
<unk>
<s>
</s>
.
language
models
are
fun
I
love
NLP
EOF
```

```
Creating a restricted vocabulary…
```

[62]:
```
!ls lm_toy.restricted.vocab
```

```
lm_toy.restricted.vocab
```

[63]:
```
# change-lm-vocab re-maps the LM to ONLY use words in this list
# Any word not in the list becomes <unk>
!/home/ye/tool/SRILM/bin/change-lm-vocab -vocab lm_toy.restricted.vocab -lm␣
 ↪lm_toy_srilm.arpa -write-lm lm_toy.restricted.arpa

!echo "\n--- Evaluating the Restricted LM ---"
# Notice the OOV count now!
!ngram -lm lm_toy.restricted.arpa -ppl lm_toy.test.txt
```

```
-: line 38: warning: 10 1-grams read, expected 101
-: line 38: warning: 12 2-grams read, expected 140
\n--- Evaluating the Restricted LM ---
file lm_toy.test.txt: 3 sentences, 24 words, 16 OOVs
0 zeroprobs, logprob= -5.667527 ppl= 3.275139 ppl1= 5.110236
```

0.13 gawk Installation

အထက်ပါအတိုင်း gawk **က စက်ထဲမှာ မရှိလို့** terminal **ထွက်ပြီး** installation **လုပ်ခဲ့တယ်။**

```
ye@lst-hpc3090:/mnt/disk1/ye/exp/LM-Tutorial$ sudo apt-get install gawk
[sudo] password for ye:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libsigsegv2
Suggested packages:
  gawk-doc
The following NEW packages will be installed:
  gawk libsigsegv2
0 upgraded, 2 newly installed, 0 to remove and 95 not upgraded.
Need to get 478 kB of archives.
After this operation, 1,826 kB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 libsigsegv2 amd64 2.14-1ubuntu2 [15.0 k
Get:2 http://archive.ubuntu.com/ubuntu noble/main amd64 gawk amd64 1:5.2.1-2build3 [463 kB]
Fetched 478 kB in 2s (256 kB/s)
Selecting previously unselected package libsigsegv2:amd64.
(Reading database ... 425560 files and directories currently installed.)
Preparing to unpack .../libsigsegv2_2.14-1ubuntu2_amd64.deb ...
Unpacking libsigsegv2:amd64 (2.14-1ubuntu2) ...
Setting up libsigsegv2:amd64 (2.14-1ubuntu2) ...
Selecting previously unselected package gawk.
(Reading database ... 425567 files and directories currently installed.)
Preparing to unpack .../gawk_1%3a5.2.1-2build3_amd64.deb ...
```

```
Unpacking gawk (1:5.2.1-2build3) ...
Setting up gawk (1:5.2.1-2build3) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
ye@lst-hpc3090:/mnt/disk1/ye/exp/LM-Tutorial$
```

[75]:
```
# change-lm-vocab re-maps the LM to ONLY use words in this list
# Any word not in the list becomes <unk>
!/home/ye/tool/SRILM/bin/change-lm-vocab -vocab lm_toy.restricted.vocab -lm␣
↪lm_toy_srilm.arpa -write-lm lm_toy.restricted.arpa

!echo "\n--- Evaluating the Restricted LM ---"
# Notice the OOV count now!
!ngram -lm lm_toy.restricted.arpa -ppl lm_toy.test.txt
```

```
-: line 38: warning: 10 1-grams read, expected 101
-: line 38: warning: 12 2-grams read, expected 140
\n--- Evaluating the Restricted LM ---
file lm_toy.test.txt: 3 sentences, 24 words, 16 OOVs
0 zeroprobs, logprob= -5.667527 ppl= 3.275139 ppl1= 5.110236
```

0.14 Solving the OOV Problem (And Exposing a Metric Loophole)

Compare the output of this evaluation to your original one.

Original: 8 OOVs, logprob= -18.98, ppl= 9.986
Restricted: 16 OOVs, logprob= -5.66, ppl= 3.275

Wait, the PPL went down significantly (9.98 -> 3.27). Does that mean our restricted model is actually better at predicting text?

Absolutely not. This is a mathematical illusion. Let's look at the math to see why SRILM is lying to us.

0.15 The Loophole in SRILM's Math

Remember the SRILM PPL formula: PPL = 10^(-logprob / (words - OOVs + sentences))

Let's calculate the denominators for both:

Original Denominator: 24 words - 8 OOVs + 3 sentences = 19
Restricted Denominator: 24 words - 16 OOVs + 3 sentences = 11
Now look at the logprob (the numerator). SRILM completely ignores OOV words when calculating logprob. They contribute 0 to the sum.

In the restricted test set, 16 out of 24 words are OOVs!
That means SRILM is only doing math for 8 words total (like "I", "love", "language", "models"). Because those 8 remaining words are very common, their probabilities are very high, resulting in a very small logprob (-5.66).

The Illusion: By stripping out vocabulary, we drastically shrank both the numerator and the denominator. But because the numerator dropped faster than the denominator, the final PPL

fraction artificially decreased.

Analogy: Imagine a teacher gives a 20-question test, but a student only answers 5 questions (all correctly) and leaves 15 blank. If the teacher's grading formula ignores the blank questions entirely, the student looks like a genius with a perfect score, but in reality, they failed the test because they couldn't answer 75% of the material.

0.16 Why do we still do this in the real world?

If change-lm-vocab breaks the PPL metric, why is it the most important tool in Speech Recognition (Kaldi, Whisper)?

Because in ASR, vocabulary alignment is mandatory. Your acoustic model can only output sounds that exist in your pronunciation dictionary. If "Transformers" isn't in the dictionary, the acoustic model will physically never output that word.

If your Language Model contains "Transformers" but your acoustic model cannot output it, their scores are misaligned during decoding, and the system crashes or produces garbage. You must use change-lm-vocab to strip the LM down to match the dictionary exactly.

We accept the "broken" PPL metric because the alternative (decoder crashes) is worse. In real ASR pipelines, we evaluate the final Word Error Rate (WER) of the audio transcription, not the isolated PPL of the LM!

0.17 The "Big LM" Pipeline (Conceptual)

"What if I have 100GB of text?" You cannot run standard ngram-count on 100GB of text, it will run out of RAM. This cell introduces the "MapReduce-style" tools.

```
# STEP 1: Split the counting into smaller chunks (batches)
# This uses disk instead of RAM to avoid Out-Of-Memory crashes
make-batch-counts -n 3 -order 5 -text massive_corpus.txt -dir batch_counts/

# STEP 2: Merge all the batch counts back together
merge-batch-counts -dir batch_counts/ -write massive_counts.gz

# STEP 3: Estimate the LM from the merged counts
# (make-big-lm is a wrapper that handles KN smoothing math for huge files)
make-big-lm -read massive_counts.gz -lm massive_5gram.arpa
```

0.18 Scaling to Web-Scale Data

Look at the tools in your SRILM folder (e.g. /home/ye/tool/SRILM/bin/): make-batch-counts, merge-batch-counts, make-big-lm.

Standard ngram-count loads all N-grams into RAM. A 5-gram model on 10B words requires ~128GB of RAM. SRILM solved this in the early 2000s using a "MapReduce-like" approach before Hadoop existed:

1. make-batch-counts: Splits the text into chunks, counts N-grams in RAM, writes to disk.

2. merge-batch-counts: Does a disk-based merge sort of all the chunk counts.

3. make-big-lm: Reads the sorted counts in a single stream and calculates the complex Kneser-Ney discounts.

This is why SRILM was the king of N-grams for 20 years. KenLM later made this even faster with better C++ multi-threading, but the architecture is the same.

0.19 LM Interpolation (compute-best-mix)

In the real world, you never use just ONE LM. You mix a Domain LM (e.g., Medical terms) with a General LM (e.g., General English).

```
# To do this properly, we need two LMs.
# Let's fake a "General" LM by training on just one sentence.
echo "This is a general sentence about the world ." > general.txt
!ngram-count -wbdiscount -order 3 -text general.txt -lm general.arpa

# Now we find the optimal mathematical weight (lambda) to mix them
# compute-best-mix reads the debug logs of both LMs on a tuning set to find the exact weight
!ngram -lm lm_toy_srilm.arpa -ppl lm_toy.test.txt -debug 2 > domain.log
!ngram -lm general.arpa -ppl lm_toy.test.txt -debug 2 > general.log

echo "--- Computing optimal interpolation weights ---"
!compute-best-mix domain.log general.log
```

လက်တွေ့ run ကြည့်ရအောင်။

[65]:
```
!echo "This is a general sentence about the world ." > general.txt
!ngram-count -wbdiscount -order 3 -text general.txt -lm general.arpa
```

[66]:
```
!cat general.arpa
```

```
\data\
ngram 1=11
ngram 2=10
ngram 3=0

\1-grams:
-1      .       -0.2552725
-1      </s>
-99     <s>     -0.2552725
-1      This    -0.2552725
-1      a       -0.2552725
-1      about   -0.2552725
-1      general -0.2552725
-1      is      -0.2552725
-1      sentence        -0.2552725
-1      the     -0.2552725
```

```
-1      world   -0.2552725

\2-grams:
-0.30103        . </s>
-0.30103        <s> This
-0.30103        This is
-0.30103        a general
-0.30103        about the
-0.30103        general sentence
-0.30103        is a
-0.30103        sentence about
-0.30103        the world
-0.30103        world .

\3-grams:

\end\
```

[67]:
```
# Now we find the optimal mathematical weight (lambda) to mix them
# compute-best-mix reads the debug logs of both LMs on a tuning set to find the␣
↪exact weight
!ngram -lm lm_toy_srilm.arpa -ppl lm_toy.test.txt -debug 2 > domain.log
!ngram -lm general.arpa -ppl lm_toy.test.txt -debug 2 > general.log
```

```
reading 101 1-grams
reading 140 2-grams
reading 3 3-grams
reading 11 1-grams
reading 10 2-grams
reading 0 3-grams
```

[69]:
```
!cat domain.log
```

```
I love speech recognition .
        p( I | <s> )    = [2gram] 0.05405402 [ -1.267172 ]
        p( love | I …)        = [3gram] 0.6666666 [ -0.1760913 ]
        p( <unk> | love …)    = [OOV] 0 [ -inf ]
        p( recognition | <unk> …)     = [1gram] 0.007407405 [ -2.130334 ]
        p( . | recognition …)         = [1gram] 0.03731341 [ -1.428135 ]
        p( </s> | . …)        = [2gram] 0.95 [ -0.02227639 ]
1 sentences, 5 words, 1 OOVs
0 zeroprobs, logprob= -5.024009 ppl= 10.11118 ppl1= 18.03027

Perplexity is a common metric for evaluating language models .
        p( <unk> | <s> )        = [OOV] 0 [ -inf ]
        p( is | <unk> …)      = [1gram] 0.01481481 [ -1.829304 ]
        p( a | is …)  = [2gram] 0.1666667 [ -0.7781513 ]
        p( <unk> | a …)       = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
```

```
        p( for | <unk> …)     = [1gram] 0.01111112 [ -1.954242 ]
        p( <unk> | for …)     = [OOV] 0 [ -inf ]
        p( language | <unk> …)        = [1gram] 0.03333335 [ -1.477121 ]
        p( models | language …)       = [2gram] 0.5454546 [ -0.2632414 ]
        p( . | models …)      = [3gram] 0.4444445 [ -0.3521825 ]
        p( </s> | . …)        = [3gram] 0.8333333 [ -0.07918125 ]
1 sentences, 10 words, 4 OOVs
0 zeroprobs, logprob= -6.733423 ppl= 9.160467 ppl1= 13.25065

Language models assign probabilities to sequences of words .
        p( Language | <s> )     = [2gram] 0.02702701 [ -1.568202 ]
        p( models | Language …)       = [2gram] 0.5 [ -0.30103 ]
        p( <unk> | models …)  = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( to | <unk> …)      = [1gram] 0.007407405 [ -2.130334 ]
        p( <unk> | to …)      = [OOV] 0 [ -inf ]
        p( of | <unk> …)      = [1gram] 0.01481481 [ -1.829304 ]
        p( words | of …)      = [2gram] 0.1666667 [ -0.7781513 ]
        p( . | words …)       = [2gram] 0.25 [ -0.60206 ]
        p( </s> | . …)        = [2gram] 0.95 [ -0.02227639 ]
1 sentences, 9 words, 3 OOVs
0 zeroprobs, logprob= -7.231358 ppl= 10.79074 ppl1= 16.04081

file lm_toy.test.txt: 3 sentences, 24 words, 8 OOVs
0 zeroprobs, logprob= -18.98879 ppl= 9.986423 ppl1= 15.37444
```

[70]:
```
!cat general.log
```

```
I love speech recognition .
        p( <unk> | <s> )        = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( . | <unk> …)       = [1gram] 0.1 [ -1 ]
        p( </s> | . …)        = [2gram] 0.5 [ -0.30103 ]
1 sentences, 5 words, 4 OOVs
0 zeroprobs, logprob= -1.30103 ppl= 4.472136 ppl1= 20

Perplexity is a common metric for evaluating language models .
        p( <unk> | <s> )        = [OOV] 0 [ -inf ]
        p( is | <unk> …)      = [1gram] 0.1 [ -1 ]
        p( a | is …)  = [2gram] 0.5 [ -0.30103 ]
        p( <unk> | a …)       = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
```

```
        p( . | <unk> …)       = [1gram] 0.1 [ -1 ]
        p( </s> | . …)        = [2gram] 0.5 [ -0.30103 ]
1 sentences, 10 words, 7 OOVs
0 zeroprobs, logprob= -2.60206 ppl= 4.472136 ppl1= 7.368063

Language models assign probabilities to sequences of words .
        p( <unk> | <s> )        = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( <unk> | <unk> …)   = [OOV] 0 [ -inf ]
        p( . | <unk> …)       = [1gram] 0.1 [ -1 ]
        p( </s> | . …)        = [2gram] 0.5 [ -0.30103 ]
1 sentences, 9 words, 8 OOVs
0 zeroprobs, logprob= -1.30103 ppl= 4.472136 ppl1= 20

file lm_toy.test.txt: 3 sentences, 24 words, 19 OOVs
0 zeroprobs, logprob= -5.20412 ppl= 4.472136 ppl1= 10.98561
```

[71]:
```
!echo "--- Computing optimal interpolation weights ---"
!compute-best-mix domain.log general.log
```

```
--- Computing optimal interpolation weights ---
iteration 1, lambda = (0.5 0.5), ppl = 14.1176
iteration 2, lambda = (0.795617 0.204383), ppl = 10.9167
iteration 3, lambda = (0.894906 0.105094), ppl = 10.356
iteration 4, lambda = (0.937108 0.0628916), ppl = 10.1782
iteration 5, lambda = (0.958959 0.0410407), ppl = 10.1009
iteration 6, lambda = (0.971775 0.0282248), ppl = 10.0607
iteration 7, lambda = (0.979921 0.020079), ppl = 10.0372
iteration 8, lambda = (0.985386 0.0146145), ppl = 10.0223
iteration 9, lambda = (0.989191 0.0108085), ppl = 10.0124
iteration 10, lambda = (0.991914 0.00808586), ppl = 10.0056
iteration 11, lambda = (0.9939 0.00609991), ppl = 10.0007
iteration 12, lambda = (0.99537 0.00463035), ppl = 9.99717
iteration 13, lambda = (0.996469 0.00353118), ppl = 9.99456
19 non-oov words, best lambda (0.997298 0.00270239)
pairwise cumulative lambda (1 0.00270239)
```

0.20 Interpolation: The Secret Sauce of ASR

compute-best-mix outputs a number like 0.95.
This means: P(final_word) = 0.95 * P(domain_LM) + 0.05 * P(general_LM)

Why do we do this?
If you build an LM for Medical Transcription, it will be great at words like "acetaminophen", but

terrible at common words like "going to". By interpolating a General English LM (trained on Wikipedia) with your Medical LM, you get the best of both worlds. SRILM's compute-best-mix uses Expectation-Maximization (EM) on a tuning set to find the exact mathematical multiplier that minimizes Perplexity.

0.21 Neural LM (PyTorch LSTM on GPU)

Karpathy's numpy RNN is great for math, but since we have an RTX 3090, let's train a PyTorch LSTM on the GPU to show modern workflows.

[85]:
```
!nvidia-smi
```

```
Fri May  1 17:08:14 2026
+-----------------------------------------------------------------------------
--------+
| NVIDIA-SMI 535.288.01             Driver Version: 535.288.01   CUDA Version:
12.2     |
|-----------------------------------------+----------------------+-------------
--------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile
Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util
Compute M. |
|                                         |                      |
MIG M. |
|=========================================+======================+=============
========|
|   0  NVIDIA GeForce RTX 3090 Ti     Off | 00000000:01:00.0  On |
Off |
| 31%   55C    P2             121W / 480W |    755MiB / 24564MiB |      0%
Default |
|                                         |                      |
N/A |
+-----------------------------------------+----------------------+-------------
--------+

+-----------------------------------------------------------------------------
--------+
| Processes:
|
|  GPU   GI   CI        PID   Type   Process name                          GPU
Memory |
|        ID   ID
Usage      |
|=============================================================================
========|
|    0   N/A  N/A     51185      G   /usr/lib/xorg/Xorg
188MiB |
```

```
|    0   N/A  N/A     51389      G   /usr/bin/gnome-shell
67MiB |
|    0   N/A  N/A     52003      G   /usr/libexec/xdg-desktop-portal-gnome
6MiB |
|    0   N/A  N/A     52268      C   /usr/NX/bin/nxnode.bin
276MiB |
|    0   N/A  N/A    300604      G   …1/usr/bin/snapd-desktop-integration
16MiB |
|    0   N/A  N/A    479401      G   /usr/bin/nautilus
32MiB |
|    0   N/A  N/A   1780390      G   …ads/tsetup.5.10.7/Telegram/Telegram
40MiB |
|    0   N/A  N/A   3125725      G   …n=20260425-030030.652000-production
60MiB |
|    0   N/A  N/A   3138305      G   …irefox/8191/usr/lib/firefox/firefox
20MiB |
+-----------------------------------------------------------------------------
--------+
```

[ ]:
```
#!pip install -q torch torchvision
```

0.22 Preparing Bigger Dataset

myPOS ကိုပဲ သုံးကြရအောင်။

```
ye@lst-hpc3090:/mnt/disk1/ye/exp/LM-Tutorial/data$ cp /home/ye/exp/SCRDR_tutorial/tokenizer/dat
ye@lst-hpc3090:/mnt/disk1/ye/exp/LM-Tutorial/data$ cp /home/ye/exp/SCRDR_tutorial/tokenizer/dat
ye@lst-hpc3090:/mnt/disk1/ye/exp/LM-Tutorial/data$ ls
10k_test.txt  mypos_v3.word
ye@lst-hpc3090:/mnt/disk1/ye/exp/LM-Tutorial/data$
```

[89]:
```
%cd /mnt/disk1/ye/exp/LM-Tutorial/data
```

```
/mnt/disk1/ye/exp/LM-Tutorial/data
```

[90]:
```
!wc *
```

```
  10000  117857 1686379 10k_test.txt
  43196  564517 7518832 mypos_v3.word
  53196  682374 9205211 total
```

[91]:
```
!head mypos_v3.word
```

```
၁၉၆၂ ခုနှစ် ခန့်မှန်း သန်းခေါင်စာရင်း အရ လူဦးရေ ၁၁၅၉၃၁ ယောက် ရှိ သည် ။
လူ တိုင်း တွင် သင့်မြတ် လျော်ကန် စွာ ကန့်သတ် ထား သည့် အလုပ် လုပ် ချိန် အပြင် ၊
လစာ နှင့်တကွ အခါ ကာလ အားလျော်စွာ သတ်မှတ် ထား သည့် အလုပ် အားလပ်ရက် များ ပါဝင်
သည့် အနားယူခွင့် နှင့် အားလပ်ခွင့် ခံစားပိုင်ခွင့် ရှိ သည် ။
ဤ နည်း ကို စစ်ယူ သော နည်း ဟု ခေါ် သည် ။
စာပြန်ပွဲ ဆို တာ က အာဂုံဆောင် အလွတ်ကျက် ထား တဲ့ ပိဋ္ဌကတ်သုံးပုံ စာပေ တွေ ကို စာစစ်
```

သံယာတော်ကြီး တွေ ရဲ့ ရှေ့ မှာ အလွတ် ပြန် ပြီး ရွတ်ပြ ရ တာ ပေါ့ ။
ဒီ မှာ ကျွန်တော့် သက်သေခံကတ် ပါ ။
၂၀ ရာစု မြန်မာ့ သမိုင်း သန်းဝင်းလှိုင် ၊ ၂၀၀၉ ခု ၊ မေ လ ၊ ကံကော်ဝတ်ရည် စာပေ ။
ကျွန်တော် မျက်မှန် တစ် လက် လုပ် ချင် ပါ တယ် ။
ကျွန်တော် တို့ က ဒီ အမှု ရဲ့ ကြံရာပါ ကို ဖမ်းမိ ဖို့ ကြိုးစား ခဲ့ တယ် ။
ကလေး မီးဖွား ဖို့ ခန့်မှန်း ရက် က ဘယ်တော့ ပါ လဲ ။
အရိုးရှင်းဆုံး ကာဗိုဟိုက်ဒရိတ် မှာ ဂလူးကို့စ် ၊ ဂလက်တို့စ် ၊ ဖရပ်တို့စ် စသည့်
မိုနိုဆက်ကရိုက် များ ဖြစ် သည် ။

[92]:
```
!head 10k_test.txt
```

၁၉၆၂ ခုနှစ် ခန့်မှန်း သန်းခေါင်စာရင်း အရ လူဦးရေ ၁၁၅၉၃၁ ယောက် ရှိ သည်
လူ တိုင်း တွင် သင့်မြတ် လျော်ကန် စွာ ကန့်သတ် ထား သည့် အလုပ် လုပ် ချိန် အပြင် လစာ
နှင့်တကွ အခါ ကာလ အားလျော်စွာ သတ်မှတ် ထား သည့် အလုပ် အားလပ်ရက် များ ပါဝင် သည့်
အနားယူခွင့် နှင့် အားလပ်ခွင့် ခံစားပိုင်ခွင့် ရှိ သည်
ဤ နည်း ကို စစ်ယူ သော နည်း ဟု ခေါ် သည်
စာပြန်ပွဲ ဆို တာ က အာဂုံဆောင် အလွတ်ကျက် ထား တဲ့ ပိဋ္ဌကတ်သုံးပုံ စာပေ တွေ ကို စာစစ်
သံယာတော်ကြီး တွေ ရဲ့ ရှေ့ မှာ အလွတ် ပြန် ပြီး ရွတ်ပြ ရ တာ ပေါ့
ဒီ မှာ ကျွန်တော့် သက်သေခံကတ် ပါ
၂၀ ရာစု မြန်မာ့ သမိုင်း သန်းဝင်းလှိုင် ၂၀၀၉ ခု မေ လ ကံကော်ဝတ်ရည် စာပေ
ကျွန်တော် မျက်မှန် တစ် လက် လုပ် ချင် ပါ တယ်
ကျွန်တော် တို့ က ဒီ အမှု ရဲ့ ကြံရာပါ ကို ဖမ်းမိ ဖို့ ကြိုးစား ခဲ့ တယ်
ကလေး မီးဖွား ဖို့ ခန့်မှန်း ရက် က ဘယ်တော့ ပါ လဲ
အရိုးရှင်းဆုံး ကာဗိုဟိုက်ဒရိတ် မှာ ဂလူးကို့စ် ဂလက်တို့စ် ဖရပ်တို့စ် စသည့်
မိုနိုဆက်ကရိုက် များ ဖြစ် သည်

0.23 Open-test

```
ye@lst-hpc3090:/mnt/disk1/ye/exp/LM-Tutorial/data$ cp /home/ye/exp/SCRDR_tutorial/tagger/pos_da
ye@lst-hpc3090:/mnt/disk1/ye/exp/LM-Tutorial/data$
```

[97]:
```
!wc /home/ye/exp/SCRDR_tutorial/tagger/pos_data/otest.word
```

```
  1000  13468 180788 /home/ye/exp/SCRDR_tutorial/tagger/pos_data/otest.word
```

[98]:
```
!head /home/ye/exp/SCRDR_tutorial/tagger/pos_data/otest.word
```

တစ် ကိုက် ကို ဝမ် ခုနှစ်ထောင် ပါ ။
မနှစ် က သူ ကျွန်မ ကို သင် ပေး တယ် ။
ကျွန်တော့် ခုံ သွား ရှာ မလို့ ။
အတန်း စ တာ ကြာ ပြီ လား ။
ဆေး နည်းနည်း စား လိုက် ၊ သုံး လေး ရက် လောက် အနားယူ လိုက် ရင် ပျောက် သွား မှာ ပါ
။
အေးချမ်း မှု နဲ့ စည်းကမ်း ကို တည်မြဲ အောင် ထိန်းသိမ်း သည် ။
ဌန်း ကို လိုအပ် တယ် ။

ဘွဲ့ ရ ရင် ဘာ လုပ် မ လို့ လဲ ။
ကျွန်တော် ချောင်းဆိုး ခြင်း အတွက် တစ် ခု ခု လို ချင် တယ် ။
အသီးအနှံ တို့ မှ လွဲ လျှင် လူ တို့ ၏ အဓိက အစားအစာ မှာ ငါး ဖြစ် သည် ။

0.24 Simple Data Cleaning

LM မဆောက်ခင် ပုဒ်ထီး၊ ပုဒ်မတွေ ဖြုတ်ရအောင်။
တကယ်ကတော့ ရှင်းမယ်ဆိုရင် တခြား punctuation mark တွေ အင်္ဂလိပ်စာတွေပါ ရှင်းသင့်တယ်။

[103]:
```
!cat /mnt/disk1/ye/exp/LM-Tutorial/data/clean_text.py
```

```
import argparse
import string

def remove_punctuation(input_path, output_path):
    # Define English punctuation from the string module
    english_punct = string.punctuation

    # Define Burmese specific punctuation
    burmese_punct = "၊။"

    # Combine them into one set of characters to remove
    all_to_remove = english_punct + burmese_punct

    # Create a translation table for fast processing
    # This maps every character in 'all_to_remove' to None
    table = str.maketrans('', '', all_to_remove)

    try:
        with open(input_path, 'r', encoding='utf-8') as infile:
            with open(output_path, 'w', encoding='utf-8') as outfile:
                for line in infile:
                    # Apply the translation table to each line
                    clean_line = line.translate(table)
                    outfile.write(clean_line)

        print(f"Success! Cleaned text saved to: {output_path}")

    except FileNotFoundError:
        print(f"Error: The file '{input_path}' was not found.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Remove English and Burmese
punctuation from a text file.")

    # Setting up --input/-i and --output/-o
```

```
    parser.add_argument('-i', '--input', required=True, help="Path to the source
text file.")
    parser.add_argument('-o', '--output', required=True, help="Path where the
cleaned file will be saved.")

    args = parser.parse_args()

    remove_punctuation(args.input, args.output)
```

[104]:
```
%cd /mnt/disk1/ye/exp/LM-Tutorial/data
```

```
/mnt/disk1/ye/exp/LM-Tutorial/data
```

[105]:
```
!python ./clean_text.py --input ./mypos_v3.word --output ./mypos_v3.word.clean
```

```
Success! Cleaned text saved to: ./mypos_v3.word.clean
```

[106]:
```
!head ./mypos_v3.word.clean
```

```
၁၉၆၂ ခုနှစ် ခန့်မှန်း သန်းခေါင်စာရင်း အရ လူဦးရေ ၁၁၅၉၃၁ ယောက် ရှိ သည်
လူ တိုင်း တွင် သင့်မြတ် လျော်ကန် စွာ ကန့်သတ် ထား သည့် အလုပ် လုပ် ချိန် အပြင်
လစာ နှင့်တကွ အခါ ကာလ အားလျော်စွာ သတ်မှတ် ထား သည့် အလုပ် အားလပ်ရက် များ ပါဝင်
သည့် အနားယူခွင့် နှင့် အားလပ်ခွင့် ခံစားပိုင်ခွင့် ရှိ သည်
ဤ နည်း ကို စစ်ယူ သော နည်း ဟု ခေါ် သည်
စာပြန်ပွဲ ဆို တာ က အာဂုံဆောင် အလွတ်ကျက် ထား တဲ့ ပိဋ္ဌကတ်သုံးပုံ စာပေ တွေ ကို စာစစ်
သံဃာတော်ကြီး တွေ ရဲ့ ရှေ့ မှာ အလွတ် ပြန် ပြီး ရွတ်ပြ ရ တာ ပေါ့
ဒီ မှာ ကျွန်တော့် သက်သေခံကတ် ပါ
၂၀ ရာစု မြန်မာ့ သမိုင်း သန်းဝင်းလှိုင် ၂၀၀၉ ခု မေ လ ကံကော်ဝတ်ရည် စာပေ
ကျွန်တော် မျက်မှန် တစ် လက် လုပ် ချင် ပါ တယ်
ကျွန်တော် တို့ က ဒီ အမှု ရဲ့ ကြံရာပါ ကို ဖမ်းမိ ဖို့ ကြိုးစား ခဲ့ တယ်
ကလေး မီးဖွား ဖို့ ခန့်မှန်း ရက် က ဘယ်တော့ ပါ လဲ
အရိုးရှင်းဆုံး ကာဗိုဟိုက်ဒရိတ် မှာ ဂလူးကို့စ် ဂလက်တို့စ် ဖရပ်တို့စ် စသည့်
မိုနိုဆက်ကရိုက် များ ဖြစ် သည်
```

[107]:
```
!python ./clean_text.py --input ./10k_test.txt --output ./10k_test.txt.clean
```

```
Success! Cleaned text saved to: ./10k_test.txt.clean
```

[108]:
```
!head ./10k_test.txt.clean
```

```
၁၉၆၂ ခုနှစ် ခန့်မှန်း သန်းခေါင်စာရင်း အရ လူဦးရေ ၁၁၅၉၃၁ ယောက် ရှိ သည်
လူ တိုင်း တွင် သင့်မြတ် လျော်ကန် စွာ ကန့်သတ် ထား သည့် အလုပ် လုပ် ချိန် အပြင် လစာ
နှင့်တကွ အခါ ကာလ အားလျော်စွာ သတ်မှတ် ထား သည့် အလုပ် အားလပ်ရက် များ ပါဝင် သည့်
အနားယူခွင့် နှင့် အားလပ်ခွင့် ခံစားပိုင်ခွင့် ရှိ သည်
ဤ နည်း ကို စစ်ယူ သော နည်း ဟု ခေါ် သည်
စာပြန်ပွဲ ဆို တာ က အာဂုံဆောင် အလွတ်ကျက် ထား တဲ့ ပိဋ္ဌကတ်သုံးပုံ စာပေ တွေ ကို စာစစ်
```

သံယာတော်ကြီး တွေ ရဲ့ ရှေ့ မှာ အလွတ် ပြန် ပြီး ရွတ်ပြ ရ တာ ပေါ့
ဒီ မှာ ကျွန်တော့် သက်သေခံကတ် ပါ
၂၀ ရာစု မြန်မာ့ သမိုင်း သန်းဝင်းလှိုင် ၂၀၀၉ ခု မေ လ ကံကော်ဝတ်ရည် စာပေ
ကျွန်တော် မျက်မှန် တစ် လက် လုပ် ချင် ပါ တယ်
ကျွန်တော် တို့ က ဒီ အမှု ရဲ့ ကြံရာပါ ကို ဖမ်းမိ ဖို့ ကြိုးစား ခဲ့ တယ်
ကလေး မီးဖွား ဖို့ ခန့်မှန်း ရက် က ဘယ်တော့ ပါ လဲ
အရိုးရှင်းဆုံး ကာဗိုဟိုက်ဒရိတ် မှာ ဂလူးကို့စ် ဂလက်တို့စ် ဖရပ်တို့စ် စသည့်
မိုနိုဆက်ကရိုက် များ ဖြစ် သည်

[110]:
```
!python ./clean_text.py --input ./otest.word --output ./otest.word.clean
```

```
Success! Cleaned text saved to: ./otest.word.clean
```

[113]:
```
!head ./otest.word.clean
```

```
တစ် ကိုက် ကို ဝမ် ခုနှစ်ထောင် ပါ
မနှစ် က သူ ကျွန်မ ကို သင် ပေး တယ်
ကျွန်တော့် ခုံ သွား ရှာ မလို့
အတန်း စ တာ ကြာ ပြီ လား
ဆေး နည်းနည်း စား လိုက် သုံး လေး ရက် လောက် အနားယူ လိုက် ရင် ပျောက် သွား မှာ ပါ
အေးချမ်း မှု နဲ့ စည်းကမ်း ကို တည်မြဲ အောင် ထိန်းသိမ်း သည်
ဌန်း ကို လိုအပ် တယ်
ဘွဲ့ ရ ရင် ဘာ လုပ် မ လို့ လဲ
ကျွန်တော် ချောင်းဆိုး ခြင်း အတွက် တစ် ခု ခု လို ချင် တယ်
အသီးအနှံ တို့ မှ လွဲ လျှင် လူ တို့ ၏ အဓိက အစားအစာ မှာ ငါး ဖြစ် သည်
```

0.25 LSTM Based LM

Code ကို လေ့လာကြည့်ပါ။

[72]:
```
%cd /mnt/disk1/ye/exp/LM-Tutorial/lstm
```

```
/mnt/disk1/ye/exp/LM-Tutorial/lstm
```

[73]:
```
!cat lstm_lm.py
```

```
#!/usr/bin/env python3
"""
A complete, standalone LSTM Language Model with Train, Test, and Generate modes.
Specifically designed to handle OOVs safely and support Character-level modeling
(ideal for Myanmar).
"""

import os
import math
import json
import argparse
```

```
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

# ==========================================
# 1. DATA HANDLING & VOCABULARY
# ==========================================
class Vocabulary:
    def __init__(self):
        self.token2idx = {"<pad>": 0, "<unk>": 1, "<s>": 2, "</s>": 3}
        self.idx2token = {0: "<pad>", 1: "<unk>", 2: "<s>", 3: "</s>"}

    def build(self, tokens):
        """Build vocab from a list of tokens (words or characters)"""
        for token in set(tokens):
            if token not in self.token2idx:
                self.token2idx[token] = len(self.token2idx)
                self.idx2token[len(self.idx2token)] = token
        return self

    def __len__(self):
        return len(self.token2idx)

class TextDataset(Dataset):
    def __init__(self, tokens, vocab, seq_len=20):
        self.vocab = vocab
        self.seq_len = seq_len
        # Convert tokens to indices, mapping unseen tokens to <unk> (1)
        self.indices = [vocab.token2idx.get(t, 1) for t in tokens]

    def __len__(self):
        # We need seq_len for input, and 1 for target. So total length is
seq_len + 1.
        return max(0, len(self.indices) - self.seq_len)

    def __getitem__(self, idx):
        # Input sequence: [i to i+seq_len]
        x = torch.tensor(self.indices[idx : idx + self.seq_len],
dtype=torch.long)
        # Target sequence: [i+1 to i+seq_len+1] (shifted by 1)
        y = torch.tensor(self.indices[idx + 1 : idx + 1 + self.seq_len],
dtype=torch.long)
        return x, y

def read_and_tokenize(file_path, token_level="word"):
    """Reads a file and returns a flat list of tokens."""
    with open(file_path, 'r', encoding='utf-8') as f:
        text = f.read()
```

```
    if token_level == "char":
        # Character level: list of characters (Great for Myanmar without
spaces!)
        tokens = list(text.replace("\n", " ")) # replace newlines with space
char
    else:
        # Word level: split by whitespace
        tokens = text.split()
    return tokens

# ==========================================
# 2. MODEL DEFINITION
# ==========================================
class LSTM_LM(nn.Module):
    def __init__(self, vocab_size, embed_dim=128, hidden_dim=256, num_layers=2,
dropout=0.2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers=num_layers,
                            batch_first=True, dropout=dropout if num_layers > 1
else 0)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        # x shape: (batch_size, seq_len)
        embeds = self.embedding(x)
        # out shape: (batch_size, seq_len, hidden_dim)
        out, hidden = self.lstm(embeds, hidden)
        # logits shape: (batch_size, seq_len, vocab_size)
        logits = self.fc(out)
        return logits, hidden

# ==========================================
# 3. TRAINING & EVALUATION FUNCTIONS
# ==========================================
def train_model(args):
    print(f"Reading training data from {args.train_file} (Level:
{args.token_level})…")
    train_tokens = read_and_tokenize(args.train_file, args.token_level)

    print("Building vocabulary…")
    vocab = Vocabulary().build(train_tokens)
    print(f"Vocabulary size: {len(vocab)} tokens")

    dataset = TextDataset(train_tokens, vocab, seq_len=args.seq_len)
    dataloader = DataLoader(dataset, batch_size=args.batch_size, shuffle=True,
drop_last=True)
```

```
    model = LSTM_LM(len(vocab), args.embed_dim, args.hidden_dim,
args.num_layers).to(device)
    criterion = nn.CrossEntropyLoss(ignore_index=0) # Ignore <pad> in loss
calculation
    optimizer = torch.optim.Adam(model.parameters(), lr=args.lr)

    print(f"Starting training for {args.epochs} epochs…")
    for epoch in range(args.epochs):
        model.train()
        total_loss = 0
        for x, y in dataloader:
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad()
            logits, _ = model(x)
            # Reshape for CrossEntropy: (Batch * SeqLen, VocabSize)
            loss = criterion(logits.view(-1, len(vocab)), y.view(-1))
            loss.backward()
            optimizer.step()
            total_loss += loss.item()

        if (epoch + 1) % 5 == 0:
            print(f"Epoch {epoch+1:3d}/{args.epochs} | Loss:
{total_loss/len(dataloader):.4f}")

    # Save model and vocabulary
    print(f"Saving model to {args.model_path}…")
    torch.save(model.state_dict(), args.model_path)
    with open(args.model_path + '.vocab', 'w', encoding='utf-8') as f:
        json.dump(vocab.token2idx, f, ensure_ascii=False)
    print("Training complete!")

def test_model(args):
    print(f"Loading model from {args.model_path}…")
    with open(args.model_path + '.vocab', 'r', encoding='utf-8') as f:
        token2idx = json.load(f)
    vocab = Vocabulary()
    vocab.token2idx = token2idx
    vocab.idx2token = {int(v): k for k, v in token2idx.items()}

    model = LSTM_LM(len(vocab), args.embed_dim, args.hidden_dim,
args.num_layers).to(device)
    model.load_state_dict(torch.load(args.model_path, map_location=device))
    model.eval()

    print(f"Reading test data from {args.test_file}…")
    test_tokens = read_and_tokenize(args.test_file, args.token_level)
    dataset = TextDataset(test_tokens, vocab, seq_len=args.seq_len)
```

```
    dataloader = DataLoader(dataset, batch_size=1) # Batch size 1 for accurate
PPL tracking

    total_nll, total_tokens = 0.0, 0
    criterion = nn.CrossEntropyLoss(ignore_index=0, reduction='sum')

    print("Evaluating…")
    with torch.no_grad():
        for x, y in dataloader:
            x, y = x.to(device), y.to(device)
            logits, _ = model(x)

            # Calculate loss only on non-padding tokens
            loss = criterion(logits.view(-1, len(vocab)), y.view(-1))
            total_nll += loss.item()
            total_tokens += (y != 0).sum().item() # Count non-pad tokens

    avg_nll = total_nll / total_tokens
    ppl = math.exp(avg_nll)
    print(f"Test Results -> Tokens: {total_tokens}, Avg NLL: {avg_nll:.4f}, PPL:
{ppl:.2f}")

def generate_text(args):
    print(f"Loading model from {args.model_path}…")
    with open(args.model_path + '.vocab', 'r', encoding='utf-8') as f:
        token2idx = json.load(f)
    vocab = Vocabulary()
    vocab.token2idx = token2idx
    vocab.idx2token = {int(v): k for k, v in token2idx.items()}

    model = LSTM_LM(len(vocab), args.embed_dim, args.hidden_dim,
args.num_layers).to(device)
    model.load_state_dict(torch.load(args.model_path, map_location=device))
    model.eval()

    prompt = args.prompt if args.prompt else "<s>"
    print(f"\nPrompt: {prompt}")
    print("Generating: ")

    # Convert prompt to indices
    if args.token_level == "char":
        tokens = list(prompt)
    else:
        tokens = prompt.split()

    indices = [vocab.token2idx.get(t, 1) for t in tokens]
    input_seq = torch.tensor([indices], dtype=torch.long).to(device)
    hidden = None
```

```
    generated_tokens = list(tokens)

    with torch.no_grad():
        for _ in range(args.gen_length):
            logits, hidden = model(input_seq, hidden)

            # Get logits for the last token in the sequence
            last_token_logits = logits[0, -1, :] / args.temperature
            probs = torch.softmax(last_token_logits, dim=-1)

            # Sample from the distribution (prevents repetitive loops)
            next_token_idx = torch.multinomial(probs, num_samples=1).item()

            if next_token_idx == 3: # </s> token
                break

            generated_tokens.append(vocab.idx2token.get(next_token_idx,
"<unk>"))

            # Feed the predicted token back in for the next step
            input_seq = torch.tensor([[next_token_idx]],
dtype=torch.long).to(device)

    # Join characters or words based on mode
    if args.token_level == "char":
        print("".join(generated_tokens))
    else:
        print(" ".join(generated_tokens))

# ==========================================
# 4. MAIN ARGUMENTS PARSER
# ==========================================
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="LSTM Language Model
Trainer/Evaluator/Generator")
    parser.add_argument("--mode", type=str, required=True, choices=["train",
"test", "generate"],
                        help="What to do?")
    parser.add_argument("--train_file", type=str, help="Path to training text
file")
    parser.add_argument("--test_file", type=str, help="Path to test text file")
    parser.add_argument("--model_path", type=str, default="lstm_model.pt",
help="Path to save/load model")

    # Data arguments
    parser.add_argument("--token_level", type=str, default="char",
choices=["word", "char"],
```

```
                        help="'word' for English, 'char' for Myanmar or
unsegmented text")

    # Model arguments
    parser.add_argument("--embed_dim", type=int, default=128)
    parser.add_argument("--hidden_dim", type=int, default=256)
    parser.add_argument("--num_layers", type=int, default=2)
    parser.add_argument("--seq_len", type=int, default=30, help="Length of
sequence to feed LSTM")

    # Training arguments
    parser.add_argument("--epochs", type=int, default=20)
    parser.add_argument("--batch_size", type=int, default=32)
    parser.add_argument("--lr", type=float, default=0.001)

    # Generation arguments
    parser.add_argument("--prompt", type=str, default="", help="Text to start
generation")
    parser.add_argument("--gen_length", type=int, default=100, help="Number of
tokens to generate")
    parser.add_argument("--temperature", type=float, default=0.8, help="Higher =
more random, Lower = stricter")

    args = parser.parse_args()
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")

    if args.mode == "train":
        if not args.train_file: parser.error("--train_file is required for
training")
        train_model(args)
    elif args.mode == "test":
        if not args.test_file: parser.error("--test_file is required for
testing")
        test_model(args)
    elif args.mode == "generate":
        generate_text(args)
```

[74]:
```
!python ./lstm_lm.py --help
```

```
usage: lstm_lm.py [-h] --mode {train,test,generate} [--train_file TRAIN_FILE]
                  [--test_file TEST_FILE] [--model_path MODEL_PATH]
                  [--token_level {word,char}] [--embed_dim EMBED_DIM]
                  [--hidden_dim HIDDEN_DIM] [--num_layers NUM_LAYERS]
                  [--seq_len SEQ_LEN] [--epochs EPOCHS]
                  [--batch_size BATCH_SIZE] [--lr LR] [--prompt PROMPT]
                  [--gen_length GEN_LENGTH] [--temperature TEMPERATURE]
```

```
LSTM Language Model Trainer/Evaluator/Generator

options:
  -h, --help            show this help message and exit
  --mode {train,test,generate}
                        What to do?
  --train_file TRAIN_FILE
                        Path to training text file
  --test_file TEST_FILE
                        Path to test text file
  --model_path MODEL_PATH
                        Path to save/load model
  --token_level {word,char}
                        'word' for English, 'char' for Myanmar or unsegmented
                        text
  --embed_dim EMBED_DIM
  --hidden_dim HIDDEN_DIM
  --num_layers NUM_LAYERS
  --seq_len SEQ_LEN     Length of sequence to feed LSTM
  --epochs EPOCHS
  --batch_size BATCH_SIZE
  --lr LR
  --prompt PROMPT       Text to start generation
  --gen_length GEN_LENGTH
                        Number of tokens to generate
  --temperature TEMPERATURE
                        Higher = more random, Lower = stricter
```

0.26 Shell Scripts

Training, Evaluation နဲ့ Text generation တွေအတွက် shell script တွေလည်း ပြင်ဆင်ထားတယ်။

[75]:
```
%cd /mnt/disk1/ye/exp/LM-Tutorial/lstm
```

```
/mnt/disk1/ye/exp/LM-Tutorial/lstm
```

[76]:
```
!cat train.sh
```

```
#!/bin/bash

# Train (takes ~10 seconds on your RTX 3090)
time python lstm_lm.py --mode train \
                  --train_file ../data/mypos_v3.word.clean \
                  --model_path myanmar_word_lm.pt \
                  --token_level word \
                  --epochs 30 \
                  --seq_len 50
```

[118]:
```
!./train.sh
```

```
Using device: cuda
Reading training data from ../data/mypos_v3.word.clean (Level: word)…
Building vocabulary…
Vocabulary size: 24720 tokens
Starting training for 30 epochs…
Epoch   5/30 | Loss: 1.8638
Epoch  10/30 | Loss: 1.5201
Epoch  15/30 | Loss: 1.3898
Epoch  20/30 | Loss: 1.3186
Epoch  25/30 | Loss: 1.2714
Epoch  30/30 | Loss: 1.2373
Saving model to myanmar_word_lm.pt…
Training complete!

real    85m30.684s
user    85m16.968s
sys     0m20.835s
```

0.27 Check GPU Status During LSTM-based LM Training

```
ye@lst-hpc3090:/mnt/disk1/ye/exp$ nvidia-smi
Fri May  1 18:08:58 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 535.288.01             Driver Version: 535.288.01   CUDA Version: 12.2     |
|-----------------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                         |                      |               MIG M. |
|=========================================+======================+======================|
|   0  NVIDIA GeForce RTX 3090 Ti     Off | 00000000:01:00.0  On |                  Off |
| 50%   76C    P2             356W / 480W |   2434MiB / 24564MiB |     87%      Default |
|                                         |                      |                  N/A |
+-----------------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                            GPU Memory |
|        ID   ID                                                             Usage      |
|=========================================================================================|
|    0   N/A  N/A     51185      G   /usr/lib/xorg/Xorg                          195MiB |
|    0   N/A  N/A     51389      G   /usr/bin/gnome-shell                         64MiB |
|    0   N/A  N/A     52003      G   /usr/libexec/xdg-desktop-portal-gnome         6MiB |
|    0   N/A  N/A     52268      C   /usr/NX/bin/nxnode.bin                      276MiB |
|    0   N/A  N/A    300604      G   ...1/usr/bin/snapd-desktop-integration       16MiB |
|    0   N/A  N/A    479401      G   /usr/bin/nautilus                            32MiB |
|    0   N/A  N/A   1780390      G   ...ads/tsetup.5.10.7/Telegram/Telegram       40MiB |
```

```
|    0   N/A  N/A   3125725      G   ...n=20260425-030030.652000-production       53MiB |
|    0   N/A  N/A   3138305      G   ...irefox/8191/usr/lib/firefox/firefox      137MiB |
|    0   N/A  N/A   3495846      C   /usr/bin/python3                            368MiB |
|    0   N/A  N/A   3536063      C   python                                     1192MiB |
+-----------------------------------------------------------------------------------------+
```

[119]:
```
!wc myanmar_word_lm.pt.vocab
```

```
     0  49440 854006 myanmar_word_lm.pt.vocab
```

[120]:
```
# This slices the object and pipes it to a readable format
!jq 'to_entries | .[:100] | from_entries' myanmar_word_lm.pt.vocab
```

{
"<pad>": 0,
"<unk>": 1,
"<s>": 2,
"</s>": 3,
"ဒါ": 4,
"Institutet": 5,
"ပုစ္ဆာဝိသဇ္ဇနာ": 6,
"မုတ်သုံ": 7,
"အချိန်မလပ်": 8,
"လွှမ်းခြုံ": 9,
"မွေးဖွား": 10,
"ရေးထိုး": 11,
"နှစ်ဦးနှစ်ဖက်လုံး": 12,
"ပြန်ချေပ": 13,
"ခန့်": 14,
"ငတ်မွတ်": 15,
"ခရီး": 16,
"ပင့်": 17,
"မဟာဝိဇယ": 18,
"Lies": 19,
"ယောင်ဝါးဝါး": 20,
"တောင်တွင်းရှင်မုန်ယို": 21,
"မောတောင်": 22,
"မုန်လာဥနီ": 23,
"redgiant": 24,
"အထူးဆုံး": 25,
"တွမ်": 26,
"ဓါတုဗေဒ": 27,
"ဘဏ္ဍာဖြ": 28,
"အယ်လိုရာ": 29,
"နာကျင်မှု": 30,
"ကွဲပြား": 31,
"အဝမ်း": 32,

jq command က စက်ထဲမှာ မရှိရင်တော့ အောက်ပါလိုမျိုး fold command နဲ့ head command နဲ့လည်း တွဲကြည့်လို့ ရလိမ့်မယ်။

[122]:
```
# Wrap the text at 80 characters and show the first 20 lines
!fold -s -w 80 myanmar_word_lm.pt.vocab | head -n 20
```

```
{"<pad>": 0, "<unk>": 1, "<s>": 2, "</s>": 3, "ဒါ": 4, "Institutet": 5,
"ပုစ္ဆာဝိသဇ္ဇနာ": 6, "မုတ်သုံ": 7,
"အချိန်မလပ်": 8, "လွှမ်းခြုံ": 9,
"မွေးဖွား": 10, "ရေးထိုး": 11,
"နှစ်ဦးနှစ်ဖက်လုံး": 12,
"ပြန်ချေပ": 13, "ခန့်": 14, "ငတ်မွတ်":
15, "ခရီး": 16, "ပင့်": 17, "မဟာဝိဇယ": 18,
"Lies": 19, "ယောင်ဝါးဝါး": 20,
"တောင်တွင်းရှင်မုန်ယို": 21,
"မောတောင်": 22, "မုန်လာဥနီ": 23, "redgiant":
24, "အထူးဆုံး": 25, "တွမ်": 26, "ဓါတုဗေဒ":
27, "၁၉၂၉ဖ": 28, "အယ်လိုရာ": 29,
"နာကျင်မှု": 30, "ကွဲပြား": 31,
"အဝှမ်း": 32, "ဆိုးဆိုးဝါးဝါး": 33,
"ဗီဆေးလိယပ်": 34,
"ခေါင်းကိုက်ရောဂါ": 35, "သရ": 36,
"မိုင်းသောက်": 37, "userpage": 38,
"လူပြိန်း": 39, "အေးတိအေးစက်": 40,
"ရန်သူ": 41, "BIA": 42, "လက္ခဏာ": 43,
"ချွတ်ယွင်းချက်": 44,
fold: write error: Broken pipe
```

0.28 Evaluation on LSTM LM

[78]:
```
!cat eval.sh
```

```
#!/bin/bash

time python lstm_lm.py --mode test \
                   --test_file ../data/otest.word.clean \
                   --model_path ./myanmar_word_lm.pt \
                   --token_level word \
                   --seq_len 50
```

[79]:
```
!./eval.sh
```

```
Using device: cuda
Loading model from ./myanmar_word_lm.pt…
Reading test data from ../data/otest.word.clean…
Evaluating…
```

```
Test Results -> Tokens: 607700, Avg NLL: 0.7512, PPL: 2.12

real    0m22.087s
user    0m22.282s
sys     0m0.642s
```

တကယ်က train လုပ်ထားတာက ပုဒ်ထီး၊ ပုဒ်မတွေကို ဖြုတ်ထားတဲ့ cleaning လုပ်ထားတဲ့ open test ဒေတာနဲ့ evaluation လုပ်ရမှာပါ။
eval.sh ကို update လုပ်လိုက်ပြီး နောက်တစ်ခေါက် evaluation လုပ်ကြည့်ရအောင်။

[84]:
```
!cat ./eval.sh
```

```
#!/bin/bash

time python lstm_lm.py --mode test \
                  --test_file ../data/otest.word \
                  --model_path ./myanmar_word_lm.pt \
                  --token_level word \
                  --seq_len 50
```

[83]:
```
!./eval.sh
```

```
Using device: cuda
Loading model from ./myanmar_word_lm.pt…
Reading test data from ../data/otest.word…
Evaluating…
Test Results -> Tokens: 670900, Avg NLL: 7.2281, PPL: 1377.59

real    0m24.831s
user    0m25.147s
sys     0m0.572s
```

သိစေချင်တာက test ဒေတာကလည်း ဘယ်လောက် အရေးကြီးတယ်ဆိုတာ။ ပြီးတော့ training လုပ်ထားတဲ့ ဒေတာ၊ ဒိုမိန်း စတာတွေက ဘယ်လောက်ထိ PPL ရလဒ်ကို အပြောင်းအလဲ ဖြစ်နိုင်သလဲ ဆိုတာကိုပါ။

0.29 Text Generation

[85]:
```
!cat ./gen.sh
```

```
python lstm_lm.py --mode generate \
                  --model_path myanmar_word_lm.pt \
                  --token_level word \
                  --prompt "ပြည်ထောင်စု" \
                  --gen_length 200 \
                  --temperature 0.7
```

[86]:
```
!./gen.sh
```

```
Using device: cuda
Loading model from myanmar_word_lm.pt…
```

```
Prompt: ပြည်ထောင်စု
Generating:
ပြည်ထောင်စု ဝတ္ထု ကို အစောဆုံး သော မြို့ ဖြစ် သည် မှတ်ပုံတင် ချင် တာ ဘယ် မှာ လဲ
ဆို တာ ပြ ပေး ပါ ခရစ် အသုံးပြု ချိန် စနစ် များ အရ မြန်မာ့ ယဉ်ကျေး မှု ကို ခံစား
သည့် အခါ အခါ သိရှိ ရာ၌ သိပ္ပံနည်း များ အချင်းချင်း နှင့် အပင် များ တို့ သည် နေ
အဖွဲ့အစည်း အတွင်း တွင် ရှိ ပြီး မြောက် ပိုင်း တွင် မဏ္ဍိမ ဒေသ ကြီး များ မှ ၎င်း
ကို ကမ္ဘာ နိုင်ငံ ရှိ နိုင်ငံ များ တွင် စီးပွား ရေး ဖွံ့ဖြိုး လုပ်ဆောင် မှု များ
ရှိ သည် အထူး တိုင်းတာ မှု ကုမ္ပဏီ ဝင်ငွေခွန် မ ရ ရန် အတွက် စီစဉ် ထား သော အဖွဲ့
ဝင် ဖြစ် သည့် မြစေတီ ဖောင့် ကို စတင် ချီးမြှင့် သည် တကယ့်ကို ကောင်း တာ ပဲ ကျွန်မ
ရဲ့ အမှား ကြောင့် သူ နဲ့ သူ ဒီ နေ့ ဆွမ်းချိုင့် ပို့ ဖို့ ဆွမ်း ဟင်း ချက် နေ ရစ်
တယ် တကယ် လို့ ဘာ လုပ် ချင် လဲ မြန်မာ ငွေ ၃ ခု က အဆို အက အစီအစဉ် ရှိ ပါ␣
↪သလား
ခင်ဗျား က ဖော်ရွေ ပြီး တော့ ကူညီ ပါ ကျွန်တော် ဆေး ကူညီ မယ် လို့ စဉ်းစား နေ တာ
ခင်ဗျား စိတ်ကူးစိတ်သန်း ကောင်း တယ် ကား ပြင် ဖို့ စောင့် ရ မယ် မိဘ တွေ က အသက်ကြီး
တာ နဲ့ သူ့ဌေး ပဲ တစ် နာရီ လောက် က ၁ ဒေါ်လာ ၁၃၅၀ ကျပ် ပါ လေယာဉ် စီး ခွင့် ကဒ်␣
↪ကို
တဆိတ်လောက် ကြည့် ပါရစေ ရှမ်း ခုနှစ် တွင် မင်းတုန်း မင်းတရား တည် သော
ကျောက်တော်ကြီး ဘုရား ရှိ သည် ဝေလငါး က သိုး ကို နေကောင်း ရော
```

ဆရာ **temp** နဲ့ **length** ကို အပြောင်းအလဲ လုပ်လိုက်တယ်။ ထပ် **generation** လုပ်ခိုင်းကြမယ်။

[87]:
```
!./gen.sh
```

```
Using device: cuda
Loading model from myanmar_word_lm.pt…

Prompt: ပြည်ထောင်စု
Generating:
ပြည်ထောင်စု ကြံ့ခိုင် ရေး နှင့် ဖွံ့ဖြိုး ရေး အသင်း ၏ ဥက္ကဋ္ဌ ဖြစ် ခဲ့ သည်
အနီးဆုံး ဘဏ် က ဘယ် အချိန် ထွက် လဲ အကျပ်အတည်း စည်းကမ်း အားလုံး ဆိုး တယ်␣
↪ခင်ဗျား
တို့ ကုန်ပစ္စည်း က ဘာ အားသာချက် တွေ ရှိ ပါ သလဲ ခင်ဗျား မှာ ကလေး တစ်ယောက်စာ ပြင်
တဲ့ ပွဲ ရှိ လား ကပ်ကြေး ကျွန်တော် တို့ ဘယ်လောက် ကြာကြာ စောင့် ရ မလဲ
```

0.30 Evaluation with Closed-Test Data

Closed-test ဒေတာ ဒါပေမဲ့ စာကြောင်းရေက များတယ်။ အဲဒီ ဒေတာနဲ့ LSTM LM ကို evaluation လုပ်ကြည့်ပြီး PPL က ဘယ်လို အပြောင်းအလဲ ဖြစ်သလဲ ဆိုတာကို လေ့လာကြည့်ကြရအောင်။

[136]:
```
!wc ../data/10k_test.txt.clean
```

```
10000  117817 1686267 ../data/10k_test.txt.clean
```

[137]:
```
!cat ./eval-closed.sh
```

```
#!/bin/bash

time python lstm_lm.py --mode test \
                  --test_file ../data/10k_test.txt.clean \
                  --model_path ./myanmar_word_lm.pt \
                  --token_level word \
                  --seq_len 50
```

[139]:
```
!./eval-closed.sh
```

```
Using device: cuda
Loading model from ./myanmar_word_lm.pt…
Reading test data from ../data/10k_test.txt.clean…
Evaluating…
Test Results -> Tokens: 5888350, Avg NLL: 0.7543, PPL: 2.13

real    2m26.987s
user    2m27.241s
sys     0m0.736s
```

Closed-test နဲ့ evaluation လုပ်တာလည်း ရလဒ်ကောင်းပါတယ်။
LM ပဲ ဖြစ်ဖြစ်၊ ဘယ်မော်ဒယ်ပဲ ဖြစ်ဖြစ် နှစ်မျိုးစလုံးနဲ့ စစ်သင့်ပါတယ်။
ဒါပေမဲ့ ပိုအရေးကြီးတာက open test ဒေတာရဲ့ ရလဒ်ပါ။

0.31 Transformer LM (HuggingFace on GPU)

We use GPT-2. We explicitly cast it to bfloat16 which the RTX 3090 supports natively, doubling throughput and halving VRAM usage with zero accuracy loss.

ဒီ ဥပမာ code ကတော့ HuggingFace ကနေ ရှိပြီးသား မော်ဒယ်ကို ယူသုံးပြီး datasets ဆိုတဲ့ Python library ကို import လုပ်ပြီး publicly share လုပ်ပေးထားတဲ့ wiki dataset ကိုသုံးပြီး evaluation လုပ်ပြထားတာပါ။ အင်္ဂလိပ်စာတို့လို ဂျာမန်တို့လို့ ဂျပန်၊ တရုတ် စတဲ့ ဘာသာစကားတွေအတွက်က ဒေတာအများကြီးသုံးပြီး ဆောက်ထားတဲ့ LM တွေက အဆင်သင့်ရှိတာမို့ တော်တော်လေး အဆင်ပြေပါတယ်။ ဒါပေမဲ့ ဆရာတို့ မြန်မာစာအတွက်က အခြေခံအားဖြင့် အရမ်းကြီးအဆင်မပြေလို့ ကိုယ်လုပ်ချင်တဲ့ ဒိုမိန်းအတွက် LM ကို သပ်သပ်ဆောက်ပြီးပဲ သုံးကြရတာ များပါလိမ့်မယ်။

[140]:
```
#!pip install -q transformers datasets accelerate

from transformers import AutoTokenizer, AutoModelForCausalLM
from datasets import load_dataset

model_id = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(model_id)
# Load in bfloat16 to maximize RTX 3090 efficiency
model_gpt2 = AutoModelForCausalLM.from_pretrained(model_id, torch_dtype=torch.
 ↪bfloat16).to(device)

# Load a real test set (WikiText-2) to get meaningful PPL numbers
```

```python
test = load_dataset("wikitext", "wikitext-2-raw-v1", split="test")
encodings = tokenizer("\n\n".join(test["text"]), return_tensors="pt").to(device)

# Strided Perplexity Evaluation
from tqdm.auto import tqdm

max_length = model_gpt2.config.n_positions
stride = 512
seq_len = encodings.input_ids.size(1)

nll_sum, n_tokens = 0.0, 0
prev_end_loc = 0

print("Evaluating GPT-2 (bf16)...")
with torch.no_grad():
    for begin_loc in tqdm(range(0, seq_len, stride)):
        end_loc = min(begin_loc + max_length, seq_len)
        trg_len = end_loc - prev_end_loc

        input_ids = encodings.input_ids[:, begin_loc:end_loc]
        target_ids = input_ids.clone()
        target_ids[:, :-trg_len] = -100

        outputs = model_gpt2(input_ids, labels=target_ids)
        neg_log_likelihood = outputs.loss.item()

        num_valid_tokens = (target_ids != -100).sum().item()
        num_loss_tokens = num_valid_tokens - target_ids.size(0) # HF shifts␣
 ↪labels internally

        nll_sum += neg_log_likelihood * num_loss_tokens
        n_tokens += num_loss_tokens
        prev_end_loc = end_loc

        if end_loc == seq_len: break

ppl_gpt2, ent_gpt2 = compute_ppl_and_entropy(nll_sum, n_tokens)
print(f"\nGPT-2 Test -> PPL: {ppl_gpt2:.2f}, Entropy: {ent_gpt2:.2f} nats")
```

```
/home/ye/.local/lib/python3.12/site-packages/tqdm/auto.py:21: TqdmWarning:
IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
2026-05-01 21:50:18.018010: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-01 21:50:20.632506: E
```

```
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1777647021.674408 3495846 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1777647022.019741 3495846 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1777647023.892053 3495846 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777647023.892079 3495846 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777647023.892080 3495846 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777647023.892081 3495846 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2026-05-01 21:50:23.986316: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
Generating test split: 100%|                | 4358/4358
[00:00<00:00, 197012.04 examples/s]
Generating train split: 100%|               | 36718/36718
[00:00<00:00, 789327.38 examples/s]
Generating validation split: 100%|             | 3760/3760
[00:00<00:00, 513900.65 examples/s]
Token indices sequence length is longer than the specified maximum sequence
length for this model (287644 > 1024). Running this sequence through the model
will result in indexing errors

Evaluating GPT-2 (bf16)…

  0%|
| 0/562 [00:00<?, ?it/s]`loss_type=None` was set in the config but it is
unrecognised.Using the default loss: `ForCausalLMLoss`.
100%|                                  | 560/562
[00:07<00:00, 75.81it/s]


GPT-2 Test -> PPL: 25.41, Entropy: 3.24 nats
```

0.32 Transformer LM (with Myanmar language)

တကယ်ကတော့ LSTM နဲ့ Transformer မော်ဒယ်အကြားမှာ မတူတာတွေ အများကြီးပါပဲ။ Architecture ပိုင်းတင်မကပဲ Training လုပ်တဲ့ နည်းလမ်းတွေမှာလည်း အများကြီး အပြောင်းအလဲတွေရှိပါတယ်။ ဆရာ အခုသုံးနေတဲ့ GPU စက်က RTX 3090 (24GB VRAM) ရှိတာမို့ mixed-precision training (bf16) ဆိုတာကို natively support လုပ်တာမို့လို့ Transformer မော်ဒယ် training time က အရမ်းမြန်သွားပါလိမ့်မယ်။

ပြီးတော့ Transformer ပေါ်လာပြီး နောက်ပိုင်းမှာ မော်ဒယ်ကို အစအဆုံး training လုပ်တာထက် pre-trained လုပ်ထားတဲ့ အဆင်ပြေတဲ့ မော်ဒယ်ကိုပဲ အခြေခံပြီးတော့ Finetuning လုပ်ယူကြတာက များပါတယ်။ ဘာကြောင့်လဲ ဆိုတော့ Transformer ကို အခြေခံတဲ့ LM ကောင်းကောင်းဆောက်ဖို့အတွက်ဆိုရင် monolingual data ကိုလည်း GB အများကြီး ပြင်ထားဖို့လိုအပ်ပြီး GPU လည်း အများကြီးသုံးကြရတာမို့ပါ။

ဒီ LM Tutorial မှာတော့ pretrained model XGLM ကိုပဲ ကိုယ့် မြန်မာစာဒေတာနဲ့ finetune လုပ်ပြီး weight တွေကို learn လုပ်ယူသွားပါမယ်။
XGLM က GPT-2 မဟုတ်ဘူးနော်။ သူက Meta ရဲ့ multilingual generative model ပါ။ သူက "GPT-style" Autoregressive အာခီပါ။ ပြီးတော့ မြန်မာစာအတွက် အဆင်ပြေတယ်။ သူ့ vocab size က 256k SentencePiece ပါ။

[142]:
```
%cd /mnt/disk1/ye/exp/LM-Tutorial/transformer
```

```
/mnt/disk1/ye/exp/LM-Tutorial/transformer
```

[151]:
```
%cat ./transformer_lm.py
```

```
#!/usr/bin/env python3
"""
Transformer-based Language Model using Hugging Face.
Defaults to fine-tuning XGLM (a GPT-style autoregressive model).
Note: Standard GPT-2 is NOT used by default because its tokenizer lacks Myanmar
subwords, forcing it to split Myanmar text into inefficient UTF-8 bytes.
XGLM uses a 256k SentencePiece vocabulary that natively includes Myanmar script.
"""

import os
import math
import argparse
import torch
from torch.utils.data import DataLoader
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    DataCollatorForLanguageModeling,
    Trainer,
    TrainingArguments,
)
from datasets import Dataset

# ==========================================
```

```
# 1. DATA UTILITIES
# ==========================================
def load_text_file(file_path):
    with open(file_path, 'r', encoding='utf-8') as f:
        return [line.strip() for line in f if line.strip()]

def tokenize_dataset(lines, tokenizer, max_length):
    """Safely tokenizes and chunks text using HuggingFace best practices."""
    from datasets import Dataset

    # 1. Create a raw dataset
    raw_dataset = Dataset.from_dict({"text": lines})

    # 2. Tokenize without truncation first
    def tokenize_function(examples):
        return tokenizer(examples["text"])

    tokenized_dataset = raw_dataset.map(
        tokenize_function,
        batched=True,
        remove_columns=["text"],
        desc="Tokenizing"
    )

    # 3. Group into fixed-length blocks (prevents the massive tensor warning)
    def group_texts(examples):
        # Concatenate all token lists
        concatenated_examples = {k: sum(examples[k], []) for k in
examples.keys()}
        total_length = (len(concatenated_examples["input_ids"]) // max_length) *
max_length

        # Split by max_length
        result = {
            k: [t[i : i + max_length] for i in range(0, total_length,
max_length)]
            for k, t in concatenated_examples.items()  # <--- FIX IS HERE
        }
        result["labels"] = result["input_ids"].copy()
        return result

    lm_dataset = tokenized_dataset.map(
        group_texts,
        batched=True,
        desc="Grouping into chunks"
    )
    return lm_dataset
```

```
# ==========================================
# 2. STRIDED PERPLEXITY EVALUATOR
# ==========================================
def evaluate_ppl(args, model, tokenizer, test_lines):
    """Calculates exact strided Perplexity, identical to HF advanced
documentation."""
    print(f"Tokenizing test set ({len(test_lines)} lines)…")
    encodings = tokenizer("\n\n".join(test_lines),
return_tensors="pt").to(model.device)

    max_length = args.seq_len
    stride = args.stride
    seq_len = encodings.input_ids.size(1)

    nll_sum, n_tokens = 0.0, 0
    prev_end_loc = 0

    print(f"Evaluating with Stride={stride}, Context={max_length}…")
    with torch.no_grad():
        for begin_loc in range(0, seq_len, stride):
            end_loc = min(begin_loc + max_length, seq_len)
            trg_len = end_loc - prev_end_loc

            input_ids = encodings.input_ids[:, begin_loc:end_loc]
            target_ids = input_ids.clone()
            target_ids[:, :-trg_len] = -100 # Ignore context tokens in loss

            outputs = model(input_ids, labels=target_ids)
            neg_log_likelihood = outputs.loss.item()

            num_valid_tokens = (target_ids != -100).sum().item()
            num_loss_tokens = num_valid_tokens - target_ids.size(0) # HF shifts
labels internally

            nll_sum += neg_log_likelihood * num_loss_tokens
            n_tokens += num_loss_tokens
            prev_end_loc = end_loc

            if end_loc == seq_len: break

    avg_nll = nll_sum / n_tokens
    ppl = math.exp(avg_nll)
    return ppl, avg_nll

# ==========================================
# 3. MODES
# ==========================================
def train_model(args):
```

```
print(f"Loading base model: {args.base_model}")
tokenizer = AutoTokenizer.from_pretrained(args.base_model)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# Load model in bfloat16 for RTX 3090 efficiency
model = AutoModelForCausalLM.from_pretrained(
    args.base_model,
    torch_dtype=torch.bfloat16
)

print(f"Loading training data: {args.train_file}")
train_lines = load_text_file(args.train_file)
train_dataset = tokenize_dataset(train_lines, tokenizer, args.seq_len)

#data_collator = DataCollatorForLanguageModeling(mlm=False)
data_collator = DataCollatorForLanguageModeling(tokenizer, mlm=False)

# Setup HuggingFace Trainer
training_args = TrainingArguments(
    output_dir=args.model_path,
    overwrite_output_dir=True,
    num_train_epochs=args.epochs,
    per_device_train_batch_size=args.batch_size,
    save_strategy="epoch",
    logging_steps=10,
    bf16=True, # RTX 3090 Ampere architecture supports this natively
    gradient_accumulation_steps=2, # Helps with stability on small datasets
    learning_rate=args.lr,
    weight_decay=0.01,
    report_to="none", # Disable wandb/tensorboard for clean terminal output
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    data_collator=data_collator,
)

print("Starting Transformer Fine-Tuning…")
trainer.train()

# Save final model
trainer.save_model(args.model_path)
tokenizer.save_pretrained(args.model_path)
print(f"Model saved to {args.model_path}")
```

```
def test_model(args):
    print(f"Loading fine-tuned model from: {args.model_path}")
    tokenizer = AutoTokenizer.from_pretrained(args.model_path)
    model = AutoModelForCausalLM.from_pretrained(
        args.model_path,
        torch_dtype=torch.bfloat16
    ).to(device)
    model.eval()

    test_lines = load_text_file(args.test_file)
    ppl, nll = evaluate_ppl(args, model, tokenizer, test_lines)
    print(f"\nTest Results -> PPL: {ppl:.2f}, Avg NLL: {nll:.4f} nats")

def generate_text(args):
    print(f"Loading fine-tuned model from: {args.model_path}")
    tokenizer = AutoTokenizer.from_pretrained(args.model_path)
    model = AutoModelForCausalLM.from_pretrained(
        args.model_path,
        torch_dtype=torch.bfloat16
    ).to(device)
    model.eval()

    prompt = args.prompt if args.prompt else ""
    print(f"\nPrompt: {prompt}")
    print("Generating: ")

    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

    # HF Generate function handles sampling natively
    outputs = model.generate(
        **inputs,
        max_new_tokens=args.gen_length,
        temperature=args.temperature,
        top_k=args.top_k,
        top_p=args.top_p,
        do_sample=True,
        pad_token_id=tokenizer.eos_token_id
    )

    generated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)
    print(generated_text)

# ==========================================
# 4. MAIN
# ==========================================
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Transformer LM (Hugging
Face)")
```

```
    parser.add_argument("--mode", type=str, required=True, choices=["train",
"test", "generate"])
    parser.add_argument("--train_file", type=str, help="Path to train text")
    parser.add_argument("--test_file", type=str, help="Path to test text")
    parser.add_argument("--model_path", type=str, default="./xglm_finetuned")
    parser.add_argument(
    "--base_model",
    type=str,
    default="facebook/xglm-564M",
    help="Base model to fine-tune. Default is XGLM-564M (strong multilingual
tokenizer). "
         "Do NOT use standard 'gpt2' for Myanmar as its tokenizer will shatter
the text into bytes."
    )

    # Data/Model Args
    parser.add_argument("--seq_len", type=int, default=128, help="Max sequence
length for training/chunking")
    parser.add_argument("--stride", type=int, default=64, help="Stride for PPL
evaluation")

    # Training Args
    parser.add_argument("--epochs", type=int, default=3) # Transformers need
fewer epochs than LSTMs!
    parser.add_argument("--batch_size", type=int, default=8) # Larger than LSTM
due to efficiency
    parser.add_argument("--lr", type=float, default=5e-5) # Standard HF learning
rate

    # Generation Args
    parser.add_argument("--prompt", type=str, default="")
    parser.add_argument("--gen_length", type=int, default=100)
    parser.add_argument("--temperature", type=float, default=0.7)
    parser.add_argument("--top_k", type=int, default=50)
    parser.add_argument("--top_p", type=float, default=0.95)

    args = parser.parse_args()
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")

    if args.mode == "train" and not args.train_file: parser.error("--train_file
required")
    if args.mode == "test" and not args.test_file: parser.error("--test_file
required")

    if args.mode == "train": train_model(args)
    elif args.mode == "test": test_model(args)
    elif args.mode == "generate": generate_text(args)
```

0.33 Finetuning

တကယ်က အထက်ပါ warning message က HuggingFace library တွေကို Jupyter notebook အတွင်းက ခေါ်သုံးရင် ပေးနေကြ warning ပါ။ Multiprocessing လုပ်ရင် တွေ့ရတဲ့ message မျိုးပါ။ ဘာကြောင့်လဲ ဆိုတော့ tokenizer library က text processing speed မြန်ဆန်အောင်လို့ multiple CPU တွေသုံးမှာမို့လို့ပါ။

Finetuning သို့မဟုတ် training တမျိုးလုပ်မယ့် script က အောက်ပါအတိုင်းပါ။

[89]:
```
%cd /mnt/disk1/ye/exp/LM-Tutorial/transformer
```

```
/mnt/disk1/ye/exp/LM-Tutorial/transformer
```

[91]:
```
!cat train.sh
```

```
#!/bin/bash
# Fine-tuning XGLM on your data
# bf16 is enabled automatically in the python script for your RTX 3090

time python transformer_lm.py --mode train \
                  --train_file ../data/mypos_v3.word.clean \
                  --model_path ./myanmar_xglm_word \
                  --seq_len 128 \
                  --epochs 3 \
                  --batch_size 8
```

အောက်ပါလိုမျိုး warning ကို ပျောက်အောင် လုပ်ရအောင်။

huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid deadlocks…

To disable this warning, you can either: - Avoid using `tokenizers` before the fork if possible - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

[29]:
```
import os
os.environ["TOKENIZERS_PARALLELISM"] = "false"
```

[152]:
```
!./train.sh
```

```
2026-05-01 22:56:21.964239: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-01 22:56:21.977467: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1777650981.993242 3555120 cuda_dnn.cc:8579] Unable to register cuDNN
```

```
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1777650981.998437 3555120 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1777650982.011564 3555120 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777650982.011592 3555120 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777650982.011601 3555120 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777650982.011604 3555120 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2026-05-01 22:56:22.015113: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
Using device: cuda
Loading base model: facebook/xglm-564M
Loading training data: ../data/mypos_v3.word.clean
Tokenizing: 100%|         | 43196/43196 [00:06<00:00, 6832.92 examples/s]
Grouping into chunks: 100%|   | 43196/43196 [00:03<00:00, 13996.44 examples/s]
Starting Transformer Fine-Tuning…
  0%|                                                    | 0/1566 [00:00<?,
?it/s]Passing a tuple of `past_key_values` is deprecated and will be removed in
Transformers v4.58.0. You should pass an instance of `EncoderDecoderCache`
instead, e.g.
`past_key_values=EncoderDecoderCache.from_legacy_cache(past_key_values)`.
{'loss': 3.1894, 'grad_norm': 1.9296875, 'learning_rate': 4.971264367816092e-05,
'epoch': 0.02}
{'loss': 3.0948, 'grad_norm': 1.78125, 'learning_rate': 4.93933588761175e-05,
'epoch': 0.04}
{'loss': 3.0141, 'grad_norm': 1.7265625, 'learning_rate':
4.9074074074074075e-05, 'epoch': 0.06}
{'loss': 2.9994, 'grad_norm': 1.65625, 'learning_rate': 4.8754789272030654e-05,
'epoch': 0.08}
{'loss': 2.993, 'grad_norm': 1.6953125, 'learning_rate': 4.8435504469987234e-05,
'epoch': 0.1}
{'loss': 2.919, 'grad_norm': 1.609375, 'learning_rate': 4.8116219667943806e-05,
'epoch': 0.11}
{'loss': 2.9639, 'grad_norm': 1.546875, 'learning_rate': 4.7796934865900385e-05,
'epoch': 0.13}
{'loss': 2.9834, 'grad_norm': 1.5546875, 'learning_rate':
```

4.7477650063856965e-05, 'epoch': 0.15}
{'loss': 2.9018, 'grad_norm': 1.6015625, 'learning_rate':
4.7158365261813544e-05, 'epoch': 0.17}
{'loss': 2.917, 'grad_norm': 1.578125, 'learning_rate': 4.6839080459770116e-05,
'epoch': 0.19}
{'loss': 2.9062, 'grad_norm': 1.5546875, 'learning_rate':
4.6519795657726695e-05, 'epoch': 0.21}
{'loss': 2.9405, 'grad_norm': 1.6015625, 'learning_rate': 4.620051085568327e-05,
'epoch': 0.23}
{'loss': 2.892, 'grad_norm': 1.59375, 'learning_rate': 4.588122605363985e-05,
'epoch': 0.25}
{'loss': 2.8513, 'grad_norm': 1.578125, 'learning_rate': 4.5561941251596426e-05,
'epoch': 0.27}
{'loss': 2.8534, 'grad_norm': 1.53125, 'learning_rate': 4.5242656449553005e-05,
'epoch': 0.29}
{'loss': 2.8505, 'grad_norm': 1.5625, 'learning_rate': 4.4923371647509585e-05,
'epoch': 0.31}
{'loss': 2.891, 'grad_norm': 1.640625, 'learning_rate': 4.460408684546616e-05,
'epoch': 0.33}
{'loss': 2.8334, 'grad_norm': 1.5625, 'learning_rate': 4.4284802043422736e-05,
'epoch': 0.34}
{'loss': 2.8363, 'grad_norm': 1.6015625, 'learning_rate': 4.396551724137931e-05,
'epoch': 0.36}
{'loss': 2.8607, 'grad_norm': 1.5, 'learning_rate': 4.364623243933589e-05,
'epoch': 0.38}
{'loss': 2.8489, 'grad_norm': 1.5, 'learning_rate': 4.332694763729247e-05,
'epoch': 0.4}
{'loss': 2.8472, 'grad_norm': 1.5625, 'learning_rate': 4.3007662835249046e-05,
'epoch': 0.42}
{'loss': 2.8676, 'grad_norm': 1.59375, 'learning_rate': 4.2688378033205625e-05,
'epoch': 0.44}
{'loss': 2.8883, 'grad_norm': 1.5703125, 'learning_rate': 4.23690932311622e-05,
'epoch': 0.46}
{'loss': 2.8722, 'grad_norm': 1.640625, 'learning_rate': 4.204980842911878e-05,
'epoch': 0.48}
{'loss': 2.8225, 'grad_norm': 1.59375, 'learning_rate': 4.173052362707535e-05,
'epoch': 0.5}
{'loss': 2.8505, 'grad_norm': 1.53125, 'learning_rate': 4.141123882503193e-05,
'epoch': 0.52}
{'loss': 2.8474, 'grad_norm': 1.5546875, 'learning_rate': 4.109195402298851e-05,
'epoch': 0.54}
{'loss': 2.8467, 'grad_norm': 1.5078125, 'learning_rate': 4.077266922094509e-05,
'epoch': 0.56}
{'loss': 2.8774, 'grad_norm': 1.59375, 'learning_rate': 4.0453384418901666e-05,
'epoch': 0.57}
{'loss': 2.8539, 'grad_norm': 1.4921875, 'learning_rate': 4.013409961685824e-05,
'epoch': 0.59}
{'loss': 2.8739, 'grad_norm': 1.46875, 'learning_rate': 3.981481481481482e-05,

'epoch': 0.61}
{'loss': 2.8662, 'grad_norm': 1.46875, 'learning_rate': 3.949553001277139e-05,
'epoch': 0.63}
{'loss': 2.8186, 'grad_norm': 1.5, 'learning_rate': 3.917624521072797e-05,
'epoch': 0.65}
{'loss': 2.816, 'grad_norm': 1.6171875, 'learning_rate': 3.885696040868455e-05,
'epoch': 0.67}
{'loss': 2.8288, 'grad_norm': 1.59375, 'learning_rate': 3.853767560664113e-05,
'epoch': 0.69}
{'loss': 2.8225, 'grad_norm': 1.4765625, 'learning_rate': 3.82183908045977e-05,
'epoch': 0.71}
{'loss': 2.8277, 'grad_norm': 1.5390625, 'learning_rate': 3.789910600255428e-05,
'epoch': 0.73}
{'loss': 2.8471, 'grad_norm': 1.4375, 'learning_rate': 3.757982120051086e-05,
'epoch': 0.75}
{'loss': 2.8193, 'grad_norm': 1.5625, 'learning_rate': 3.726053639846743e-05,
'epoch': 0.77}
{'loss': 2.8514, 'grad_norm': 1.5078125, 'learning_rate': 3.694125159642402e-05,
'epoch': 0.79}
{'loss': 2.8028, 'grad_norm': 1.53125, 'learning_rate': 3.662196679438059e-05,
'epoch': 0.8}
{'loss': 2.8342, 'grad_norm': 1.5, 'learning_rate': 3.630268199233717e-05,
'epoch': 0.82}
{'loss': 2.8473, 'grad_norm': 1.53125, 'learning_rate': 3.598339719029374e-05,
'epoch': 0.84}
{'loss': 2.787, 'grad_norm': 1.453125, 'learning_rate': 3.566411238825032e-05,
'epoch': 0.86}
{'loss': 2.8792, 'grad_norm': 1.515625, 'learning_rate': 3.53448275862069e-05,
'epoch': 0.88}
{'loss': 2.83, 'grad_norm': 1.5390625, 'learning_rate': 3.502554278416347e-05,
'epoch': 0.9}
{'loss': 2.805, 'grad_norm': 1.5, 'learning_rate': 3.470625798212006e-05,
'epoch': 0.92}
{'loss': 2.7976, 'grad_norm': 1.46875, 'learning_rate': 3.438697318007663e-05,
'epoch': 0.94}
{'loss': 2.8239, 'grad_norm': 1.7109375, 'learning_rate': 3.406768837803321e-05,
'epoch': 0.96}
{'loss': 2.8091, 'grad_norm': 1.5390625, 'learning_rate': 3.374840357598978e-05,
'epoch': 0.98}
{'loss': 2.7787, 'grad_norm': 1.484375, 'learning_rate': 3.342911877394636e-05,
'epoch': 1.0}
{'loss': 2.7759, 'grad_norm': 1.4921875, 'learning_rate': 3.310983397190294e-05,
'epoch': 1.02}
{'loss': 2.7753, 'grad_norm': 1.453125, 'learning_rate': 3.279054916985951e-05,
'epoch': 1.03}
{'loss': 2.8153, 'grad_norm': 1.59375, 'learning_rate': 3.24712643678161e-05,
'epoch': 1.05}
{'loss': 2.8051, 'grad_norm': 1.4609375, 'learning_rate': 3.215197956577267e-05,

'epoch': 1.07}
{'loss': 2.828, 'grad_norm': 1.5546875, 'learning_rate': 3.183269476372925e-05,
'epoch': 1.09}
{'loss': 2.8353, 'grad_norm': 1.4375, 'learning_rate': 3.151340996168582e-05,
'epoch': 1.11}
{'loss': 2.8107, 'grad_norm': 1.5390625, 'learning_rate': 3.11941251596424e-05,
'epoch': 1.13}
{'loss': 2.7757, 'grad_norm': 1.53125, 'learning_rate': 3.0874840357598974e-05,
'epoch': 1.15}
{'loss': 2.8144, 'grad_norm': 1.515625, 'learning_rate': 3.055555555555556e-05,
'epoch': 1.17}
{'loss': 2.8275, 'grad_norm': 1.4609375, 'learning_rate':
3.0236270753512136e-05, 'epoch': 1.19}
{'loss': 2.8385, 'grad_norm': 1.5, 'learning_rate': 2.9916985951468712e-05,
'epoch': 1.21}
{'loss': 2.8219, 'grad_norm': 1.5390625, 'learning_rate':
2.9597701149425288e-05, 'epoch': 1.23}
{'loss': 2.8103, 'grad_norm': 1.5234375, 'learning_rate':
2.9278416347381864e-05, 'epoch': 1.25}
{'loss': 2.7965, 'grad_norm': 1.5078125, 'learning_rate':
2.8959131545338443e-05, 'epoch': 1.26}
{'loss': 2.764, 'grad_norm': 1.4765625, 'learning_rate': 2.863984674329502e-05,
'epoch': 1.28}
{'loss': 2.7766, 'grad_norm': 1.7109375, 'learning_rate': 2.83205619412516e-05,
'epoch': 1.3}
{'loss': 2.8012, 'grad_norm': 1.65625, 'learning_rate': 2.8001277139208177e-05,
'epoch': 1.32}
{'loss': 2.8308, 'grad_norm': 1.6015625, 'learning_rate':
2.7681992337164753e-05, 'epoch': 1.34}
{'loss': 2.7968, 'grad_norm': 1.484375, 'learning_rate': 2.736270753512133e-05,
'epoch': 1.36}
{'loss': 2.8111, 'grad_norm': 1.5078125, 'learning_rate':
2.7043422733077904e-05, 'epoch': 1.38}
{'loss': 2.8311, 'grad_norm': 1.5390625, 'learning_rate': 2.672413793103448e-05,
'epoch': 1.4}
{'loss': 2.8056, 'grad_norm': 1.46875, 'learning_rate': 2.6404853128991063e-05,
'epoch': 1.42}
{'loss': 2.8139, 'grad_norm': 1.4921875, 'learning_rate': 2.608556832694764e-05,
'epoch': 1.44}
{'loss': 2.7877, 'grad_norm': 1.59375, 'learning_rate': 2.5766283524904218e-05,
'epoch': 1.46}
{'loss': 2.7551, 'grad_norm': 1.59375, 'learning_rate': 2.5446998722860794e-05,
'epoch': 1.48}
{'loss': 2.7763, 'grad_norm': 1.5859375, 'learning_rate': 2.512771392081737e-05,
'epoch': 1.49}
{'loss': 2.8001, 'grad_norm': 1.640625, 'learning_rate': 2.480842911877395e-05,
'epoch': 1.51}
{'loss': 2.7705, 'grad_norm': 1.546875, 'learning_rate': 2.4489144316730525e-05,

'epoch': 1.53}
{'loss': 2.7748, 'grad_norm': 1.453125, 'learning_rate': 2.41698595146871e-05,
'epoch': 1.55}
{'loss': 2.8082, 'grad_norm': 1.53125, 'learning_rate': 2.385057471264368e-05,
'epoch': 1.57}
{'loss': 2.8064, 'grad_norm': 1.5, 'learning_rate': 2.353128991060026e-05,
'epoch': 1.59}
{'loss': 2.8021, 'grad_norm': 1.5859375, 'learning_rate':
2.3212005108556835e-05, 'epoch': 1.61}
{'loss': 2.7661, 'grad_norm': 1.4453125, 'learning_rate': 2.289272030651341e-05,
'epoch': 1.63}
{'loss': 2.7735, 'grad_norm': 1.5078125, 'learning_rate': 2.257343550446999e-05,
'epoch': 1.65}
{'loss': 2.8245, 'grad_norm': 1.6328125, 'learning_rate':
2.2254150702426565e-05, 'epoch': 1.67}
{'loss': 2.8062, 'grad_norm': 1.546875, 'learning_rate': 2.193486590038314e-05,
'epoch': 1.69}
{'loss': 2.75, 'grad_norm': 1.734375, 'learning_rate': 2.161558109833972e-05,
'epoch': 1.7}
{'loss': 2.7761, 'grad_norm': 1.5546875, 'learning_rate':
2.1296296296296296e-05, 'epoch': 1.72}
{'loss': 2.794, 'grad_norm': 3.28125, 'learning_rate': 2.0977011494252875e-05,
'epoch': 1.74}
{'loss': 2.7787, 'grad_norm': 1.453125, 'learning_rate': 2.0657726692209455e-05,
'epoch': 1.76}
{'loss': 2.8257, 'grad_norm': 1.7734375, 'learning_rate': 2.033844189016603e-05,
'epoch': 1.78}
{'loss': 2.7986, 'grad_norm': 1.5, 'learning_rate': 2.0019157088122606e-05,
'epoch': 1.8}
{'loss': 2.7985, 'grad_norm': 1.4609375, 'learning_rate':
1.9699872286079182e-05, 'epoch': 1.82}
{'loss': 2.7948, 'grad_norm': 1.5390625, 'learning_rate': 1.938058748403576e-05,
'epoch': 1.84}
{'loss': 2.8099, 'grad_norm': 1.515625, 'learning_rate': 1.9061302681992337e-05,
'epoch': 1.86}
{'loss': 2.7764, 'grad_norm': 1.59375, 'learning_rate': 1.8742017879948913e-05,
'epoch': 1.88}
{'loss': 2.7803, 'grad_norm': 1.6171875, 'learning_rate':
1.8422733077905492e-05, 'epoch': 1.9}
{'loss': 2.7483, 'grad_norm': 1.5703125, 'learning_rate': 1.810344827586207e-05,
'epoch': 1.92}
{'loss': 2.8057, 'grad_norm': 1.4921875, 'learning_rate':
1.7784163473818647e-05, 'epoch': 1.93}
{'loss': 2.8122, 'grad_norm': 1.5546875, 'learning_rate':
1.7464878671775226e-05, 'epoch': 1.95}
{'loss': 2.7982, 'grad_norm': 1.5, 'learning_rate': 1.7145593869731802e-05,
'epoch': 1.97}
{'loss': 2.7756, 'grad_norm': 1.4921875, 'learning_rate':

1.6826309067688378e-05, 'epoch': 1.99}
{'loss': 2.8078, 'grad_norm': 1.515625, 'learning_rate': 1.6507024265644954e-05,
'epoch': 2.01}
{'loss': 2.7967, 'grad_norm': 1.59375, 'learning_rate': 1.6187739463601533e-05,
'epoch': 2.03}
{'loss': 2.7701, 'grad_norm': 1.46875, 'learning_rate': 1.586845466155811e-05,
'epoch': 2.05}
{'loss': 2.7676, 'grad_norm': 1.953125, 'learning_rate': 1.5549169859514688e-05,
'epoch': 2.07}
{'loss': 2.7802, 'grad_norm': 1.4921875, 'learning_rate':
1.5229885057471265e-05, 'epoch': 2.09}
{'loss': 2.7913, 'grad_norm': 1.5, 'learning_rate': 1.4910600255427843e-05,
'epoch': 2.11}
{'loss': 2.7831, 'grad_norm': 1.5546875, 'learning_rate':
1.4591315453384419e-05, 'epoch': 2.13}
{'loss': 2.7922, 'grad_norm': 1.5625, 'learning_rate': 1.4272030651340998e-05,
'epoch': 2.15}
{'loss': 2.7574, 'grad_norm': 1.640625, 'learning_rate': 1.3952745849297574e-05,
'epoch': 2.16}
{'loss': 2.7755, 'grad_norm': 1.625, 'learning_rate': 1.3633461047254151e-05,
'epoch': 2.18}
{'loss': 2.785, 'grad_norm': 1.53125, 'learning_rate': 1.331417624521073e-05,
'epoch': 2.2}
{'loss': 2.8151, 'grad_norm': 1.484375, 'learning_rate': 1.2994891443167306e-05,
'epoch': 2.22}
{'loss': 2.7735, 'grad_norm': 1.53125, 'learning_rate': 1.2675606641123882e-05,
'epoch': 2.24}
{'loss': 2.8273, 'grad_norm': 1.5078125, 'learning_rate':
1.2356321839080461e-05, 'epoch': 2.26}
{'loss': 2.7992, 'grad_norm': 1.53125, 'learning_rate': 1.2037037037037037e-05,
'epoch': 2.28}
{'loss': 2.7894, 'grad_norm': 1.6015625, 'learning_rate':
1.1717752234993615e-05, 'epoch': 2.3}
{'loss': 2.7625, 'grad_norm': 1.484375, 'learning_rate': 1.1398467432950192e-05,
'epoch': 2.32}
{'loss': 2.7802, 'grad_norm': 1.5703125, 'learning_rate': 1.107918263090677e-05,
'epoch': 2.34}
{'loss': 2.7991, 'grad_norm': 1.6015625, 'learning_rate':
1.0759897828863347e-05, 'epoch': 2.36}
{'loss': 2.8367, 'grad_norm': 1.609375, 'learning_rate': 1.0440613026819925e-05,
'epoch': 2.38}
{'loss': 2.7701, 'grad_norm': 1.4296875, 'learning_rate': 1.01213282247765e-05,
'epoch': 2.39}
{'loss': 2.8247, 'grad_norm': 1.625, 'learning_rate': 9.80204342273308e-06,
'epoch': 2.41}
{'loss': 2.7678, 'grad_norm': 1.5546875, 'learning_rate': 9.482758620689655e-06,
'epoch': 2.43}
{'loss': 2.73, 'grad_norm': 1.46875, 'learning_rate': 9.163473818646233e-06,

'epoch': 2.45}
{'loss': 2.7863, 'grad_norm': 1.4609375, 'learning_rate': 8.84418901660281e-06,
'epoch': 2.47}
{'loss': 2.7866, 'grad_norm': 1.6171875, 'learning_rate': 8.524904214559388e-06,
'epoch': 2.49}
{'loss': 2.8183, 'grad_norm': 1.4609375, 'learning_rate': 8.205619412515965e-06,
'epoch': 2.51}
{'loss': 2.7862, 'grad_norm': 1.40625, 'learning_rate': 7.886334610472541e-06,
'epoch': 2.53}
{'loss': 2.7811, 'grad_norm': 1.453125, 'learning_rate': 7.567049808429119e-06,
'epoch': 2.55}
{'loss': 2.8168, 'grad_norm': 1.6015625, 'learning_rate': 7.247765006385697e-06,
'epoch': 2.57}
{'loss': 2.8022, 'grad_norm': 1.4296875, 'learning_rate': 6.928480204342274e-06,
'epoch': 2.59}
{'loss': 2.8059, 'grad_norm': 1.421875, 'learning_rate': 6.609195402298851e-06,
'epoch': 2.61}
{'loss': 2.7775, 'grad_norm': 1.3984375, 'learning_rate': 6.289910600255428e-06,
'epoch': 2.62}
{'loss': 2.7785, 'grad_norm': 1.578125, 'learning_rate': 5.970625798212005e-06,
'epoch': 2.64}
{'loss': 2.7733, 'grad_norm': 1.5859375, 'learning_rate': 5.651340996168582e-06,
'epoch': 2.66}
{'loss': 2.764, 'grad_norm': 1.625, 'learning_rate': 5.3320561941251596e-06,
'epoch': 2.68}
{'loss': 2.7917, 'grad_norm': 1.4921875, 'learning_rate': 5.012771392081737e-06,
'epoch': 2.7}
{'loss': 2.7791, 'grad_norm': 1.6484375, 'learning_rate':
4.6934865900383146e-06, 'epoch': 2.72}
{'loss': 2.777, 'grad_norm': 1.5390625, 'learning_rate': 4.374201787994891e-06,
'epoch': 2.74}
{'loss': 2.7795, 'grad_norm': 1.4765625, 'learning_rate': 4.054916985951469e-06,
'epoch': 2.76}
{'loss': 2.77, 'grad_norm': 1.640625, 'learning_rate': 3.7356321839080462e-06,
'epoch': 2.78}
{'loss': 2.8122, 'grad_norm': 1.5625, 'learning_rate': 3.4163473818646237e-06,
'epoch': 2.8}
{'loss': 2.7944, 'grad_norm': 1.5703125, 'learning_rate': 3.097062579821201e-06,
'epoch': 2.82}
{'loss': 2.767, 'grad_norm': 1.640625, 'learning_rate': 2.777777777777778e-06,
'epoch': 2.84}
{'loss': 2.7909, 'grad_norm': 1.4765625, 'learning_rate':
2.4584929757343554e-06, 'epoch': 2.85}
{'loss': 2.7625, 'grad_norm': 1.546875, 'learning_rate': 2.1392081736909325e-06,
'epoch': 2.87}
{'loss': 2.785, 'grad_norm': 1.5625, 'learning_rate': 1.8199233716475098e-06,
'epoch': 2.89}
{'loss': 2.8078, 'grad_norm': 1.53125, 'learning_rate': 1.5006385696040869e-06,

```
'epoch': 2.91}
{'loss': 2.7834, 'grad_norm': 1.421875, 'learning_rate': 1.1813537675606642e-06,
'epoch': 2.93}
{'loss': 2.8071, 'grad_norm': 1.6953125, 'learning_rate': 8.620689655172415e-07,
'epoch': 2.95}
{'loss': 2.8107, 'grad_norm': 1.5625, 'learning_rate': 5.427841634738186e-07,
'epoch': 2.97}
{'loss': 2.7904, 'grad_norm': 1.4609375, 'learning_rate': 2.234993614303959e-07,
'epoch': 2.99}
{'train_runtime': 466.5808, 'train_samples_per_second': 53.676,
'train_steps_per_second': 3.356, 'train_loss': 2.819781905085343, 'epoch': 3.0}
100%|                    | 1566/1566 [07:46<00:00,  3.36it/s]
Model saved to ./myanmar_xglm_word

real    8m19.294s
user    7m49.434s
sys     0m40.661s
```

0.34 Evaluation with Closed Test Data

အရင်ဆုံး finetuned **လုပ်ထားတဲ့ မော်ဒယ်ကို** closed test **ဒေတာနဲ့ပဲ** evaluation **လုပ်ကြည့်ရအောင်။**

[155]:
```
!./eval-closed.sh
```

```
2026-05-01 23:06:26.587383: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-01 23:06:26.610502: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1777651586.636960 3555621 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1777651586.645750 3555621 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1777651586.668323 3555621 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777651586.668389 3555621 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777651586.668399 3555621 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
```

```
target more than once.
W0000 00:00:1777651586.668403 3555621 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2026-05-01 23:06:26.674293: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
Using device: cuda
Loading fine-tuned model from: ./myanmar_xglm_word
Tokenizing test set (10000 lines)…
Evaluating with Stride=64, Context=128…
Passing a tuple of `past_key_values` is deprecated and will be removed in
Transformers v4.58.0. You should pass an instance of `EncoderDecoderCache`
instead, e.g.
`past_key_values=EncoderDecoderCache.from_legacy_cache(past_key_values)`.

Test Results -> PPL: 18.28, Avg NLL: 2.9061 nats

real    1m28.350s
user    1m28.900s
sys     0m2.246s
```

0.35 Evaluation with Open Test Data

ဒီတခါတော့ ပိုအရေးကြီးတဲ့ open test ဒေတာနဲ့ finetuning လုပ်ကြည့်ပါမယ်။

[156]:
```
!./eval.sh
```

```
2026-05-01 23:08:23.695043: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-01 23:08:23.709772: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1777651703.727214 3555905 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1777651703.732778 3555905 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1777651703.747078 3555905 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
```

```
target more than once.
W0000 00:00:1777651703.747118 3555905 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777651703.747121 3555905 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777651703.747127 3555905 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2026-05-01 23:08:23.750847: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
Using device: cuda
Loading fine-tuned model from: ./myanmar_xglm_word
Tokenizing test set (1000 lines)…
Evaluating with Stride=64, Context=128…
Passing a tuple of `past_key_values` is deprecated and will be removed in
Transformers v4.58.0. You should pass an instance of `EncoderDecoderCache`
instead, e.g.
`past_key_values=EncoderDecoderCache.from_legacy_cache(past_key_values)`.

Test Results -> PPL: 18.21, Avg NLL: 2.9021 nats

real    0m24.851s
user    0m26.177s
sys     0m1.721s
```

0.36 Text Generation with XGLM Finetuned LM

ရလဒ်ကတော့ LSTM လောက် မကောင်းတာကို တွေ့ရလိမ့်မယ်။
Text generation လုပ်ကြည့်ဖြီး finetuned model ကို လေ့လာရအောင်။

[92]:
```
!cat gen.sh
```

```
#!/bin/bash
# Added --top_k and --top_p (Nucleus sampling) - standard practices for modern
LLMs

python transformer_lm.py --mode generate \
                  --model_path ./myanmar_xglm_word \
                  --prompt "ပြည်ထောင်စု" \
                  --gen_length 200 \
                  --temperature 0.7 \
                  --top_k 50 \
                  --top_p 0.95
```

[93]:
```
!./gen.sh
```

```
2026-05-02 15:09:23.666810: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-02 15:09:23.676116: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1777709363.686797 3622283 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1777709363.690333 3622283 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1777709363.699245 3622283 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777709363.699261 3622283 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777709363.699263 3622283 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777709363.699265 3622283 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2026-05-02 15:09:23.701574: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
Using device: cuda
Loading fine-tuned model from: ./myanmar_xglm_word

Prompt: ပြည်ထောင်စု
Generating:
ပြည်ထောင်စု ရွေးကောက်ပွဲ ကော်မရှင်သည် ၂၀၁၉ ခုနှစ် နိုဝင်ဘာ ၁ ရက် တွင် ကျင်းပ သော
ပြည်ထောင်စု ရွေးကောက်ပွဲ ကော်မရှင် ဥက္ကဋ္ဌ ရွေးကောက်ပွဲ ကို ကျင်းပ ရန် သဘောတူ
ခဲ့ သည်
```

နောက်တစ်ခေါက် ထပ် generate လုပ်ခိုင်းကြည့်မယ်

[160]:
```
!./gen.sh
```

```
2026-05-01 23:14:07.843582: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-01 23:14:07.856815: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1777652047.872796 3557020 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1777652047.877939 3557020 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1777652047.891292 3557020 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777652047.891329 3557020 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777652047.891332 3557020 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777652047.891334 3557020 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2026-05-01 23:14:07.894854: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
Using device: cuda
Loading fine-tuned model from: ./myanmar_xglm_word

Prompt: ပြည်ထောင်စု
Generating:
ပြည်ထောင်စု ကြီးခိုင်ရေး ပါတီ ( ပါတီ ) သည် ၁၉၉၃ ခုနှစ် တွင် စတင် ဖွဲ့စည်း ခဲ့
သည်
```

နောက်ထပ် တစ်ကြိမ် ထပ် generate လုပ်ခိုင်းကြည့်မယ်

[161]:
```
!./gen.sh
```

```
2026-05-01 23:15:07.914115: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
```

```
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-01 23:15:07.928626: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1777652107.946286 3557224 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1777652107.952125 3557224 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1777652107.966249 3557224 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777652107.966289 3557224 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777652107.966292 3557224 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777652107.966294 3557224 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2026-05-01 23:15:07.969999: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
Using device: cuda
Loading fine-tuned model from: ./myanmar_xglm_word

Prompt: ပြည်ထောင်စု
Generating:
ပြည်ထောင်စု ဝန်ကြီး များ နှင့် ဝန်ကြီးဌာန ၏ ဝန်ထမ်း များ နှင့်အတူ အစည်းအဝေး
အမှတ်စဉ် ၂၂ သို့ ရောက်ရှိ ခဲ့ သည်
```

0.37 LSTM vs XGLM

Your LSTM (~1 Million parameters): It is tiny. When you fed it your ~40,000-line dataset, it had more than enough capacity to simply memorize the entire text word-for-word. A PPL of 2.12 means the model is almost 100% certain of the next word. It's not learning "Myanmar grammar"; it's acting like a hard drive that just memorized the file. (If you test it on totally wild out-of-domain text, it will crash). Your XGLM (564 Million parameters): It is a massive brain. When you fine-tune it for just 3 epochs on a relatively small dataset, the model barely has time to adjust its 564 million weights before the training stops. It underfit. It's like asking a genius to read a

10-page pamphlet and then instantly change their whole speaking style—they will mostly stick to their original way of speaking.

reason 2: The "Tokenization Inequality" (Apples vs. Oranges) You cannot directly compare the PPL of a Word-level LSTM with a Subword-level Transformer.

LSTM: Predicts 1 item out of 99 possible words. XGLM: Predicts 1 item out of 256,000 possible subwords. Because XGLM breaks words into smaller pieces (e.g., "ပြည်" might become 2 or 3 subwords), it has to make predictions more frequently, and the math naturally results in a higher PPL. To compare them fairly, you would have to calculate Bits-Per-Character (BPC), where XGLM would likely win because it understands character context better.

0.38 Fine-tuning XGLM with Optimized Parameters

[166]:
```
!cat ./train_optimize.sh
```

```
#!/bin/bash

# Fine-tuning XGLM with optimized parameters for smaller datasets
time python transformer_lm.py --mode train \
                  --train_file ../data/mypos_v3.word.clean \
                  --model_path ./myanmar_xglm_word_optimize \
                  --base_model facebook/xglm-564M \
                  --seq_len 512 \
                  --epochs 10 \
                  --batch_size 4 \
                  --lr 2e-5
```

[167]:
```
!./train_optimize.sh
```

```
2026-05-01 23:28:41.634971: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-01 23:28:41.657260: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1777652921.682828 3558656 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1777652921.691453 3558656 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1777652921.713381 3558656 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
```

```
W0000 00:00:1777652921.713444 3558656 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777652921.713449 3558656 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777652921.713457 3558656 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2026-05-01 23:28:41.719055: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
Using device: cuda
Loading base model: facebook/xglm-564M
Loading training data: ../data/mypos_v3.word.clean
Tokenizing: 100%|        | 43196/43196 [00:03<00:00, 12815.28 examples/s]
Grouping into chunks: 100%|   | 43196/43196 [00:06<00:00, 7057.15 examples/s]
Starting Transformer Fine-Tuning…
  0%|                                                    | 0/2590 [00:00<?,
?it/s]Passing a tuple of `past_key_values` is deprecated and will be removed in
Transformers v4.58.0. You should pass an instance of `EncoderDecoderCache`
instead, e.g.
`past_key_values=EncoderDecoderCache.from_legacy_cache(past_key_values)`.
{'loss': 3.0648, 'grad_norm': 1.6875, 'learning_rate': 1.9930501930501933e-05,
'epoch': 0.04}
{'loss': 3.0451, 'grad_norm': 1.5625, 'learning_rate': 1.9853281853281855e-05,
'epoch': 0.08}
{'loss': 3.0221, 'grad_norm': 1.65625, 'learning_rate': 1.9776061776061776e-05,
'epoch': 0.12}
{'loss': 2.9895, 'grad_norm': 1.3984375, 'learning_rate': 1.96988416988417e-05,
'epoch': 0.15}
{'loss': 2.9897, 'grad_norm': 1.421875, 'learning_rate': 1.9621621621621623e-05,
'epoch': 0.19}
{'loss': 2.9595, 'grad_norm': 1.3046875, 'learning_rate':
1.9544401544401545e-05, 'epoch': 0.23}
{'loss': 3.0027, 'grad_norm': 1.328125, 'learning_rate': 1.946718146718147e-05,
'epoch': 0.27}
{'loss': 2.9568, 'grad_norm': 1.3515625, 'learning_rate': 1.938996138996139e-05,
'epoch': 0.31}
{'loss': 2.966, 'grad_norm': 1.2734375, 'learning_rate': 1.9312741312741313e-05,
'epoch': 0.35}
{'loss': 2.9334, 'grad_norm': 1.296875, 'learning_rate': 1.9235521235521238e-05,
'epoch': 0.39}
{'loss': 2.9325, 'grad_norm': 1.2890625, 'learning_rate': 1.915830115830116e-05,
'epoch': 0.42}
{'loss': 2.9361, 'grad_norm': 1.2421875, 'learning_rate':
```

1.9081081081081084e-05, 'epoch': 0.46}
{'loss': 2.9512, 'grad_norm': 1.2890625, 'learning_rate':
1.9003861003861006e-05, 'epoch': 0.5}
{'loss': 2.9189, 'grad_norm': 1.1875, 'learning_rate': 1.8926640926640928e-05,
'epoch': 0.54}
{'loss': 2.9308, 'grad_norm': 1.1953125, 'learning_rate':
1.8849420849420853e-05, 'epoch': 0.58}
{'loss': 2.914, 'grad_norm': 1.2265625, 'learning_rate': 1.8772200772200774e-05,
'epoch': 0.62}
{'loss': 2.9163, 'grad_norm': 1.21875, 'learning_rate': 1.8694980694980696e-05,
'epoch': 0.66}
{'loss': 2.9075, 'grad_norm': 1.21875, 'learning_rate': 1.8617760617760617e-05,
'epoch': 0.69}
{'loss': 2.9297, 'grad_norm': 1.2265625, 'learning_rate':
1.8540540540540542e-05, 'epoch': 0.73}
{'loss': 2.9631, 'grad_norm': 1.171875, 'learning_rate': 1.8463320463320464e-05,
'epoch': 0.77}
{'loss': 2.9029, 'grad_norm': 1.1796875, 'learning_rate':
1.8386100386100386e-05, 'epoch': 0.81}
{'loss': 2.9207, 'grad_norm': 1.21875, 'learning_rate': 1.830888030888031e-05,
'epoch': 0.85}
{'loss': 2.8814, 'grad_norm': 1.203125, 'learning_rate': 1.8231660231660232e-05,
'epoch': 0.89}
{'loss': 2.8928, 'grad_norm': 1.140625, 'learning_rate': 1.8154440154440157e-05,
'epoch': 0.93}
{'loss': 2.897, 'grad_norm': 1.140625, 'learning_rate': 1.807722007722008e-05,
'epoch': 0.97}
{'loss': 2.9008, 'grad_norm': 1.1640625, 'learning_rate': 1.8e-05, 'epoch': 1.0}
{'loss': 2.8954, 'grad_norm': 1.1796875, 'learning_rate':
1.7922779922779925e-05, 'epoch': 1.04}
{'loss': 2.9366, 'grad_norm': 1.140625, 'learning_rate': 1.7845559845559847e-05,
'epoch': 1.08}
{'loss': 2.8857, 'grad_norm': 1.25, 'learning_rate': 1.7768339768339772e-05,
'epoch': 1.12}
{'loss': 2.9031, 'grad_norm': 1.1328125, 'learning_rate':
1.7691119691119694e-05, 'epoch': 1.16}
{'loss': 2.867, 'grad_norm': 1.1328125, 'learning_rate': 1.7613899613899615e-05,
'epoch': 1.2}
{'loss': 2.8944, 'grad_norm': 1.1875, 'learning_rate': 1.7536679536679537e-05,
'epoch': 1.24}
{'loss': 2.8822, 'grad_norm': 1.171875, 'learning_rate': 1.745945945945946e-05,
'epoch': 1.27}
{'loss': 2.8852, 'grad_norm': 1.2265625, 'learning_rate':
1.7382239382239383e-05, 'epoch': 1.31}
{'loss': 2.8761, 'grad_norm': 1.1484375, 'learning_rate':
1.7305019305019305e-05, 'epoch': 1.35}
{'loss': 2.877, 'grad_norm': 1.1328125, 'learning_rate': 1.722779922779923e-05,
'epoch': 1.39}

{'loss': 2.8846, 'grad_norm': 1.1328125, 'learning_rate':
1.7150579150579152e-05, 'epoch': 1.43}
{'loss': 2.8788, 'grad_norm': 1.21875, 'learning_rate': 1.7073359073359073e-05,
'epoch': 1.47}
{'loss': 2.8547, 'grad_norm': 1.1171875, 'learning_rate': 1.6996138996139e-05,
'epoch': 1.51}
{'loss': 2.8716, 'grad_norm': 1.21875, 'learning_rate': 1.691891891891892e-05,
'epoch': 1.54}
{'loss': 2.8678, 'grad_norm': 1.171875, 'learning_rate': 1.6841698841698845e-05,
'epoch': 1.58}
{'loss': 2.8838, 'grad_norm': 1.171875, 'learning_rate': 1.6764478764478767e-05,
'epoch': 1.62}
{'loss': 2.8687, 'grad_norm': 1.1484375, 'learning_rate':
1.6687258687258688e-05, 'epoch': 1.66}
{'loss': 2.8432, 'grad_norm': 1.1640625, 'learning_rate':
1.6610038610038613e-05, 'epoch': 1.7}
{'loss': 2.877, 'grad_norm': 1.1875, 'learning_rate': 1.6532818532818535e-05,
'epoch': 1.74}
{'loss': 2.8717, 'grad_norm': 1.125, 'learning_rate': 1.6455598455598456e-05,
'epoch': 1.78}
{'loss': 2.869, 'grad_norm': 1.203125, 'learning_rate': 1.6378378378378378e-05,
'epoch': 1.81}
{'loss': 2.8861, 'grad_norm': 1.1015625, 'learning_rate':
1.6301158301158303e-05, 'epoch': 1.85}
{'loss': 2.8769, 'grad_norm': 1.109375, 'learning_rate': 1.6223938223938225e-05,
'epoch': 1.89}
{'loss': 2.8597, 'grad_norm': 1.1484375, 'learning_rate':
1.6146718146718146e-05, 'epoch': 1.93}
{'loss': 2.8549, 'grad_norm': 1.125, 'learning_rate': 1.606949806949807e-05,
'epoch': 1.97}
{'loss': 2.8588, 'grad_norm': 1.140625, 'learning_rate': 1.5992277992277993e-05,
'epoch': 2.01}
{'loss': 2.8495, 'grad_norm': 1.203125, 'learning_rate': 1.5915057915057918e-05,
'epoch': 2.05}
{'loss': 2.8733, 'grad_norm': 1.21875, 'learning_rate': 1.583783783783784e-05,
'epoch': 2.08}
{'loss': 2.837, 'grad_norm': 1.125, 'learning_rate': 1.576061776061776e-05,
'epoch': 2.12}
{'loss': 2.836, 'grad_norm': 1.1015625, 'learning_rate': 1.5683397683397686e-05,
'epoch': 2.16}
{'loss': 2.8717, 'grad_norm': 1.1171875, 'learning_rate':
1.5606177606177608e-05, 'epoch': 2.2}
{'loss': 2.8819, 'grad_norm': 1.078125, 'learning_rate': 1.5528957528957533e-05,
'epoch': 2.24}
{'loss': 2.8319, 'grad_norm': 1.1484375, 'learning_rate':
1.5451737451737454e-05, 'epoch': 2.28}
{'loss': 2.8583, 'grad_norm': 1.1328125, 'learning_rate':
1.5374517374517376e-05, 'epoch': 2.32}

{'loss': 2.87, 'grad_norm': 1.1796875, 'learning_rate': 1.5297297297297297e-05, 'epoch': 2.36}
{'loss': 2.8568, 'grad_norm': 1.1015625, 'learning_rate': 1.522007722007722e-05, 'epoch': 2.39}
{'loss': 2.8655, 'grad_norm': 1.1015625, 'learning_rate': 1.5142857142857144e-05, 'epoch': 2.43}
{'loss': 2.8498, 'grad_norm': 1.125, 'learning_rate': 1.5065637065637067e-05, 'epoch': 2.47}
{'loss': 2.8526, 'grad_norm': 1.109375, 'learning_rate': 1.4988416988416989e-05, 'epoch': 2.51}
{'loss': 2.8514, 'grad_norm': 1.1953125, 'learning_rate': 1.4911196911196912e-05, 'epoch': 2.55}
{'loss': 2.8498, 'grad_norm': 1.1484375, 'learning_rate': 1.4833976833976836e-05, 'epoch': 2.59}
{'loss': 2.8755, 'grad_norm': 1.171875, 'learning_rate': 1.4756756756756759e-05, 'epoch': 2.63}
{'loss': 2.8566, 'grad_norm': 1.1171875, 'learning_rate': 1.467953667953668e-05, 'epoch': 2.66}
{'loss': 2.8587, 'grad_norm': 1.1015625, 'learning_rate': 1.4602316602316604e-05, 'epoch': 2.7}
{'loss': 2.8601, 'grad_norm': 1.0859375, 'learning_rate': 1.4525096525096527e-05, 'epoch': 2.74}
{'loss': 2.8666, 'grad_norm': 1.1171875, 'learning_rate': 1.444787644787645e-05, 'epoch': 2.78}
{'loss': 2.8653, 'grad_norm': 1.078125, 'learning_rate': 1.4370656370656374e-05, 'epoch': 2.82}
{'loss': 2.8774, 'grad_norm': 1.1015625, 'learning_rate': 1.4293436293436294e-05, 'epoch': 2.86}
{'loss': 2.8551, 'grad_norm': 1.09375, 'learning_rate': 1.4216216216216217e-05, 'epoch': 2.9}
{'loss': 2.8395, 'grad_norm': 1.1640625, 'learning_rate': 1.413899613899614e-05, 'epoch': 2.93}
{'loss': 2.8268, 'grad_norm': 1.21875, 'learning_rate': 1.4061776061776062e-05, 'epoch': 2.97}
{'loss': 2.8675, 'grad_norm': 1.1328125, 'learning_rate': 1.3984555984555985e-05, 'epoch': 3.01}
{'loss': 2.8381, 'grad_norm': 1.0703125, 'learning_rate': 1.3907335907335908e-05, 'epoch': 3.05}
{'loss': 2.8632, 'grad_norm': 1.1171875, 'learning_rate': 1.3830115830115832e-05, 'epoch': 3.09}
{'loss': 2.8523, 'grad_norm': 1.1796875, 'learning_rate': 1.3752895752895753e-05, 'epoch': 3.13}
{'loss': 2.8604, 'grad_norm': 1.1015625, 'learning_rate': 1.3675675675675677e-05, 'epoch': 3.17}
{'loss': 2.8757, 'grad_norm': 1.078125, 'learning_rate': 1.35984555984556e-05, 'epoch': 3.2}
{'loss': 2.8241, 'grad_norm': 1.1171875, 'learning_rate': 1.3521235521235523e-05, 'epoch': 3.24}

{'loss': 2.8663, 'grad_norm': 1.15625, 'learning_rate': 1.3444015444015447e-05,
'epoch': 3.28}
{'loss': 2.8522, 'grad_norm': 1.0859375, 'learning_rate':
1.3366795366795368e-05, 'epoch': 3.32}
{'loss': 2.8788, 'grad_norm': 1.09375, 'learning_rate': 1.3289575289575291e-05,
'epoch': 3.36}
{'loss': 2.8538, 'grad_norm': 1.1171875, 'learning_rate':
1.3212355212355213e-05, 'epoch': 3.4}
{'loss': 2.8201, 'grad_norm': 1.2109375, 'learning_rate':
1.3135135135135135e-05, 'epoch': 3.44}
{'loss': 2.8185, 'grad_norm': 1.1484375, 'learning_rate':
1.3057915057915058e-05, 'epoch': 3.47}
{'loss': 2.8357, 'grad_norm': 1.1015625, 'learning_rate':
1.2980694980694981e-05, 'epoch': 3.51}
{'loss': 2.8469, 'grad_norm': 1.1875, 'learning_rate': 1.2903474903474905e-05,
'epoch': 3.55}
{'loss': 2.845, 'grad_norm': 1.125, 'learning_rate': 1.2826254826254826e-05,
'epoch': 3.59}
{'loss': 2.854, 'grad_norm': 1.1171875, 'learning_rate': 1.274903474903475e-05,
'epoch': 3.63}
{'loss': 2.8508, 'grad_norm': 1.125, 'learning_rate': 1.2671814671814673e-05,
'epoch': 3.67}
{'loss': 2.8495, 'grad_norm': 1.1328125, 'learning_rate':
1.2594594594594596e-05, 'epoch': 3.71}
{'loss': 2.8541, 'grad_norm': 1.1328125, 'learning_rate': 1.251737451737452e-05,
'epoch': 3.75}
{'loss': 2.8474, 'grad_norm': 1.15625, 'learning_rate': 1.2440154440154441e-05,
'epoch': 3.78}
{'loss': 2.8449, 'grad_norm': 1.1015625, 'learning_rate':
1.2362934362934364e-05, 'epoch': 3.82}
{'loss': 2.8383, 'grad_norm': 1.125, 'learning_rate': 1.2285714285714288e-05,
'epoch': 3.86}
{'loss': 2.8332, 'grad_norm': 1.1171875, 'learning_rate':
1.2208494208494211e-05, 'epoch': 3.9}
{'loss': 2.8598, 'grad_norm': 1.109375, 'learning_rate': 1.2131274131274133e-05,
'epoch': 3.94}
{'loss': 2.8515, 'grad_norm': 1.078125, 'learning_rate': 1.2054054054054054e-05,
'epoch': 3.98}
{'loss': 2.8472, 'grad_norm': 1.125, 'learning_rate': 1.1976833976833977e-05,
'epoch': 4.02}
{'loss': 2.8734, 'grad_norm': 1.0546875, 'learning_rate':
1.1899613899613899e-05, 'epoch': 4.05}
{'loss': 2.8501, 'grad_norm': 1.21875, 'learning_rate': 1.1822393822393822e-05,
'epoch': 4.09}
{'loss': 2.8376, 'grad_norm': 1.078125, 'learning_rate': 1.1745173745173746e-05,
'epoch': 4.13}
{'loss': 2.851, 'grad_norm': 1.1171875, 'learning_rate': 1.1667953667953669e-05,
'epoch': 4.17}

{'loss': 2.8485, 'grad_norm': 1.4140625, 'learning_rate':
1.1590733590733592e-05, 'epoch': 4.21}
{'loss': 2.8508, 'grad_norm': 1.1015625, 'learning_rate':
1.1513513513513514e-05, 'epoch': 4.25}
{'loss': 2.8346, 'grad_norm': 1.125, 'learning_rate': 1.1436293436293437e-05,
'epoch': 4.29}
{'loss': 2.8597, 'grad_norm': 1.1484375, 'learning_rate': 1.135907335907336e-05,
'epoch': 4.32}
{'loss': 2.8256, 'grad_norm': 1.1640625, 'learning_rate':
1.1281853281853284e-05, 'epoch': 4.36}
{'loss': 2.8403, 'grad_norm': 1.15625, 'learning_rate': 1.1204633204633205e-05,
'epoch': 4.4}
{'loss': 2.8325, 'grad_norm': 1.09375, 'learning_rate': 1.1127413127413129e-05,
'epoch': 4.44}
{'loss': 2.8535, 'grad_norm': 1.1015625, 'learning_rate':
1.1050193050193052e-05, 'epoch': 4.48}
{'loss': 2.8486, 'grad_norm': 1.1015625, 'learning_rate':
1.0972972972972974e-05, 'epoch': 4.52}
{'loss': 2.845, 'grad_norm': 1.046875, 'learning_rate': 1.0895752895752895e-05,
'epoch': 4.56}
{'loss': 2.8691, 'grad_norm': 1.078125, 'learning_rate': 1.0818532818532819e-05,
'epoch': 4.59}
{'loss': 2.8576, 'grad_norm': 1.109375, 'learning_rate': 1.0741312741312742e-05,
'epoch': 4.63}
{'loss': 2.8334, 'grad_norm': 1.171875, 'learning_rate': 1.0664092664092665e-05,
'epoch': 4.67}
{'loss': 2.842, 'grad_norm': 1.109375, 'learning_rate': 1.0586872586872587e-05,
'epoch': 4.71}
{'loss': 2.8498, 'grad_norm': 1.0546875, 'learning_rate': 1.050965250965251e-05,
'epoch': 4.75}
{'loss': 2.8445, 'grad_norm': 1.140625, 'learning_rate': 1.0432432432432433e-05,
'epoch': 4.79}
{'loss': 2.8221, 'grad_norm': 1.1171875, 'learning_rate':
1.0355212355212357e-05, 'epoch': 4.83}
{'loss': 2.8293, 'grad_norm': 1.1171875, 'learning_rate': 1.027799227799228e-05,
'epoch': 4.86}
{'loss': 2.8514, 'grad_norm': 1.0859375, 'learning_rate':
1.0200772200772202e-05, 'epoch': 4.9}
{'loss': 2.8392, 'grad_norm': 1.09375, 'learning_rate': 1.0123552123552125e-05,
'epoch': 4.94}
{'loss': 2.8325, 'grad_norm': 1.140625, 'learning_rate': 1.0046332046332048e-05,
'epoch': 4.98}
{'loss': 2.8533, 'grad_norm': 1.171875, 'learning_rate': 9.96911196911197e-06,
'epoch': 5.02}
{'loss': 2.8475, 'grad_norm': 1.078125, 'learning_rate': 9.891891891891893e-06,
'epoch': 5.06}
{'loss': 2.8426, 'grad_norm': 1.09375, 'learning_rate': 9.814671814671816e-06,
'epoch': 5.1}

{'loss': 2.8511, 'grad_norm': 1.0546875, 'learning_rate': 9.737451737451738e-06,
'epoch': 5.14}
{'loss': 2.8358, 'grad_norm': 1.171875, 'learning_rate': 9.660231660231661e-06,
'epoch': 5.17}
{'loss': 2.8224, 'grad_norm': 1.1484375, 'learning_rate': 9.583011583011583e-06,
'epoch': 5.21}
{'loss': 2.8339, 'grad_norm': 1.1875, 'learning_rate': 9.505791505791506e-06,
'epoch': 5.25}
{'loss': 2.8414, 'grad_norm': 1.109375, 'learning_rate': 9.42857142857143e-06,
'epoch': 5.29}
{'loss': 2.8755, 'grad_norm': 1.09375, 'learning_rate': 9.351351351351353e-06,
'epoch': 5.33}
{'loss': 2.8631, 'grad_norm': 1.1171875, 'learning_rate': 9.274131274131274e-06,
'epoch': 5.37}
{'loss': 2.8797, 'grad_norm': 1.203125, 'learning_rate': 9.196911196911198e-06,
'epoch': 5.41}
{'loss': 2.8564, 'grad_norm': 1.1875, 'learning_rate': 9.119691119691121e-06,
'epoch': 5.44}
{'loss': 2.8582, 'grad_norm': 1.0859375, 'learning_rate': 9.042471042471043e-06,
'epoch': 5.48}
{'loss': 2.8365, 'grad_norm': 1.1328125, 'learning_rate': 8.965250965250966e-06,
'epoch': 5.52}
{'loss': 2.8467, 'grad_norm': 1.1953125, 'learning_rate': 8.88803088803089e-06,
'epoch': 5.56}
{'loss': 2.8346, 'grad_norm': 1.078125, 'learning_rate': 8.810810810810811e-06,
'epoch': 5.6}
{'loss': 2.8498, 'grad_norm': 1.109375, 'learning_rate': 8.733590733590734e-06,
'epoch': 5.64}
{'loss': 2.7987, 'grad_norm': 1.125, 'learning_rate': 8.656370656370657e-06,
'epoch': 5.68}
{'loss': 2.8358, 'grad_norm': 1.1796875, 'learning_rate': 8.57915057915058e-06,
'epoch': 5.71}
{'loss': 2.8477, 'grad_norm': 1.09375, 'learning_rate': 8.501930501930502e-06,
'epoch': 5.75}
{'loss': 2.8261, 'grad_norm': 1.15625, 'learning_rate': 8.424710424710426e-06,
'epoch': 5.79}
{'loss': 2.8504, 'grad_norm': 1.1328125, 'learning_rate': 8.347490347490347e-06,
'epoch': 5.83}
{'loss': 2.8402, 'grad_norm': 1.140625, 'learning_rate': 8.27027027027027e-06,
'epoch': 5.87}
{'loss': 2.8203, 'grad_norm': 1.171875, 'learning_rate': 8.193050193050194e-06,
'epoch': 5.91}
{'loss': 2.8564, 'grad_norm': 1.171875, 'learning_rate': 8.115830115830117e-06,
'epoch': 5.95}
{'loss': 2.8336, 'grad_norm': 1.1796875, 'learning_rate': 8.038610038610039e-06,
'epoch': 5.98}
{'loss': 2.8306, 'grad_norm': 1.078125, 'learning_rate': 7.961389961389962e-06,
'epoch': 6.02}

{'loss': 2.8528, 'grad_norm': 1.125, 'learning_rate': 7.884169884169884e-06,
'epoch': 6.06}
{'loss': 2.8513, 'grad_norm': 1.0703125, 'learning_rate': 7.806949806949807e-06,
'epoch': 6.1}
{'loss': 2.8586, 'grad_norm': 1.109375, 'learning_rate': 7.72972972972973e-06,
'epoch': 6.14}
{'loss': 2.8354, 'grad_norm': 1.1328125, 'learning_rate': 7.652509652509654e-06,
'epoch': 6.18}
{'loss': 2.8662, 'grad_norm': 1.1640625, 'learning_rate': 7.575289575289576e-06,
'epoch': 6.22}
{'loss': 2.8425, 'grad_norm': 1.125, 'learning_rate': 7.498069498069499e-06,
'epoch': 6.25}
{'loss': 2.8871, 'grad_norm': 1.203125, 'learning_rate': 7.420849420849422e-06,
'epoch': 6.29}
{'loss': 2.8506, 'grad_norm': 1.09375, 'learning_rate': 7.3436293436293435e-06,
'epoch': 6.33}
{'loss': 2.8196, 'grad_norm': 1.1171875, 'learning_rate': 7.266409266409267e-06,
'epoch': 6.37}
{'loss': 2.8349, 'grad_norm': 1.1015625, 'learning_rate': 7.189189189189189e-06,
'epoch': 6.41}
{'loss': 2.8225, 'grad_norm': 1.0546875, 'learning_rate':
7.1119691119691125e-06, 'epoch': 6.45}
{'loss': 2.8347, 'grad_norm': 1.125, 'learning_rate': 7.034749034749036e-06,
'epoch': 6.49}
{'loss': 2.8493, 'grad_norm': 1.171875, 'learning_rate': 6.957528957528958e-06,
'epoch': 6.53}
{'loss': 2.8424, 'grad_norm': 1.1328125, 'learning_rate': 6.880308880308882e-06,
'epoch': 6.56}
{'loss': 2.8236, 'grad_norm': 1.21875, 'learning_rate': 6.803088803088803e-06,
'epoch': 6.6}
{'loss': 2.8102, 'grad_norm': 1.1328125, 'learning_rate': 6.725868725868726e-06,
'epoch': 6.64}
{'loss': 2.8396, 'grad_norm': 1.15625, 'learning_rate': 6.648648648648649e-06,
'epoch': 6.68}
{'loss': 2.8496, 'grad_norm': 1.046875, 'learning_rate': 6.571428571428572e-06,
'epoch': 6.72}
{'loss': 2.8444, 'grad_norm': 1.0546875, 'learning_rate': 6.494208494208495e-06,
'epoch': 6.76}
{'loss': 2.8394, 'grad_norm': 1.125, 'learning_rate': 6.416988416988418e-06,
'epoch': 6.8}
{'loss': 2.8376, 'grad_norm': 1.09375, 'learning_rate': 6.3397683397683405e-06,
'epoch': 6.83}
{'loss': 2.839, 'grad_norm': 1.09375, 'learning_rate': 6.262548262548263e-06,
'epoch': 6.87}
{'loss': 2.8328, 'grad_norm': 1.140625, 'learning_rate': 6.185328185328185e-06,
'epoch': 6.91}
{'loss': 2.8467, 'grad_norm': 1.0703125, 'learning_rate': 6.108108108108109e-06,
'epoch': 6.95}

{'loss': 2.8715, 'grad_norm': 1.125, 'learning_rate': 6.030888030888031e-06,
'epoch': 6.99}
{'loss': 2.8196, 'grad_norm': 1.15625, 'learning_rate': 5.9536679536679545e-06,
'epoch': 7.03}
{'loss': 2.8391, 'grad_norm': 1.125, 'learning_rate': 5.876447876447877e-06,
'epoch': 7.07}
{'loss': 2.834, 'grad_norm': 1.1484375, 'learning_rate': 5.7992277992278e-06,
'epoch': 7.1}
{'loss': 2.8366, 'grad_norm': 1.1171875, 'learning_rate': 5.722007722007722e-06,
'epoch': 7.14}
{'loss': 2.8609, 'grad_norm': 1.234375, 'learning_rate': 5.644787644787645e-06,
'epoch': 7.18}
{'loss': 2.8361, 'grad_norm': 1.1015625, 'learning_rate': 5.567567567567568e-06,
'epoch': 7.22}
{'loss': 2.8188, 'grad_norm': 1.0859375, 'learning_rate': 5.490347490347491e-06,
'epoch': 7.26}
{'loss': 2.8554, 'grad_norm': 1.15625, 'learning_rate': 5.413127413127413e-06,
'epoch': 7.3}
{'loss': 2.8369, 'grad_norm': 1.09375, 'learning_rate': 5.335907335907337e-06,
'epoch': 7.34}
{'loss': 2.8295, 'grad_norm': 1.1171875, 'learning_rate': 5.258687258687259e-06,
'epoch': 7.37}
{'loss': 2.853, 'grad_norm': 1.1171875, 'learning_rate': 5.1814671814671824e-06,
'epoch': 7.41}
{'loss': 2.8527, 'grad_norm': 1.1328125, 'learning_rate': 5.104247104247104e-06,
'epoch': 7.45}
{'loss': 2.8224, 'grad_norm': 1.125, 'learning_rate': 5.027027027027027e-06,
'epoch': 7.49}
{'loss': 2.8104, 'grad_norm': 1.125, 'learning_rate': 4.94980694980695e-06,
'epoch': 7.53}
{'loss': 2.8241, 'grad_norm': 1.1171875, 'learning_rate': 4.872586872586873e-06,
'epoch': 7.57}
{'loss': 2.8755, 'grad_norm': 1.171875, 'learning_rate': 4.7953667953667956e-06,
'epoch': 7.61}
{'loss': 2.8516, 'grad_norm': 1.09375, 'learning_rate': 4.718146718146718e-06,
'epoch': 7.64}
{'loss': 2.84, 'grad_norm': 1.140625, 'learning_rate': 4.640926640926641e-06,
'epoch': 7.68}
{'loss': 2.856, 'grad_norm': 1.125, 'learning_rate': 4.563706563706564e-06,
'epoch': 7.72}
{'loss': 2.8478, 'grad_norm': 1.1171875, 'learning_rate': 4.486486486486487e-06,
'epoch': 7.76}
{'loss': 2.8678, 'grad_norm': 1.25, 'learning_rate': 4.4092664092664095e-06,
'epoch': 7.8}
{'loss': 2.8642, 'grad_norm': 1.078125, 'learning_rate': 4.332046332046332e-06,
'epoch': 7.84}
{'loss': 2.8347, 'grad_norm': 1.1640625, 'learning_rate': 4.254826254826255e-06,
'epoch': 7.88}

{'loss': 2.8429, 'grad_norm': 1.078125, 'learning_rate': 4.177606177606178e-06, 'epoch': 7.92}
{'loss': 2.8249, 'grad_norm': 1.1328125, 'learning_rate': 4.100386100386101e-06, 'epoch': 7.95}
{'loss': 2.8539, 'grad_norm': 1.1328125, 'learning_rate': 4.0231660231660235e-06, 'epoch': 7.99}
{'loss': 2.8477, 'grad_norm': 1.1328125, 'learning_rate': 3.945945945945947e-06, 'epoch': 8.03}
{'loss': 2.8168, 'grad_norm': 1.0625, 'learning_rate': 3.868725868725869e-06, 'epoch': 8.07}
{'loss': 2.842, 'grad_norm': 1.1171875, 'learning_rate': 3.7915057915057917e-06, 'epoch': 8.11}
{'loss': 2.8364, 'grad_norm': 1.046875, 'learning_rate': 3.7142857142857146e-06, 'epoch': 8.15}
{'loss': 2.8388, 'grad_norm': 1.0859375, 'learning_rate': 3.637065637065637e-06, 'epoch': 8.19}
{'loss': 2.8346, 'grad_norm': 1.140625, 'learning_rate': 3.55984555984556e-06, 'epoch': 8.22}
{'loss': 2.8259, 'grad_norm': 1.140625, 'learning_rate': 3.482625482625483e-06, 'epoch': 8.26}
{'loss': 2.8272, 'grad_norm': 1.1640625, 'learning_rate': 3.4054054054054057e-06, 'epoch': 8.3}
{'loss': 2.8205, 'grad_norm': 1.078125, 'learning_rate': 3.328185328185328e-06, 'epoch': 8.34}
{'loss': 2.8512, 'grad_norm': 1.078125, 'learning_rate': 3.250965250965251e-06, 'epoch': 8.38}
{'loss': 2.8622, 'grad_norm': 1.109375, 'learning_rate': 3.173745173745174e-06, 'epoch': 8.42}
{'loss': 2.8838, 'grad_norm': 1.0859375, 'learning_rate': 3.0965250965250972e-06, 'epoch': 8.46}
{'loss': 2.8285, 'grad_norm': 1.171875, 'learning_rate': 3.0193050193050193e-06, 'epoch': 8.49}
{'loss': 2.8297, 'grad_norm': 1.1640625, 'learning_rate': 2.942084942084942e-06, 'epoch': 8.53}
{'loss': 2.8296, 'grad_norm': 1.140625, 'learning_rate': 2.8648648648648654e-06, 'epoch': 8.57}
{'loss': 2.8437, 'grad_norm': 1.1015625, 'learning_rate': 2.7876447876447875e-06, 'epoch': 8.61}
{'loss': 2.8461, 'grad_norm': 1.15625, 'learning_rate': 2.7104247104247108e-06, 'epoch': 8.65}
{'loss': 2.8516, 'grad_norm': 1.1484375, 'learning_rate': 2.6332046332046336e-06, 'epoch': 8.69}
{'loss': 2.8672, 'grad_norm': 1.21875, 'learning_rate': 2.5559845559845565e-06, 'epoch': 8.73}
{'loss': 2.8286, 'grad_norm': 1.1171875, 'learning_rate': 2.478764478764479e-06, 'epoch': 8.76}
{'loss': 2.8464, 'grad_norm': 1.1484375, 'learning_rate': 2.401544401544402e-06, 'epoch': 8.8}

{'loss': 2.8677, 'grad_norm': 1.1796875, 'learning_rate':
2.3243243243243247e-06, 'epoch': 8.84}
{'loss': 2.8344, 'grad_norm': 1.078125, 'learning_rate': 2.247104247104247e-06,
'epoch': 8.88}
{'loss': 2.8395, 'grad_norm': 1.125, 'learning_rate': 2.16988416988417e-06,
'epoch': 8.92}
{'loss': 2.8323, 'grad_norm': 1.1640625, 'learning_rate': 2.092664092664093e-06,
'epoch': 8.96}
{'loss': 2.859, 'grad_norm': 1.0703125, 'learning_rate': 2.0154440154440154e-06,
'epoch': 9.0}
{'loss': 2.8183, 'grad_norm': 1.15625, 'learning_rate': 1.9382239382239383e-06,
'epoch': 9.03}
{'loss': 2.8566, 'grad_norm': 1.1015625, 'learning_rate':
1.8610038610038612e-06, 'epoch': 9.07}
{'loss': 2.8653, 'grad_norm': 1.1484375, 'learning_rate': 1.783783783783784e-06,
'epoch': 9.11}
{'loss': 2.8564, 'grad_norm': 1.1796875, 'learning_rate':
1.7065637065637067e-06, 'epoch': 9.15}
{'loss': 2.8332, 'grad_norm': 1.1328125, 'learning_rate':
1.6293436293436296e-06, 'epoch': 9.19}
{'loss': 2.8124, 'grad_norm': 1.1953125, 'learning_rate':
1.5521235521235523e-06, 'epoch': 9.23}
{'loss': 2.8583, 'grad_norm': 1.2109375, 'learning_rate': 1.474903474903475e-06,
'epoch': 9.27}
{'loss': 2.8399, 'grad_norm': 1.140625, 'learning_rate': 1.3976833976833978e-06,
'epoch': 9.31}
{'loss': 2.832, 'grad_norm': 1.1015625, 'learning_rate': 1.3204633204633205e-06,
'epoch': 9.34}
{'loss': 2.8333, 'grad_norm': 1.1015625, 'learning_rate':
1.2432432432432434e-06, 'epoch': 9.38}
{'loss': 2.8276, 'grad_norm': 1.1328125, 'learning_rate': 1.166023166023166e-06,
'epoch': 9.42}
{'loss': 2.8188, 'grad_norm': 1.171875, 'learning_rate': 1.088803088803089e-06,
'epoch': 9.46}
{'loss': 2.8528, 'grad_norm': 1.15625, 'learning_rate': 1.0115830115830116e-06,
'epoch': 9.5}
{'loss': 2.8535, 'grad_norm': 1.171875, 'learning_rate': 9.343629343629345e-07,
'epoch': 9.54}
{'loss': 2.8415, 'grad_norm': 1.1171875, 'learning_rate': 8.571428571428572e-07,
'epoch': 9.58}
{'loss': 2.8507, 'grad_norm': 1.1171875, 'learning_rate': 7.7992277992278e-07,
'epoch': 9.61}
{'loss': 2.8407, 'grad_norm': 1.109375, 'learning_rate': 7.027027027027028e-07,
'epoch': 9.65}
{'loss': 2.8595, 'grad_norm': 1.1171875, 'learning_rate': 6.254826254826256e-07,
'epoch': 9.69}
{'loss': 2.8843, 'grad_norm': 1.140625, 'learning_rate': 5.482625482625483e-07,
'epoch': 9.73}

```
{'loss': 2.8227, 'grad_norm': 1.1484375, 'learning_rate': 4.710424710424711e-07,
'epoch': 9.77}
{'loss': 2.8415, 'grad_norm': 1.6875, 'learning_rate': 3.938223938223939e-07,
'epoch': 9.81}
{'loss': 2.8483, 'grad_norm': 1.1015625, 'learning_rate': 3.166023166023166e-07,
'epoch': 9.85}
{'loss': 2.847, 'grad_norm': 1.078125, 'learning_rate': 2.393822393822394e-07,
'epoch': 9.88}
{'loss': 2.8402, 'grad_norm': 1.125, 'learning_rate': 1.6216216216216218e-07,
'epoch': 9.92}
{'loss': 2.8303, 'grad_norm': 1.140625, 'learning_rate': 8.494208494208495e-08,
'epoch': 9.96}
{'loss': 2.8378, 'grad_norm': 1.078125, 'learning_rate': 7.722007722007723e-09,
'epoch': 10.0}
{'train_runtime': 1377.8893, 'train_samples_per_second': 15.037,
'train_steps_per_second': 1.88, 'train_loss': 2.8588300646041813, 'epoch': 10.0}
100%|                      | 2590/2590 [22:57<00:00,  1.88it/s]
Model saved to ./myanmar_xglm_word_optimize

real    23m37.363s
user    20m9.350s
sys     3m40.633s
```

0.39 Evaluation with Original XGLM-564M LM

Finetuning က တကယ် အလုပ်လုပ်ရဲ့လား ဆိုတာကို သိနိုင်ဖို့ Meta ရဲ့ အော်ရဂျင်နယ် XGLM-564M language model ကို တိုက်ရိုက် evaluation လုပ်ကြည့်ကြရအောင်။ 10K (စာကြောင်း တစ်သောင်း) test ဖိုင်နဲ့ပဲလုပ်ကြည့်မယ်။

[168]:
```
!cat ./eval-base.sh
```

```
#!/bin/bash
# Evaluate the PRE-TRAINED base model WITHOUT fine-tuning

time python transformer_lm.py --mode test \
                  --test_file ../data/10k_test.txt.clean \
                  --model_path facebook/xglm-564M \
                  --base_model facebook/xglm-564M \
                  --seq_len 512 \
                  --stride 256
```

[169]:
```
!./eval-base.sh
```

```
2026-05-02 00:00:11.337484: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-02 00:00:11.360414: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
```

```
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1777654811.386470 3560106 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1777654811.395229 3560106 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1777654811.417614 3560106 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777654811.417676 3560106 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777654811.417688 3560106 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777654811.417692 3560106 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2026-05-02 00:00:11.423878: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
Using device: cuda
Loading fine-tuned model from: facebook/xglm-564M
Tokenizing test set (10000 lines)…
Evaluating with Stride=256, Context=512…
Passing a tuple of `past_key_values` is deprecated and will be removed in
Transformers v4.58.0. You should pass an instance of `EncoderDecoderCache`
instead, e.g.
`past_key_values=EncoderDecoderCache.from_legacy_cache(past_key_values)`.

Test Results -> PPL: 21.72, Avg NLL: 3.0780 nats

real    0m48.044s
user    0m49.806s
sys     0m4.483s
```

0.40 Evaluation with Finetune Optimized Model

နောက်ဆုံး epoch 10 ပြီးတော့ –seq_len 512 နဲ့ဆောက်ထားတဲ့ မော်ဒယ်ကို သုံးပြီး ဘယ်လောက်ထိ finetuning model က တိုးတက်မှုရှိသလဲ ဆိုတာကို လေ့လာကြည့်ကြရအောင်။

ပြင်ထားတဲ့ shell script က အောက်ပါအတိုင်းပါ။

[172]:
```
!cat ./eval-optimize-10k.sh
```

```
#!/bin/bash
# Note the --stride 64. This gives the model context from previous sentences
# when calculating PPL, leading to a much fairer (lower) score than stride=128.

time python transformer_lm.py --mode test \
                  --test_file ../data/10k_test.txt.clean \
                  --model_path ./myanmar_xglm_word_optimize \
                  --seq_len 128 \
                  --stride 64
```

[173]:
```
!./eval-optimize-10k.sh
```

```
2026-05-02 00:09:48.310978: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-02 00:09:48.325226: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1777655388.342309 3560950 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1777655388.347894 3560950 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1777655388.361877 3560950 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777655388.361912 3560950 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777655388.361915 3560950 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777655388.361919 3560950 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2026-05-02 00:09:48.365670: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
Using device: cuda
```

```
Loading fine-tuned model from: ./myanmar_xglm_word_optimize
Tokenizing test set (10000 lines)…
Evaluating with Stride=64, Context=128…
Passing a tuple of `past_key_values` is deprecated and will be removed in
Transformers v4.58.0. You should pass an instance of `EncoderDecoderCache`
instead, e.g.
`past_key_values=EncoderDecoderCache.from_legacy_cache(past_key_values)`.

Test Results -> PPL: 19.52, Avg NLL: 2.9716 nats

real    1m21.259s
user    1m22.233s
sys     0m1.770s
```

ဒီတစ်ခါတော့ 1K test data နဲ့ evaluation လုပ်ကြည့်မယ်။

[170]:
```
!cat ./eval-optimize.sh
```

```
#!/bin/bash
# Note the --stride 64. This gives the model context from previous sentences
# when calculating PPL, leading to a much fairer (lower) score than stride=128.

time python transformer_lm.py --mode test \
                  --test_file ../data/otest.word.clean \
                  --model_path ./myanmar_xglm_word_optimize \
                  --seq_len 128 \
                  --stride 64
```

[171]:
```
!./eval-optimize.sh
```

```
2026-05-02 00:06:13.966489: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-02 00:06:13.979884: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1777655173.995708 3560682 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1777655174.000734 3560682 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1777655174.013827 3560682 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
```

```
W0000 00:00:1777655174.013864 3560682 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777655174.013867 3560682 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777655174.013869 3560682 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2026-05-02 00:06:14.017281: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
Using device: cuda
Loading fine-tuned model from: ./myanmar_xglm_word_optimize
Tokenizing test set (1000 lines)…
Evaluating with Stride=64, Context=128…
Passing a tuple of `past_key_values` is deprecated and will be removed in
Transformers v4.58.0. You should pass an instance of `EncoderDecoderCache`
instead, e.g.
`past_key_values=EncoderDecoderCache.from_legacy_cache(past_key_values)`.

Test Results -> PPL: 19.49, Avg NLL: 2.9697 nats

real    0m13.989s
user    0m15.485s
sys     0m1.307s
```

Finetuning လုပ်တာက အော်ရဂျင်နယ်မော်ဒယ်ကို တိုက်ရိုက်သုံးတာထက် **PPL** ကျသွားတယ်။ ဆိုလိုတာက ရလဒ်ကောင်းတယ်ဆိုတာကိုတော့ သေချာပေါက် ပြောနိုင်ပါတယ်။ သို့သော်လည်း နှိုင်းယှဉ်ကြည့်ရင် တွေ့ရတဲ့အတိုင်းပဲ epoch 3 ခုနဲ့ ရလဒ်က epoch 10 မော်ဒယ်ထက် ပိုကောင်းနေတယ်ဆိုတာကို တွေ့ရတယ်။ လက်တွေ့မှာက PPL ရလဒ်ကိုပဲ ကြည့်ပြီးတော့လည်း ပြောရတာက ခက်ပါတယ်။ အထူးသဖြင့် အခုလိုမျိုး အရမ်း အပြတ်အသတ် ကွာမနေတဲ့ အခါမှာ။ ဘာပဲဖြစ်ဖြစ် လက်တွေ့အမျိုးမျိုး စမ်းကြည့်၊ အကဲဖြတ်ကြည့်မှပဲ အတိအကျ ပြောနိုင်ပါလိမ့်မယ်။ ဥပမာ PPL ရော text generation ရဲ့ quality ရော ကိုပါ နှိုင်းယှဉ်ကြည့်တာမျိုး။

0.41 Text Generation with Optimized XGLM Model

[174]:
```
!cat ./gen-optimize.sh
```

```
#!/bin/bash
# Added --top_k and --top_p (Nucleus sampling) - standard practices for modern
LLMs

python transformer_lm.py --mode generate \
                  --model_path ./myanmar_xglm_word_optimize \
                  --prompt "ပြည်ထောင်စု" \
```

```
                    --gen_length 200 \
                    --temperature 0.7 \
                    --top_k 50 \
                    --top_p 0.95
```

[175]:
```
!./gen-optimize.sh
```

```
2026-05-02 00:19:48.887081: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-02 00:19:48.901696: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1777655988.918871 3561361 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1777655988.924405 3561361 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1777655988.938847 3561361 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777655988.938892 3561361 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777655988.938902 3561361 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777655988.938905 3561361 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2026-05-02 00:19:48.942582: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
Using device: cuda
Loading fine-tuned model from: ./myanmar_xglm_word_optimize

Prompt: ပြည်ထောင်စု
Generating:
ပြည်ထောင်စုသမ္မတမြန်မာနိုင်ငံတော် ဖွဲ့စည်းပုံအခြေခံဥပဒေ အတည်ပြုရန်
ကြိုးပမ်းဆောင်ရွက်နေသည့် အစိုးရ အဖွဲ့၏ အဓိက ရည်ရွယ်ချက် တစ်ခုဖြစ်သော
```

ဖွဲ့စည်းပုံအခြေခံဥပဒေ အတည်ပြုရေးအတွက် အစိုးရအဖွဲ့၏ သဘောထား သဘောထား ကို ရှင်းလင်းတင်ပြရန် ဖြစ်ပါသည်။

[176]:
```
!./gen-optimize.sh
```

```
2026-05-02 00:20:28.675238: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-02 00:20:28.689541: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1777656028.706473 3561450 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1777656028.711986 3561450 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1777656028.725956 3561450 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777656028.725991 3561450 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777656028.725994 3561450 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777656028.725996 3561450 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2026-05-02 00:20:28.729662: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
Using device: cuda
Loading fine-tuned model from: ./myanmar_xglm_word_optimize

Prompt: ပြည်ထောင်စု
Generating:
ပြည်ထောင်စု ဖွဲ့စည်းပုံ အခြေခံ ဥပဒေ နှင့် ပြည်ထောင်စု ဖွဲ့စည်းပုံ အခြေခံ ဥပဒေ
နှင့် စည်းမျဉ်းစည်းကမ်း များ ( ဖွဲ့စည်းပုံ အခြေခံ ဥပဒေ နှင့် စည်းမျဉ်းစည်းကမ်း
များ ( ဖွဲ့စည်းပုံ အခြေခံ ဥပဒေ နှင့် စည်းမျဉ်းစည်းကမ်း များ ( ဖွဲ့စည်းပုံ အခြေခံ
ဥပဒေ နှင့် စည်းမျဉ်းစည်းကမ်း များ ( ဖွဲ့စည်းပုံ အခြေခံ ဥပဒေ နှင့်
```

စည်းမျဉ်းစည်းကမ်း များ (ဖွဲ့စည်းပုံ အခြေခံ ဥပဒေ နှင့် စည်းမျဉ်းစည်းကမ်း များ (ဥပဒေကြမ်းအမှတ် (၅)) နှင့် အမိန့်ကြော်ငြာစာအမှတ် (၁၂) တို့နှင့် သက်ဆိုင်သော ဥပဒေကြမ်းများ)

[177]:
```
!./gen-optimize.sh
```

```
2026-05-02 00:21:08.868092: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-02 00:21:08.881482: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1777656068.897375 3561553 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1777656068.902452 3561553 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1777656068.915788 3561553 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777656068.915822 3561553 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777656068.915824 3561553 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1777656068.915827 3561553 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
2026-05-02 00:21:08.919280: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
Using device: cuda
Loading fine-tuned model from: ./myanmar_xglm_word_optimize

Prompt: ပြည်ထောင်စု
Generating:
ပြည်ထောင်စု အချုပ်အခြာ အာဏာ
```

0.42 Model Folder Structure

မော်ဒယ်တွေကို သိမ်းထားတဲ့ ဖိုလ်ဒါကိုလည်း လေ့လာနိုင်အောင် ဒီ Notebook မှာ ရိုက်ပြပေးထားလိုက်ပါမယ်။

[179]:
```
!tree ./myanmar_xglm_word/
```

```
./myanmar_xglm_word/
  checkpoint-1044
      config.json
      generation_config.json
      model.safetensors
      optimizer.pt
      rng_state.pth
      scheduler.pt
      sentencepiece.bpe.model
      special_tokens_map.json
      tokenizer_config.json
      tokenizer.json
      trainer_state.json
      training_args.bin
  checkpoint-1566
      config.json
      generation_config.json
      model.safetensors
      optimizer.pt
      rng_state.pth
      scheduler.pt
      sentencepiece.bpe.model
      special_tokens_map.json
      tokenizer_config.json
      tokenizer.json
      trainer_state.json
      training_args.bin
  checkpoint-522
      config.json
      generation_config.json
      model.safetensors
      optimizer.pt
      rng_state.pth
      scheduler.pt
      sentencepiece.bpe.model
      special_tokens_map.json
      tokenizer_config.json
      tokenizer.json
      trainer_state.json
      training_args.bin
  config.json
  generation_config.json
  model.safetensors
```

```
    sentencepiece.bpe.model
    special_tokens_map.json
    tokenizer_config.json
    tokenizer.json
    training_args.bin

4 directories, 44 files
```

[180]:
```
!tree ./myanmar_xglm_word_optimize/
```

```
./myanmar_xglm_word_optimize/
    checkpoint-1036
        config.json
        generation_config.json
        model.safetensors
        optimizer.pt
        rng_state.pth
        scheduler.pt
        sentencepiece.bpe.model
        special_tokens_map.json
        tokenizer_config.json
        tokenizer.json
        trainer_state.json
        training_args.bin
    checkpoint-1295
        config.json
        generation_config.json
        model.safetensors
        optimizer.pt
        rng_state.pth
        scheduler.pt
        sentencepiece.bpe.model
        special_tokens_map.json
        tokenizer_config.json
        tokenizer.json
        trainer_state.json
        training_args.bin
    checkpoint-1554
        config.json
        generation_config.json
        model.safetensors
        optimizer.pt
        rng_state.pth
        scheduler.pt
        sentencepiece.bpe.model
        special_tokens_map.json
        tokenizer_config.json
        tokenizer.json
```

```
    trainer_state.json
    training_args.bin
checkpoint-1813
    config.json
    generation_config.json
    model.safetensors
    optimizer.pt
    rng_state.pth
    scheduler.pt
    sentencepiece.bpe.model
    special_tokens_map.json
    tokenizer_config.json
    tokenizer.json
    trainer_state.json
    training_args.bin
checkpoint-2072
    config.json
    generation_config.json
    model.safetensors
    optimizer.pt
    rng_state.pth
    scheduler.pt
    sentencepiece.bpe.model
    special_tokens_map.json
    tokenizer_config.json
    tokenizer.json
    trainer_state.json
    training_args.bin
checkpoint-2331
    config.json
    generation_config.json
    model.safetensors
    optimizer.pt
    rng_state.pth
    scheduler.pt
    sentencepiece.bpe.model
    special_tokens_map.json
    tokenizer_config.json
    tokenizer.json
    trainer_state.json
    training_args.bin
checkpoint-259
    config.json
    generation_config.json
    model.safetensors
    optimizer.pt
    rng_state.pth
    scheduler.pt
```

```
    sentencepiece.bpe.model
    special_tokens_map.json
    tokenizer_config.json
    tokenizer.json
    trainer_state.json
    training_args.bin
checkpoint-2590
    config.json
    generation_config.json
    model.safetensors
    optimizer.pt
    rng_state.pth
    scheduler.pt
    sentencepiece.bpe.model
    special_tokens_map.json
    tokenizer_config.json
    tokenizer.json
    trainer_state.json
    training_args.bin
checkpoint-518
    config.json
    generation_config.json
    model.safetensors
    optimizer.pt
    rng_state.pth
    scheduler.pt
    sentencepiece.bpe.model
    special_tokens_map.json
    tokenizer_config.json
    tokenizer.json
    trainer_state.json
    training_args.bin
checkpoint-777
    config.json
    generation_config.json
    model.safetensors
    optimizer.pt
    rng_state.pth
    scheduler.pt
    sentencepiece.bpe.model
    special_tokens_map.json
    tokenizer_config.json
    tokenizer.json
    trainer_state.json
    training_args.bin
config.json
generation_config.json
model.safetensors
```

```
  sentencepiece.bpe.model
  special_tokens_map.json
  tokenizer_config.json
  tokenizer.json
  training_args.bin

11 directories, 128 files
```

0.43 RAG with a Modern 8B-Class Model (Quantized on GPU)

24GB VRAM means we don't have to settle for GPT-2 for RAG. We can load Gemma-2-2b-it (or even Llama-3-8B) in 4-bit quantization. It uses ~5GB VRAM, leaving plenty of room. We use sentence-transformers on GPU for blazing-fast retrieval.

ပထမတော့ အထက်မှာ ရေးထားတဲ့အတိုင်းပဲ quantization လုပ်ဖို့ ကြိုးစားသေးတယ်။ float32 ကို bfloat16 ပြောင်းသုံးဖို့ကိစ္စပါ။

#!pip install -q bitsandbytes –break-system-packages

တစ်ခုရှိတာက ဒီ bitsandbytes library က complex C++/CUDA library ဖြစ်တာကြောင့် ဆရာ့စက်မှာ အဆင်မပြေဘူး။ ဆရာ့စက်မှာ VRAM က Gemma-2-2b-it ကို load လုပ်လို့ ရပါတယ်။ အဲဒါကြောင့် bitsandbytes ကို မသုံးပဲသွားမယ်။

ကျောင်းသားတွေက ကိုယ့်စက်ရဲ့ VRAM ပေါ်မူတည်ပြီး လိုအပ်ရင် bitsandbytes library ကို သုံးပြီး quantization လုပ်ပါ။

ပြီးတော့ sentence-transformers library က ကိုယ့်စက်ထဲမှာ မရှိရင်လည်း installation လုပ်ရလိမ့်မယ်။

0.44 Why RAG?

The Problem: LLMs တွေက အတိအကျမှတ်ထားတာမျိုး မဟုတ်ဘူး။ တနည်းအားဖြင့်က မေ့တတ်တယ်။ အဲဒါကြောင့် ငါတို့က LLM တစ်ခု (Gemma) ကို ငါတို့ မေလ ၁ရက်နေ့က trained ခဲ့တာကို မှတ်မိသေးလား။ အဲဒီတုန်းက LSTM ရဲ့ PPL က ဘယ်လောက်ရခဲ့တာလဲ လို့ မေးရင် သူမဖြေပေးနိုင်ပါဘူး။ ဘာဖြစ်လို့လဲ ဆိုတော့ LLM မော်ဒယ်တွေက အင်တာနက်က ရှိသမျှ စာသားတွေ (i.e. terabytes of internet text) ကို အကုန် ချုံ့သိမ်းထားတာမျိုးပါ။ လက်တွေ့မှာက weight တန်ဖိုးတွေအနေနဲ့ပဲ သူသိတာပါ။ သူက တိတိကျကျ မှတ်သားထားတဲ့ memory လိုမျိုး မရှိပါဘူး။ သူက pattern တွေအနေနဲ့ပဲ လေ့လာထားပြီး အဲဒီကနေမှာ ဖြစ်နိုင်တဲ့ စာသားတွေကို ခန့်မှန်းပြီး ကျပန်း ထုတ်ပေးနေတာပါ။ အဲဒါကြောင့် သူ့ရဲ့ အဖြေတွေက ရှိုးတာမျိုး၊ မှန်သလိုလို ဖြစ်နေတာမျိုးနဲ့ မှားနေတာတွေ အများကြီး ဖြစ်နိုင်ပါတယ်။

The RAG Solution: အဲဒီ ပြဿနာကို သုတေသနပညာရှင်တွေက RAG ဆိုတဲ့ နည်းလမ်းနဲ့ ပြေလည်နိုင်မလား ဆိုပြီး ကြိုးစားကြည့်နေကြတာပါ။ ကိုယ့် ဒိုမိန်းနဲ့ ဆိုင်တဲ့ memory ကို ထပ်ဖြည့်တဲ့ ပုံစံမျိုးပါပဲ။ RAG ရဲ့ အရှည်က Retrieval-Augmented Generation လို့ခေါ်ဆိုပြီးတော့ AI ကို Retriever နဲ့ Generator ဆိုပြီး နှစ်ပိုင်းခွဲလိုက်တဲ့ ပုံစံမျိုးပါ။

The Retriever (SentenceTransformer): ဒါက ကိုယ့်ဒိုမိန်းနဲ့ ဆိုင်တဲ့ အချက်အလက် အတိအကျမှတ်သားထားတဲ့ မှတ်ဉာဏ် (i.e. memory) ပါပဲ။ LLM ကို မရမက သင်ပေး၊ မှတ်ခိုင်းနေတာထက် ကိုယ်ကသုံးမယ့် ဒေတာဗေစ့် ကို vector database အနေနဲ့ ကြိုတင် ပြင်ဆင်ထားတဲ့ သဘောပါ။ Python code ထဲမှာ သုံးထားတဲ့ kb_embs လိုမျိုးပါ။ ယူဇာက မေးခွန်းမေးလိုက်တိုင်းမှာ Retriever က အဲဒီကြိုပြင်ထားတဲ့

vector database ထဲကနေ Dot Product/Cosine Similarity တို့နဲ့ ရှာဖွေပြီး အဖြစ်နိုင်ဆုံး (သို့) သက်ဆိုင်မှုအများဆုံး စာပိုဒ်တွေကို ဆွဲထုတ် ယူလာပါတယ်။

The Generator (Gemma-2): Generator ဆိုတာကတော့ "Reasoning Engine" ပါပဲ။ အကျိုးသင့်၊ အကြောင်းသင့် logically ကျ၊ မကျ ကို ခန့်မှန်းပေးတဲ့ အင်ဂျင်ပါပဲ။ Retriever ကဆွဲထုတ်ပေးတဲ့ စာပိုဒ်တွေကို text prompt ထဲမှာပါ ဖြည့်စွက်ပြီးမှ၊ ခုပြင်ပေးထားတဲ့ စာပိုဒ်တွေကိုဖတ်ကြည့်ပြီး အဖြေပေးပါ ဆိုတဲ့ ပုံစံမျိုးနဲ့ LLM ကို ခိုင်းတဲ့ သဘောပါပဲ။

RAG ကို သုံးခြင်းအားဖြင့် အားသာချက်ကတော့ text ဖိုင် သို့မဟုတ် databaseကို ပြင်တာ၊ ဖြည့်စွက်တာ လုပ်ပြီး knowledge ကို တိုးသွားလို့ ရပါတယ်။ LLM ကို ပြန် train လုပ်တာမျိုး၊ finetuning လုပ်တာမျိုး လုပ်ဖို့ မလိုတဲ့အတွက် ကုန်ကျစရိတ် သက်သာစေတဲ့ အပိုင်းပါပဲ။ အဲဒါကြောင့် ကုမ္ပဏီတွေမှာ RAG ကို အသုံးချနေကြတာတွေ အများကြီးရှိပါတယ်။

[94]:
```
from sentence_transformers import SentenceTransformer
import numpy as np
import textwrap
import torch

# 1. GPU-Accelerated Embeddings
print("Loading Retriever...")
encoder = SentenceTransformer("all-MiniLM-L6-v2", device=device)

# Our tiny mock Knowledge Base (Vector DB)
kb = [
    "Hugging Face Transformers is a library for state-of-the-art NLP.",
    "Retrieval-Augmented Generation helps reduce hallucinations by grounding␣
↪LLMs on retrieved documents.",
    "Perplexity is the exponentiated average negative log-likelihood.",
    "Cross-entropy between true distribution p and model q is H(p,q) = -Σ p(x)␣
↪log q(x).",
]
kb_embs = encoder.encode(kb, convert_to_numpy=True, normalize_embeddings=True)

def retrieve_top_k(query: str, k=2):
    q_emb = encoder.encode([query], convert_to_numpy=True,␣
↪normalize_embeddings=True)[0]
    # Dot product similarity (because vectors are L2-normalized, this equals␣
↪Cosine Similarity)
    scores = kb_embs @ q_emb
    top_idx = np.argsort(scores)[::-1][:k]
    return [(kb[i], float(scores[i])) for i in top_idx]

# 2. Load Generator (Natively in bfloat16 - No bitsandbytes needed!)
print("Loading Generator (bfloat16)...")
from transformers import AutoTokenizer, AutoModelForCausalLM

# Using a modern instruction-tuned model
```

```python
gen_model_id = "google/gemma-2-2b-it"
tokenizer_gen = AutoTokenizer.from_pretrained(gen_model_id)
# We just use torch_dtype=torch.bfloat16. Simple, fast, stable.
model_gen = AutoModelForCausalLM.from_pretrained(
    gen_model_id,
    torch_dtype=torch.bfloat16,
    device_map="auto" # Automatically puts it on your RTX 3090
).to(device)

# 3. RAG Pipeline
def rag_answer(query: str):
    # Step A: Retrieve relevant documents based on semantic similarity
    hits = retrieve_top_k(query, k=2)
    context = "\n".join([f"- {txt}" for txt, _ in hits])

    # Step B: Construct a prompt forcing the LLM to use ONLY the context
    prompt = textwrap.dedent(f"""\
    Answer the question based only on the provided context.

    Context:
    {context}

    Question: {query}

    Answer:""").strip()

    inputs = tokenizer_gen(prompt, return_tensors="pt").to(device)

    # Step C: Generate the answer
    with torch.no_grad():
        outputs = model_gen.generate(**inputs, max_new_tokens=100,␣
 ↪temperature=0.1)

    answer = tokenizer_gen.decode(outputs[0], skip_special_tokens=True)
    return answer.split("Answer:")[-1].strip()

print("\n--- RAG Inference ---")
print("Q: How is RAG useful?")
print("A:", rag_answer("How is RAG useful?"))
print("\nQ: What is the math behind Perplexity?")
print("A:", rag_answer("What is the math behind Perplexity?"))
```

```
Loading Retriever…

/home/ye/.local/lib/python3.12/site-packages/torch/nn/modules/module.py:1784:
FutureWarning: `encoder_attention_mask` is deprecated and will be removed in
version 4.55.0 for `BertSdpaSelfAttention.forward`.
  return forward_call(*args, **kwargs)
```

```
Loading Generator (bfloat16)…

Loading checkpoint shards: 100%|                          | 2/2
[00:01<00:00,  1.09it/s]
The following generation flags are not valid and may be ignored:
['temperature']. Set `TRANSFORMERS_VERBOSITY=info` for more details.


--- RAG Inference ---
Q: How is RAG useful?

/home/ye/.local/lib/python3.12/site-packages/torch/nn/modules/module.py:1784:
FutureWarning: `encoder_attention_mask` is deprecated and will be removed in
version 4.55.0 for `BertSdpaSelfAttention.forward`.
  return forward_call(*args, **kwargs)
The following generation flags are not valid and may be ignored:
['temperature']. Set `TRANSFORMERS_VERBOSITY=info` for more details.

A: RAG helps reduce hallucinations by grounding LLMs on retrieved documents.


Q: What is the math behind Perplexity?
A: Perplexity is the exponentiated average negative log-likelihood.
```

0.45 RAG for Myanmar Language

ဒီတခါတော့ မြန်မာစာနဲ့ RAG ကို ဆောက်ပြပြီး မေးခွန်းမေးကြည့်ကြရအောင်။

BBC article link: https://www.bbc.com/burmese/articles/cjr9r8veve0o

[1]:
```
%cd /mnt/disk1/ye/exp/LM-Tutorial/transformer
```

```
/mnt/disk1/ye/exp/LM-Tutorial/transformer
```

[11]:
```
import textwrap
import torch
import numpy as np
from sentence_transformers import SentenceTransformer
from transformers import AutoTokenizer, AutoModelForCausalLM

# ==========================================
# 1. THE KNOWLEDGE BASE (The BBC Article)
# ==========================================
# In a real app, this would be thousands of documents chunked and stored in a␣
↪Vector DB.
bbc_paragraphs = [
    "မြန်မာကျပ် မလို တရုတ် ယွမ် သာ လိုတဲ့ ရှမ်းပြည်က နယ်စပ်မြို့တွေ တရုတ်-မြန်မာ ␣
↪နယ်စပ် မြို့တွေ မှာ မြန်မာ ကျပ်ငွေ ထက် တရုတ် ငွေကြေး ယွမ်ကို ဦးစားပေး သုံးတာ က ␣
↪အရင်ကတည်းက ရှိခဲ့ပြီး မဆန်းလှပါဘူး။",
```

```
    "၂၀၂၇ စစ်ဆင်ရေးအလွန်မှာတော့ ရှမ်းပြည်နယ် (မြောက်ပိုင်း) မှာ ယွမ်တန်ဖိုးက ဘယ်တုန်းကမှ␣
↪နဲ့မတူအောင် မြင့်တက် လာပြီး ကျပ်ငွေ က နောက်ကောက်ကျန်ရစ်ခဲ့ ပါတယ်။ လိုချင်သူ ␣
↪ရှားလာ ပါတယ်။",
    "၀ တပ်ဖွဲ့ (UWSA) နဲ့ ကိုးကန့်တပ် (MNDAA) ထိန်းချုပ်ထားတဲ့ မြို့တွေမှာ ကျပ်ငွေအစား ␣
↪ယွမ်ငွေကိုပဲ ဘာလို့ သုံးလာကြတာလဲ။ ရှမ်းမြောက်က သိန္နီ၊ ကွမ်လုံ၊ လောက်ကိုင်နဲ့ ␣
↪ချင်းရွှေဟော်မြို့တွေမှာ အရင်ကထက် တရုတ်ယွမ်ငွေသုံးစွဲတာ ပိုပြီးတွင်ကျယ်လာတယ်လို့ ␣
↪အဲဒီနေရာတွေမှာ နေထိုင်နေကြတဲ့ ဒေသခံတချို့က ဘီဘီစီကိုပြောပါတယ်။",
    "အဲဒီမြို့တွေကို ၂၀၂၃ ခုနှစ်၊ အောက်တိုဘာ ၂၇ ရက်က စတင်ခဲ့တဲ့ မြောက်ပိုင်းညီနောင် ␣
↪သုံးဖွဲ့ရဲ့ ၁၀၂၇ စစ်ဆင်ရေး ပထမပိုင်းမှာပဲ ကိုးကန့်တပ်က ထိန်းချုပ်လုက်တာဖြစ်ပါတယ်။",
    "သိန္နီဒေသခံတစ်ဦးကတော့ မြို့ထဲက ကုန်ခြောက်ဆိုင်၊ သစ်သီးဆိုင်၊ စားသောက်ဆိုင်နဲ့ ␣
↪စျေးတွေမှာ ကျပ်ငွေနဲ့ ပေးချေလို့ရပေမဲ့ ကျပ်ငွေသုံးမယ်ဆိုရင် ကုိင်ရတဲ့ ပိုက်ဆံအရွက်ရေက ␣
↪ပိုများတာကြောင့် ယွမ်ငွေကို ပိုပြီး သုံးနေရတယ်လို့ ဆိုပါတယ်။"
]

# ==========================================
# 2. THE MULTILINGUAL RETRIEVER
# ==========================================
print("Loading Multilingual Retriever...")
# We MUST use a multilingual model here. English-only MiniLM will fail on␣
↪Myanmar.
encoder = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2",␣
↪device=device)

print("Embedding Knowledge Base...")
kb_embs = encoder.encode(bbc_paragraphs, convert_to_numpy=True,␣
↪normalize_embeddings=True)

def retrieve_top_k(query: str, k=2):
    q_emb = encoder.encode([query], convert_to_numpy=True,␣
↪normalize_embeddings=True)[0]
    scores = kb_embs @ q_emb
    top_idx = np.argsort(scores)[::-1][:k]
    return [bbc_paragraphs[i] for i in top_idx]

# ==========================================
# 3. THE GENERATOR (Gemma 2)
# ==========================================
#print("Loading Generator...")
#gen_model_id = "google/gemma-2-2b-it"
#tokenizer_gen = AutoTokenizer.from_pretrained(gen_model_id)
#model_gen = AutoModelForCausalLM.from_pretrained(
#    gen_model_id,
#    torch_dtype=torch.bfloat16,
#    device_map="auto"
#).to(device)
```

```
# ==========================================
# 3. THE GENERATOR (Gemma 2)
# ==========================================
print("Loading Generator...")
gen_model_id = "google/gemma-2-2b-it"
tokenizer_gen = AutoTokenizer.from_pretrained(gen_model_id)

# THE SILVER BULLET FIX:
# 1. device_map="auto" safely handles VRAM and prevents OOMs.
# 2. attn_implementation="eager" disables PyTorch's torch.compile,
#    permanently preventing the FailOnRecompileLimitHit error in Jupyter!
model_gen = AutoModelForCausalLM.from_pretrained(
    gen_model_id,
    torch_dtype=torch.bfloat16,
    device_map="auto",
    attn_implementation="eager"
)

# ==========================================
# 4. RAG PIPELINE (Myanmar Prompt)
# ==========================================
def rag_answer_myanmar(query: str):
    # Step A: Retrieve context in Myanmar
    retrieved_docs = retrieve_top_k(query, k=2)
    context_string = "\n".join([f"- {doc}" for doc in retrieved_docs])

    # Step B: Create a Myanmar prompt instructing the LLM to read and extract
    # We explicitly tell it to rely ONLY on the text to prevent hallucinations
    prompt = f""" အောက်ပါ စာသားအချက်အလက်များကို ဖော်ပြထားပါသည်။ ␣
↪ထိုစာသားများအပေါ် အခြေခံပြီး မေးခွန်းကို မြန်မာလို ဖြေပေးပါ။

စာသားအချက်အလက်များ:
{context_string}

မေးခွန်း: {query}

အဖြေ: """

    inputs = tokenizer_gen(prompt, return_tensors="pt").to(device)

    # Step C: Generate
    with torch.no_grad():
        outputs = model_gen.generate(
            **inputs,
            max_new_tokens=150,
```

```
            pad_token_id=tokenizer_gen.eos_token_id,
            do_sample=False # Greedy decoding for factual extraction
        )

    # Extract just the generated part
    answer = tokenizer_gen.decode(outputs[0], skip_special_tokens=True)
    return answer.split("အဖြေ:")[-1].strip()

# ==========================================
# 5. RUN THE DEMO
# ==========================================
print("\n" + "="*50)
print("--- MYANMAR RAG INFERENCE ---")
print("="*50)

q1 = "ဘာကြောင့် ယွမ်ငွေ သုံးသလဲ"
print(f"\n မေးခွန်း ၁: {q1}")
print(f"အဖြေ: {rag_answer_myanmar(q1)}\n")

q2 = "ရှမ်းမြောက် ဘယ်မြို့တွေ မှာလဲ"
print(f"\n မေးခွန်း ၂: {q2}")
print(f"အဖြေ: {rag_answer_myanmar(q2)}")
```

```
Loading Multilingual Retriever…
Embedding Knowledge Base…
Loading Generator…

Loading checkpoint shards: 100%|                        | 2/2
[00:01<00:00,  1.04it/s]


==================================================
--- MYANMAR RAG INFERENCE ---
==================================================

 မေးခွန်း ၁: ဘာကြောင့် ယွမ်ငွေ သုံးသလဲ
အဖြေ: ကျပ်ငွေ သုံးမယ်ဆိုရင် ကုိင်ရတဲ့ ပိုက်ဆံအရွက်ရေက ပိုများတာကြောင့်
ယွမ်ငွေကို ပိုပြီး သုံးနေရတယ်လို့ ဆိုပါတယ်။

ကျပ်ငွေ သုံးမယ်ဆိုရင် ကုိင်ရတဲ့ ပိုက်ဆံအရွက်ရေက ပိုများတာကြောင့် ယ


 မေးခွန်း ၂: ရှမ်းမြောက် ဘယ်မြို့တွေ မှာလဲ

/home/ye/.local/lib/python3.12/site-packages/torch/nn/modules/module.py:1784:
FutureWarning: `encoder_attention_mask` is deprecated and will be removed in
version 4.55.0 for `BertSdpaSelfAttention.forward`.
```

```
    return forward_call(*args, **kwargs)
```

အဖြေ: ** ရှမ်းမြောက် ဘယ်မြို့တွေ မှာလဲ**

** အဲဒီမြို့တွေကို ၂၀၂၃ ခုနှစ်၊ အောက်တိုဘာ ၂၇ ရက်က စတင်ခဲ့တဲ့ မြောက်ပိုင်းညီနောင် သုံးဖွဲ့ရဲ့ ၁၀၂၇ စစ်ဆင်ရေး ပထမပိုင်းမှာပဲ ကိုးကန့်တပ်က ထိန်းချုပ်လုက်တာဖြ

1 Q&A RAG

ဒီတစ်ခါတော့ paragraph ပုံစံမျိုးနဲ့ RAG ကို ဆောက်တာမဟုတ်ပဲ။ အမေး၊ အဖြေ ပုံစံမျိုးနဲ့ ဆောက်ကြည့်ကြမယ်။

[10]:
```
import numpy as np

# ==========================================
# 1. STRUCTURED QA KNOWLEDGE BASE
# ==========================================
# Storing as a list of dictionaries allows us to separate the Question from the␣
↪Answer
qa_database = [
    {"q": "မေလ ၂ ရက်နေ့ အတန်းမှာ ဘာ သင် မှာလဲ", "a": "မြန်မာစာ နဲ့ ထိုင်းစာ ကို ␣
↪ကွန်ပျူတာ သုံးပြီး ဘာသာပြန် မယ်"},
    {"q": "မေလ ၁ ရက်နေ့ က ထိုင်းမှာ ရုံးပိတ်လား", "a": "ပိတ်တယ်"},
    {"q": "ဆရာ က စက်ဝိုင်း နဲ့ တြိဂံ ဘယ်ဟာ ကို ပိုကြိုက် သလဲ", "a": "စက်ဝိုင်းကို ပို␣
↪ကြိုက်တယ်"},
    {"q": "ဆရာ က ဘယ်လမှာ မွေးတာလဲ", "a": "ဧပြီ လ မှာ"},
    {"q": "မလှ က ကျောင်းပြေးဖူးလား", "a": "သုံးခါပဲ ပြေးဖူးပါတယ်"}
]

# ==========================================
# 2. EMBEDDING THE QA PAIRS
# ==========================================
print("Embedding QA Database...")
# We embed "Q: ... A: ..." together so the Retriever learns the mapping between␣
↪them
qa_texts = [f"Q: {item['q']} A: {item['a']}" for item in qa_database]
qa_embs = encoder.encode(qa_texts, convert_to_numpy=True,␣
↪normalize_embeddings=True)

def retrieve_exact_qa(query: str):
    q_emb = encoder.encode([query], convert_to_numpy=True,␣
↪normalize_embeddings=True)[0]
    scores = qa_embs @ q_emb

    # Get the index of the highest scoring match
```

```
    best_idx = np.argmax(scores)
    best_score = scores[best_idx]

    # Return the dictionary and the similarity score (useful for debugging)
    return qa_database[best_idx], float(best_score)

# ==========================================
# 3. STRICT RAG PIPELINE FOR EXTRACTING ANSWERS
# ==========================================
def rag_exact_answer(query: str):
    # Step A: Retrieve the best matching QA pair
    best_match, score = retrieve_exact_qa(query)

    # Step B: Create a strict prompt
    context = f"Q: {best_match['q']}\nA: {best_match['a']}"

    prompt = f""" ဒါက သိထားရမည့် မေးခွန်းနဲ့ အဖြေ ဖြစ်ပါတယ်။
မေးခွန်းက အောက်ပါ မေးခွန်းနဲ့ ကိုက်ညီလျှင် အဖြေကို အဲဒီအတိုင်း ပြန်လည်ထုတ်ပေးပါ။
မမှန်ကန်လျှင် "မမှန်ပါ" လို့သာ ပြောပေးပါ။

စာသားအချက်အလက် :
{context}

မေးခွန်း: {query}

အဖြေ:"""

    inputs = tokenizer_gen(prompt, return_tensors="pt").to(device)

    # Step C: Generate
    with torch.no_grad():
        outputs = model_gen.generate(
            **inputs,
            max_new_tokens=50,  # Answers are short, we don't need 150 tokens
            pad_token_id=tokenizer_gen.eos_token_id,
            do_sample=False
        )

    answer = tokenizer_gen.decode(outputs[0], skip_special_tokens=True)
    final_answer = answer.split("အဖြေ:")[-1].strip()

    return final_answer, score

# ==========================================
# 4. RUN THE DEMO
```

```python
# ==========================================
print("\n" + "="*50)
print("--- EXACT QA RAG INFERENCE ---")
print("="*50)

# Test 1: Exact Match
q1 = "ဆရာ က ဘယ်လမှာ မွေးတာလဲ"
print(f"\n မေးခွန်း ၁ (အတိအကျကိုက်ညီ): {q1}")
ans1, score1 = rag_exact_answer(q1)
print(f"Retriever Score: {score1:.2f} (1.0 is a perfect match)")
print(f" အဖြေ: {ans1}")

# Test 2: Paraphrased Match (Testing the Retriever's intelligence)
q2 = "စတုဂံ ကို ပိုကြိုက်တယ်"
print(f"\n မေးခွန်း ၂ (အဓိပ္ပာယ် နားလည် မလည်စမ်းတာ): {q2}")
ans2, score2 = rag_exact_answer(q2)
print(f"Retriever Score: {score2:.2f} (Lower score)")
print(f" အဖြေ: {ans2}")

# Test 3: Out of Domain (Testing the strict prompt)
q3 = "ဘာကြောင့် ယွမ် သုံးတာလဲ"
print(f"\n မေးခွန်း ၃ (ဒိုမိန်း မတူတာကိုစမ်းတာ): {q3}")
ans3, score3 = rag_exact_answer(q3)
print(f"Retriever Score: {score3:.2f} (Very low score)")
print(f" အဖြေ: {ans3}")
```

```
Embedding QA Database…

==================================================
--- EXACT QA RAG INFERENCE ---
==================================================

 မေးခွန်း ၁ (အတိအကျကိုက်ညီ): ဆရာ က ဘယ်လမှာ မွေးတာလဲ
Retriever Score: 0.87 (1.0 is a perfect match)
 အဖြေ: ဆရာ က ဘယ်လမှာ မွေးတာလဲ ဧပြီ လ မှာ

** ကျွန်တော် ကျွန်တော်

 မေးခွန်း ၂ (အဓိပ္ပာယ် နားလည် မလည်စမ်းတာ): စတုဂံ ကို ပိုကြိုက်တယ်
Retriever Score: 0.47 (Lower score)
 အဖြေ: စက်ဝိုင်း ကို ပိုကြိုက်တယ်

** ကျွန်တော် ကျွန်တော် ကျွန်တော် ကျွန်တော
```

```
မေးခွန်း ၃ (ဒိုမိန်း မတူတာကိုစမ်းတာ): ဘာကြောင့် ယွမ် သုံးတာလဲ
Retriever Score: 0.29 (Very low score)
```
အဖြေ: ဘာကြောင့် ယွမ် သုံးတာလဲ

** ကျွန်တော် ကျွန်တော် ကျွန်တော် က

1.1 The Evaluation

We cannot directly compare the PPL of KenLM, LSTM, and Transformers.

- KenLM calculates PPL per Word (e.g., predicting 1 out of 10,000 words).
- XGLM calculates PPL per Subword (e.g., breaking "မြန်မာ" into 3 pieces, predicting 1 out of 256,000 pieces).
- Because XGLM makes predictions more frequently (due to smaller pieces), its PPL mathematically looks much higher/worse, even if it's a better model!

The Solution: Bits-Per-Character (BPC). BPC normalizes everything back to raw characters. It asks: "How many bits of information does the model need to guess the next single character?"

KenLM BPC: `(log10 prob of a word * log2(10)) / number of characters in that word`
By comparing BPC, we are finally comparing Apples to Apples.

ရှေ့မှာက KenLM ကို toy ဒေတာနဲ့ပဲ ဆောက်ရသေးတာမို့ KenLM ကိုလည်း myPOS corpus နဲ့ အရင်ဆောက်ပါမယ်။ ပြီးမှ LSTM, Transformer တို့နဲ့အတူ နှိုင်းယှဉ်ဖို့ လုပ်ပါမယ်။ မဟုတ်ရင် KenLM နဲ့ ဆောက်ထားတဲ့ corpus က တအားသေးနေတာမို့ နှိုင်းယှဉ်လို့ အဆင်မပြေပါ။

[96]:
```
%cd /mnt/disk1/ye/exp/LM-Tutorial
```

```
/mnt/disk1/ye/exp/LM-Tutorial
```

[20]:
```
!mkdir kenlm
```

[22]:
```
# -o 5: 5-gram model
# -S 16G: Use 16GB of RAM (speeds up disk sorting)
# -T: Use your fast NVMe drive for temp files
!lmplz -o 5 -S 16G -T /tmp/kenlm_tmp < ./data/mypos_v3.word.clean > kenlm/
 ↪myanmar_lm.arpa
```

```
=== 1/5 Counting and sorting n-grams ===
Reading /mnt/disk1/ye/exp/LM-Tutorial/data/mypos_v3.word.clean
----5---10---15---20---25---30---35---40---45---50---55---60---65---70---75---80
---85---90---95--100
********************************************************************************
********************
Unigram tokens 510463 types 24719
=== 2/5 Calculating and sorting adjusted counts ===
Chain sizes: 1:296628 2:1676055808 3:3142604800 4:5028167168 5:7332744192
Statistics:
1 24719 D1=0.679643 D2=1.0199 D3+=1.3394
```

```
2 173900 D1=0.763124 D2=1.12194 D3+=1.42456
3 335010 D1=0.867187 D2=1.26549 D3+=1.41956
4 397924 D1=0.932677 D2=1.34069 D3+=1.50121
5 398900 D1=0.948056 D2=1.45671 D3+=1.63352
Memory estimate for binary LM:
type       kB
probing 28893 assuming -p 1.5
probing 34303 assuming -r models -p 1.5
trie    13557 without quantization
trie     7241 assuming -q 8 -b 8 quantization
trie    12256 assuming -a 22 array pointer compression
trie     5940 assuming -a 22 -q 8 -b 8 array pointer compression and
quantization
=== 3/5 Calculating and sorting initial probabilities ===
Chain sizes: 1:296628 2:2782400 3:6700200 4:9550176 5:11169200
----5---10---15---20---25---30---35---40---45---50---55---60---65---70---75---80
---85---90---95--100
##############################################################################
####################
=== 4/5 Calculating and writing order-interpolated probabilities ===
Chain sizes: 1:296628 2:2782400 3:6700200 4:9550176 5:11169200
----5---10---15---20---25---30---35---40---45---50---55---60---65---70---75---80
---85---90---95--100
##############################################################################
####################
=== 5/5 Writing ARPA model ===
----5---10---15---20---25---30---35---40---45---50---55---60---65---70---75---80
---85---90---95--100
******************************************************************************
********************
Name:lmplz      VmPeak:16949208 kB      VmRSS:7120 kB   RSSMax:2998396 kB
user:1.20549    sys:1.08554     CPU:2.29106     real:1.90142
```

1.2 Convert ARPA to Binary

Binary ဖိုင်အဖြစ်ပြောင်းထားလိုက်ရင် Jupyter Notebook ထဲကနေ ခေါ်သုံးရင်လည်း မြန်ပါတယ်။

[23]:
```
!build_binary ./kenlm/myanmar_lm.arpa ./kenlm/myanmar_lm.binary
```

```
Reading ./kenlm/myanmar_lm.arpa
----5---10---15---20---25---30---35---40---45---50---55---60---65---70---75---80
---85---90---95--100
******************************************************************************
********************
SUCCESS
```

[24]:
```
!ls -lh ./kenlm/*
```

```
-rw-rw-r-- 1 ye ye 95M May  2 03:20 ./kenlm/myanmar_lm.arpa
```

```
-rw-r--r-- 1 ye ye 29M May  2 03:22 ./kenlm/myanmar_lm.binary
```

1.3 Prepare KenLM Evaluation Python Code

KenLM မှာက PPL ကို တွက်ဖို့ commandline tool အနေနဲ့ ရေးမထားပါဘူး။ အဲဒါကြောင့် python code တစ်ပုဒ်ရေးလိုက်ပါတယ်။ အောက်ပါအတိုင်းပါ။

[20]:
```
!cat ./kenlm/eval_kenlm.py
```

```
import kenlm, sys, math

model_path = sys.argv[1]
test_path = sys.argv[2]

model = kenlm.LanguageModel(model_path)
sum_log10 = 0.0
n_words = 0

with open(test_path, 'r', encoding='utf-8') as f:
    for line in f:
        line = line.strip()
        if not line: continue
        for prob, length, oov in model.full_scores(line):
            sum_log10 += prob
        n_words += len(line.split())

# Convert log10 to natural log, then calculate PPL
#sum_nats = sum_log10 * math.log(10)
#ppl = math.exp(-sum_nats / n_words) if n_words else float("inf")
#print(f"PPL: {ppl:.2f}")

# Cleaner, unambiguous version
sum_nats = -sum_log10 * math.log(10)
ppl = math.exp(sum_nats / n_words)
entropy = sum_nats / n_words
print(f"PPL: {ppl:.2f}")
print(f"Entropy (nats): {entropy:.4f}")
```

```
huggingface/tokenizers: The current process just got forked, after parallelism
has already been used. Disabling parallelism to avoid deadlocks…
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true |
false)
```

[102]:
```
!python ./kenlm/eval_kenlm.py ./kenlm/myanmar_lm.binary ./data/otest.word.clean
```

```
PPL: 9.10
Entropy (nats): 2.2085
```

[10]:
```
%cd /mnt/disk1/ye/exp/LM-Tutorial/
```

```
/mnt/disk1/ye/exp/LM-Tutorial
```

[19]:
```
import math
import json
import pandas as pd
import torch
from pathlib import Path
import sys

sys.path.append(str(Path.cwd() / "lstm"))
sys.path.append(str(Path.cwd() / "transformer"))

import kenlm
from lstm_lm import TextDataset as LSTM_Dataset, LSTM_LM as LSTM_Model,␣
↪Vocabulary
from transformer_lm import evaluate_ppl as transformer_evaluate_ppl

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

TEST_FILE = "./data/otest.word.clean"

# ==========================================
# 1. EVALUATE KENLM (N-gram)
# ==========================================
print("Evaluating KenLM...")
# CHANGE THIS to your new binary file!
model_kenlm = kenlm.LanguageModel("./kenlm/myanmar_lm.binary")

def get_kenlm_metrics(path):
    sum_log10, n_words, n_chars = 0.0, 0, 0
    with open(path, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            for prob, length, oov in model_kenlm.full_scores(line):
                sum_log10 += prob
                n_words += 1           # counts </s> token too
            n_chars += len(line)

    # sum_log10 is negative → negate once for all positive quantities
    sum_nats   = -sum_log10 * math.log(10)
    total_bits = -sum_log10 * math.log2(10)
```

```
    ppl_kenlm = math.exp(sum_nats / n_words)  if n_words else float("inf")
    ent_kenlm = sum_nats / n_words             if n_words else float("inf")
    bpc_kenlm = total_bits / n_chars           if n_chars else float("inf")

    return ppl_kenlm, ent_kenlm, bpc_kenlm

ppl_kenlm, ent_kenlm, bpc_kenlm = get_kenlm_metrics(TEST_FILE)

# ==========================================
# 2. EVALUATE LSTM (Word-level)
# ==========================================
print("Evaluating LSTM...")
with open("./lstm/myanmar_word_lm.pt.vocab", "r", encoding="utf-8") as f:
    token2idx = json.load(f)
vocab_lstm = Vocabulary()
vocab_lstm.token2idx = token2idx
vocab_lstm.idx2token = {int(v): k for k, v in token2idx.items()}

model_lstm = LSTM_Model(len(vocab_lstm)).to(device)
model_lstm.load_state_dict(torch.load("./lstm/myanmar_word_lm.pt",␣
 ↪map_location=device))
model_lstm.eval()

with open(TEST_FILE, 'r', encoding='utf-8') as f:
    test_tokens = f.read().split()

# The production LSTM uses sequences of length 50
test_ds = LSTM_Dataset(test_tokens, vocab_lstm, seq_len=50)

# Use the exact same loss function used during training
criterion = torch.nn.CrossEntropyLoss()
nll_sum, n_tokens = 0.0, 0

with torch.no_grad():
    for i in range(len(test_ds)):
        x, y = test_ds[i]

        # Add batch dimension: shape (50,) becomes (1, 50)
        x = x.unsqueeze(0).to(device)
        y = y.unsqueeze(0).to(device)

        # Forward pass
        out, _ = model_lstm(x)

        # Calculate loss (identical to training loop in lstm_lm.py)
        # out shape: (1, 50, vocab_size)
        # y shape: (1, 50)
```

```
        loss = criterion(out.view(-1, out.size(-1)), y.view(-1))

        # Accumulate total negative log likelihood
        nll_sum += loss.item() * y.numel()
        n_tokens += y.numel()

# Calculate final metrics
ppl_lstm = math.exp(nll_sum / n_tokens)
ent_lstm = nll_sum / n_tokens

# ==========================================
# 3. EVALUATE TRANSFORMER (XGLM)
# ==========================================
print("Evaluating XGLM...")
class Args:
    pass
args = Args()
args.seq_len = 128
args.stride = 64

from transformers import AutoTokenizer, AutoModelForCausalLM
tokenizer_xglm = AutoTokenizer.from_pretrained("./transformer/
 ↪myanmar_xglm_word")
model_xglm = AutoModelForCausalLM.from_pretrained(
    "./transformer/myanmar_xglm_word",
    torch_dtype=torch.bfloat16,
    device_map="auto",
    attn_implementation="eager"
)

with open(TEST_FILE, 'r', encoding='utf-8') as f:
    test_lines_xglm = [line.strip() for line in f if line.strip()]

ppl_xglm, nll_xglm = transformer_evaluate_ppl(args, model_xglm, tokenizer_xglm,␣
 ↪test_lines_xglm)
ent_xglm = nll_xglm

# ==========================================
# 4. COMPILE THE RESULTS TABLE
# ==========================================
results = [
    {
        "Model": "KenLM (Myanmar 5-gram)",
        "Test PPL": f"{ppl_kenlm:.2f}",
        "Test Entropy (nats)": f"{ent_kenlm:.2f}",
        "Test BPC": f"{bpc_kenlm:.3f}"
    },
```

```
    {
        "Model": "PyTorch LSTM (GPU)",
        "Test PPL": f"{ppl_lstm:.2f}",
        "Test Entropy (nats)": f"{ent_lstm:.2f}",
        "Test BPC": "N/A (Word level)"
    },
    {
        "Model": "XGLM (Transformer, GPU)",
        "Test PPL": f"{ppl_xglm:.2f}",
        "Test Entropy (nats)": f"{ent_xglm:.2f}",
        "Test BPC": "N/A (Subword level)"
    }
]

df = pd.DataFrame(results)
print(df)
```

```
Using device: cuda
Evaluating KenLM…
Evaluating LSTM…
Evaluating XGLM…
Tokenizing test set (1000 lines)…
Evaluating with Stride=64, Context=128…
                     Model Test PPL Test Entropy (nats)             Test BPC
0   KenLM (Myanmar 5-gram)     7.70                2.04                0.586
1       PyTorch LSTM (GPU)     2.12                0.75     N/A (Word level)
2  XGLM (Transformer, GPU)    18.21                2.90  N/A (Subword level)
```

1.4 SRILM with Myanmar Corpus

စာသင်ပြီးသွားတော့ SRILM ကိုလည်း အထက်က ဇယားမှာထည့်ပြီး နှိုင်းယှဉ်ချင်တာနဲ့ SRILM ကိုလည်း my-POS corpus နဲ့ ဆောက်ပြီး အထက်က python code ကိုပဲ update လုပ်ပြီး run ခဲ့တယ်။

[21]:
```
%pwd
```

[21]: '/mnt/disk1/ye/exp/LM-Tutorial'

[30]:
```
import os
# This updates the PATH for the duration of the notebook session
os.environ['PATH'] = "/home/ye/tool/SRILM/bin/i686-m64:" + os.environ['PATH']

# Verify
!echo $PATH
```

```
/home/ye/tool/SRILM/bin/i686-m64:/home/ye/tool/SRILM/bin/i686-
m64:/home/ye/tool/speech_tools/bin:/home/ye/tool/festival/bin:/home/ye/tool/fest
vox/bin:/home/ye/tool/flite/bin:/home/ye/.nvm/versions/node/v20.19.3/bin:/usr/li
b/nvidia-cuda-toolkit/bin:/usr/bin:/bin:/home/ye/tool/marian/build:/home/ye/.loc
al/bin:/home/ye/tool/speech_tools/bin:/home/ye/tool/festival/bin:/home/ye/tool/f
```

```
estvox/bin:/home/ye/tool/flite/bin:/home/ye/.nvm/versions/node/v20.19.3/bin:/hom
e/ye/.cargo/bin:/usr/lib/nvidia-cuda-toolkit/bin:/usr/bin:/bin:/home/ye/tool/mar
ian/build:/home/ye/miniforge3/condabin:/usr/local/sbin:/usr/local/bin:/usr/sbin:
/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/home/ye/.local/bin:/h
ome/ye/tool/kenlm/build/bin/:/home/ye/.local/bin:/home/ye/.local/bin:/home/ye/to
ol/kenlm/build/bin/
```

[31]:
```
# Basic 5-gram with Kneser-Ney smoothing (recommended)
!ngram-count \
    -order 5 \
    -text ./data/mypos_v3.word.clean \
    -lm ./srilm/myanmar_srilm.arpa \
    -kndiscount \
    -interpolate \
    -unk \
    -map-unk "<unk>"
```

1.4.1 Flag explanations:

-kndiscount: Kneser-Ney smoothing (best general-purpose smoothing, comparable to KenLM's default)
-interpolate: interpolates higher and lower order n-grams (standard practice)
-unk / **-map-unk**: handles out-of-vocabulary words

[32]:
```
!ls ./srilm/
```

```
myanmar_srilm.arpa
```

1.5 Convert ARPA to KenLM Binary (Optional but faster loading)

[33]:
```
!build_binary ./srilm/myanmar_srilm.arpa ./srilm/myanmar_srilm.binary
```

```
Reading ./srilm/myanmar_srilm.arpa
----5---10---15---20---25---30---35---40---45---50---55---60---65---70---75---80
---85---90---95--100
*******************************************************************************
********************
SUCCESS
```

[34]:
```
!ls ./srilm
```

```
myanmar_srilm.arpa  myanmar_srilm.binary
```

[37]:
```
!ls -lh ./srilm/*
```

```
-rw-rw-r-- 1 ye ye  14M May  2 20:35 ./srilm/myanmar_srilm.arpa
-rw-r--r-- 1 ye ye 7.0M May  2 20:38 ./srilm/myanmar_srilm.binary
```

[38]:
```
# Quick PPL check using SRILM's own tool
!ngram \
```

```
    -order 5 \
    -lm ./srilm/myanmar_srilm.arpa \
    -ppl ./data/otest.word.clean \
    -unk
```

```
file ./data/otest.word.clean: 1000 sentences, 12204 words, 0 OOVs
0 zeroprobs, logprob= -21133.62 ppl= 39.86085 ppl1= 53.91329
```

1.6 Updated Code

အထက်က ဆောက်ထားတဲ့ SRILM မော်ဒယ်ကိုထပ်ဖြည့်ထားတာပါ။
ပိုမြန်အောင် Binary LM ကိုပဲ သုံးကြရအောင်။

[40]:
```
import math
import json
import pandas as pd
import torch
from pathlib import Path
import sys

sys.path.append(str(Path.cwd() / "lstm"))
sys.path.append(str(Path.cwd() / "transformer"))

import kenlm
from lstm_lm import TextDataset as LSTM_Dataset, LSTM_LM as LSTM_Model,␣
↪Vocabulary
from transformer_lm import evaluate_ppl as transformer_evaluate_ppl

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

TEST_FILE = "./data/otest.word.clean"

# ==========================================
# SHARED METRIC FUNCTION (reused for both
# KenLM and SRILM since both produce ARPA)
# ==========================================
def get_ngram_metrics(path, model):
    """
    Works for any kenlm-loadable model (KenLM binary or SRILM ARPA).
    Returns (ppl, entropy_nats, bpc).
    """
    sum_log10, n_words, n_chars = 0.0, 0, 0
    with open(path, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
```

```
            for prob, length, oov in model.full_scores(line):
                sum_log10 += prob
                n_words += 1        # count </s> token too
            n_chars += len(line)

    sum_nats   = -sum_log10 * math.log(10)
    total_bits = -sum_log10 * math.log2(10)

    ppl = math.exp(sum_nats / n_words)  if n_words else float("inf")
    ent = sum_nats / n_words            if n_words else float("inf")
    bpc = total_bits / n_chars          if n_chars else float("inf")

    return ppl, ent, bpc

# ==========================================
# 1. EVALUATE KENLM (N-gram)
# ==========================================
print("Evaluating KenLM...")
model_kenlm = kenlm.LanguageModel("./kenlm/myanmar_lm.binary")
ppl_kenlm, ent_kenlm, bpc_kenlm = get_ngram_metrics(TEST_FILE, model_kenlm)

# ==========================================
# 2. EVALUATE SRILM (N-gram)
# ==========================================
print("Evaluating SRILM...")
# Load ARPA directly - or use .binary if you ran build_binary
model_srilm = kenlm.LanguageModel("./srilm/myanmar_srilm.binary")
ppl_srilm, ent_srilm, bpc_srilm = get_ngram_metrics(TEST_FILE, model_srilm)

# ==========================================
# 3. EVALUATE LSTM (Word-level)
# ==========================================
print("Evaluating LSTM...")
with open("./lstm/myanmar_word_lm.pt.vocab", "r", encoding="utf-8") as f:
    token2idx = json.load(f)
vocab_lstm = Vocabulary()
vocab_lstm.token2idx = token2idx
vocab_lstm.idx2token = {int(v): k for k, v in token2idx.items()}

model_lstm = LSTM_Model(len(vocab_lstm)).to(device)
model_lstm.load_state_dict(torch.load("./lstm/myanmar_word_lm.pt",␣
 ↪map_location=device))
model_lstm.eval()

with open(TEST_FILE, 'r', encoding='utf-8') as f:
    test_tokens = f.read().split()
```

```
test_ds   = LSTM_Dataset(test_tokens, vocab_lstm, seq_len=50)
criterion = torch.nn.CrossEntropyLoss()
nll_sum, n_tokens = 0.0, 0

with torch.no_grad():
    for i in range(len(test_ds)):
        x, y = test_ds[i]
        x = x.unsqueeze(0).to(device)
        y = y.unsqueeze(0).to(device)
        out, _ = model_lstm(x)
        loss = criterion(out.view(-1, out.size(-1)), y.view(-1))
        nll_sum  += loss.item() * y.numel()
        n_tokens += y.numel()

ppl_lstm = math.exp(nll_sum / n_tokens)
ent_lstm = nll_sum / n_tokens

# ==========================================
# 4. EVALUATE TRANSFORMER (XGLM)
# ==========================================
print("Evaluating XGLM...")
class Args:
    pass
args = Args()
args.seq_len = 128
args.stride  = 64

from transformers import AutoTokenizer, AutoModelForCausalLM
tokenizer_xglm = AutoTokenizer.from_pretrained("./transformer/
 ↪myanmar_xglm_word")
model_xglm = AutoModelForCausalLM.from_pretrained(
    "./transformer/myanmar_xglm_word",
    torch_dtype=torch.bfloat16,
    device_map="auto",
    attn_implementation="eager"
)

with open(TEST_FILE, 'r', encoding='utf-8') as f:
    test_lines_xglm = [line.strip() for line in f if line.strip()]

ppl_xglm, nll_xglm = transformer_evaluate_ppl(args, model_xglm, tokenizer_xglm,␣
 ↪test_lines_xglm)
ent_xglm = nll_xglm

# ==========================================
# 5. COMPILE THE RESULTS TABLE
# ==========================================
```

```python
results = [
    {
        "Model":                    "KenLM (Myanmar 5-gram)",
        "Test PPL":                 f"{ppl_kenlm:.2f}",
        "Test Entropy (nats)":      f"{ent_kenlm:.2f}",
        "Test BPC":                 f"{bpc_kenlm:.3f}"
    },
    {
        "Model":                    "SRILM (Myanmar 5-gram)",
        "Test PPL":                 f"{ppl_srilm:.2f}",
        "Test Entropy (nats)":      f"{ent_srilm:.2f}",
        "Test BPC":                 f"{bpc_srilm:.3f}"
    },
    {
        "Model":                    "PyTorch LSTM (GPU)",
        "Test PPL":                 f"{ppl_lstm:.2f}",
        "Test Entropy (nats)":      f"{ent_lstm:.2f}",
        "Test BPC":                 "N/A (Word level)"
    },
    {
        "Model":                    "XGLM (Transformer, GPU)",
        "Test PPL":                 f"{ppl_xglm:.2f}",
        "Test Entropy (nats)":      f"{ent_xglm:.2f}",
        "Test BPC":                 "N/A (Subword level)"
    }
]

df = pd.DataFrame(results)
print(df)
```

```
Using device: cuda
Evaluating KenLM…
Evaluating SRILM…
Evaluating LSTM…
Evaluating XGLM…
Tokenizing test set (1000 lines)…
Evaluating with Stride=64, Context=128…
                     Model Test PPL Test Entropy (nats)             Test BPC
0   KenLM (Myanmar 5-gram)     7.70                2.04                0.586
1   SRILM (Myanmar 5-gram)    39.86                3.69                1.058
2       PyTorch LSTM (GPU)     2.12                0.75     N/A (Word level)
3  XGLM (Transformer, GPU)    18.21                2.90  N/A (Subword level)
```

1.7 Summary

အခုချိန်မှာတော့ traditional လို့ ပြောရမယ့် Statistical LM ကနေ၊ Neural Network based LM တွေ (LSTM, Transformer) နဲ့ Large Language Model တွေနဲ့တွဲသုံးတဲ့ RAG အထိပါ ဒေတာပြင်တာကနေ မော်ဒယ်ဆောက်တဲ့အပိုင်း၊ ပြီးတော့ ဆောက်ထားတဲ့ မော်ဒယ်တွေကနေ ဘယ်လို text generation လုပ်လို့ ရသလဲ

စသည်ဖြင့် အဆင့်ဆင့် လက်တွေ့လုပ်ပြခဲ့ပါတယ်။ နောက်ဆုံး ဆောက်ထားတာမတူညီကြတဲ့ LM တွေအကြား မှာ Bits-Per-Character (BPC) နဲ့ ပြန်ညှိပြီး evaluation လုပ်တာကိုလည်း သင်ကြားပေးခဲ့ပါတယ်။

ဆရာ ပြောရဲပါတယ်။ LM နဲ့ပတ်သက်ပြီး ဒီလောက်စုံအောင် လက်တွေ့လုပ်ပြထားတဲ့ Notebook က ရှားပါလိမ့်မယ်။ လုပ်ပြထားတဲ့ အဆင့်တွေအားလုံးကို ဒေတာကိုပြောင်းလိုက်ပြီး ကိုယ်တိုင် အစအဆုံး ပြီးအောင် run ကြည့်ရင်းနဲ့ ကြိုးစားလေ့လာသွားကြပါလို့။

1.8 Reference Links

- SRILM Homepage: http://www.speech.sri.com/projects/srilm/
- SRILM GitHub: https://github.com/BitSpeech/SRILM
- SRILM Installation Note: https://github.com/ye-kyaw-thu/error-overflow/blob/master/SRILM-installation-note.md
- IRSTLM: https://github.com/irstlm-team/irstlm
- KenLM: https://github.com/kpu/kenlm
- KenLM Paper: https://kheafield.com/papers/avenue/kenlm.pdf
- Training an n-gram Language Model and Estimating Sentence Probability by Dr. Masato Hagiwara: https://masatohagiwara.net/training-an-n-gram-language-model-and-estimating-sentence-probability.html
- myPOS Corpus Version 3 (ဒီ Tutorial အတွက် သုံးခဲ့တဲ့ monolingual corpus ပါ): https://github.com/ye-kyaw-thu/myPOS

```
[ ]:
```